

VECTOR

The First 10 Years

- APL94 Conference Update 7
- Dyalog/W Version 7 Announced 11
- Smith & Pearson on the
Windows GUI 64
- MacLeod Draws the Line 83
- Kromberg on Client-Server 114
- Allan Gay on Defined Operators 126

... plus

- 16-page Educational Supplement 17



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.10 No.4 April 1994

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Proposed Membership Rates 1994-95

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	50	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

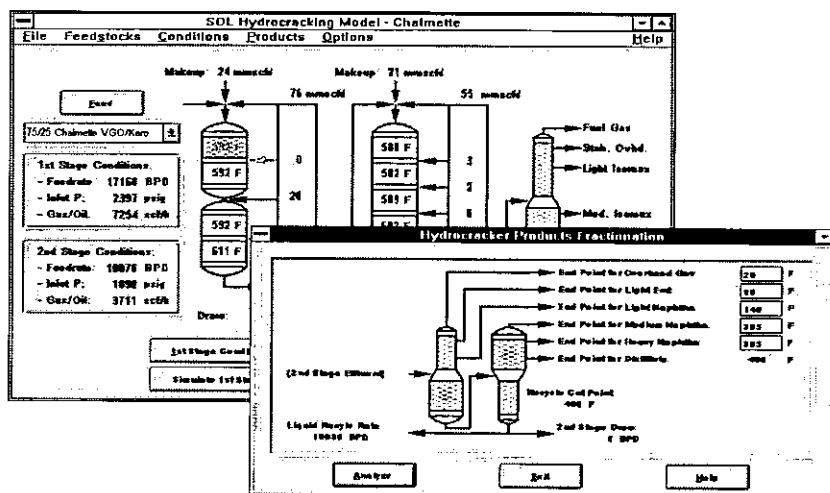
Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gillling East, YORK YO6 4JJ. Tel: 0439-788385.

Contents

		Page
Editorial: The First Ten Years	Anthony Camacho	3
APL NEWS		
Quick Reference Diary		5
Update on APL94	Alain Delmotte	7
British APL Association News:		
Obituary: Mike Gregory	John Searle	10
News from Sustaining Members	Gill Smith	11
The Education Vector	Alan Mayer	17
REVIEWS SECTION		
APL Product Guide: Updates	Gill Smith	33
Calculus by K.E. Iverson	Keith Smillie	39
RECENT MEETINGS		
McIntyre's J Workshop at Greenock	Norman Thomson	44
Visit To APL Centres in Russia November 1993	Gary Littlejohn	44
February Meeting: Eastwood and Cyriax	Adrian Smith	48
GENERAL ARTICLES		
Standardization Beyond the Language	Gfeller and Kromberg	58
A Common Approach to the Windows GUI	Adrian Smith	
(Dyalog APL/W and APL*PLUS III)	and Duncan Pearson	64
What's in a Name?	Sylvia Camacho	78
Drawing the Line	George MacLeod	83
AMORTIZE: a Windows Application in J	Thomas M. Olsen	92
TECHNICAL SECTION		
Technical Editorial: in Search of Contexts	Adrian Smith	106
Hackers' Corner: Flying Windows	Jonathan Barman	108
Finding the 10 to the 10th Prime	Gene McDonnell	110
APL - a Client-Server Language?	Morten Kromberg	114
A Note on Probabilities in the F-Distribution	Keith Smillie	124
Defined Operator Simulation for APL*PLUS II	Allan Gay	126
Inner Products for Range Tests	Jonathan Barman	133
Another Use of Not-Equals-Scan	Anthony Camacho	136
Index to Advertisers		143

dyalog APL

The Definitive APL for Windows



Experience counts

Since we launched Dyalog APL/W at APL92, nearly two years ahead of our nearest competitor, our customers have developed hundreds of successful industrial-strength GUI applications. With the benefit of their experience, we have enhanced and refined Dyalog APL/W into the mature, stable and above all useful Windows development tool that it is today.

Not only does Dyalog APL/W provide the most comprehensive set of GUI facilities available for any APL, but you can be confident that your workspaces will run *unchanged* on Unix workstations under OSF/Motif.

With Version 7.0 we have added support for Visual Basic Custom Controls, Metafiles, MDI, 3-D Forms and Controls, fully customised Session menus, toolbar and status bar, *Namespaces* for encapsulation, and a host of other improvements.

That's why Dyalog APL/W remains the professional choice. For a free trial copy, contact Dyadic or your local distributor today.

Dyadic Systems Limited., Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 7AL, United Kingdom. Tel: (0256) 811125 Fax: (0256) 811130 Email: sales@dyadic.com.

Editorial: The First Ten Years

by Anthony Camacho

With this issue Vector completes ten years since Vol.1 No.1, dated May 1984. We have Robert Bittlestone to thank for persuading us to make the quantum leap from a 20 or 30 page newsletter to an up-market glossy-covered 144-page magazine.

We started with the idea that we would publish on time: on the whole we have. We started with the aim of attracting readers world-wide, not just from the United Kingdom: on the whole we have succeeded, but we believe that many more APLers would benefit from reading it. Vector is not expensive and we are proposing to reduce the price to our overseas members. Please encourage your friends to subscribe.

The next ten years begins with Vector 11.1, to be dated July 1994, which will be a tenth anniversary issue. We will give it to all the delegates to Swansea and to APL94 at Antwerp; it will be special. May I encourage people who haven't contributed to Vector to begin now? In Vector 11.1 we expect to have articles from McDonnell, Spunde, Thomson, Langlet, McIntyre, Smillie, the usual gang and some newcomers. We hope to have a tenth anniversary competition with a really worthwhile prize.

Vector 11.1 will have a complete and updated product guide. Don't forget that the product guide entries are free of charge. Vector 11.1 will have the largest circulation that any Vector has ever had and even one-man bands should get themselves listed in it. Contact Gill Smith at 0439-788 385 with your entry.

Book your advertisements now!

APL in Business at Swansea: 18-22 July 1994

The programme for this conference is ready to be published. It will be the *best training course this year* for APLers who want to integrate their work seamlessly with Windows using DDE, OLE2 and DLLs.

If you attend you may also learn about genetic algorithms, Client Server computing, processing of time series and how to produce publication quality graphics. The mornings will be devoted to plenary sessions; the afternoons will be workshops (in three or more streams) in which the morning plenary speakers

will develop their themes and give you the hands-on experience necessary so that you can apply the techniques when the conference is over.

The evenings can be spent on additional workshops for those that wish, a 'street market' at which books, software and anything you want to bring will be on sale to other delegates, and the conference dinner. Get your name on the list to be sent the details. Contact:

Haydn Williams (Conference Administrator),
EBMS, Swansea University,
Singleton Park,
Swansea SA2 8PP.

Tel: 0792-295555
Fax: 0792-295626

Preparing for Swansea

from Adrian Smith (Workshop Co-ordinator)

APL in Business is intended to be a hard-working 3 days! The objective is simple: to prepare you (and your staff) for the future of APL. In a nutshell, this means learning to program your APL alongside Client-Server Architectures, Graphical User Interfaces, and in co-operation with all the wonderful software that populates the Microsoft universe.

The transition from the traditional APL environments (mainframes or stand-alone DOS machines) will not be an easy one, so get started now! Read Morten Kromberg (page 114) on Client-Server — Morten will be sharing a Wednesday afternoon session with Microsoft — and follow up with Gene McDonnell's lovely tale of Client-Server in action (page 110). Read about a common approach to the Windows GUI (page 64), then spend an afternoon playing with it (choose Dyalog/W or PLUS III) on the Tuesday. Read George MacLeod (page 83) on the tricky details of high-quality graphics, and see the product in action (or maybe even buy it) at the evening street market.

Learn the latest techniques for handling time-series, control charts, actuarial problems, asset management — and then try out your new-found skills in the evenings (the labs will be open as late as you want them to be) with experts on hand to guide you. But remember: *this is not a jolly!* It has taken me over a year to get comfortable with this new world of Windows programming, so start reading now, absorb what you can, and go to Swansea prepared.

Quick Reference Diary 1994

Date	Venue	Event
March 25th	IEE	BAA Meeting and EGM
May 27th	IEE	BAA Annual General Meeting
July 18-22	Swansea	APL in Business
Sept 11-15	Antwerp	APL94
October 28th	IEE	BAA Meeting: Vendor Forum

British APL Association meetings are normally held in the IEE, Savoy Place.
Nearest tube outlets: Temple or Embankment.

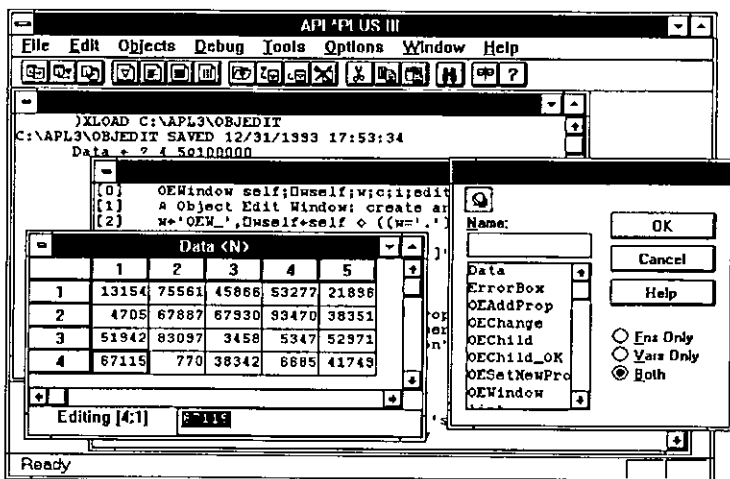
APL Training Courses for 1994

If you would like to have your courses or seminars listed in Vector, please contact Gill Smith with the details.

Dates for Future Issues of VECTOR

	Vol.11 No.1	Vol.11 No.2	Vol.11 No.3
Copy date	3rd June 94	2nd Sept 94	2nd Dec 94
Ad booking	10th June 94	9th Sept 94	9th Dec 94
Ad Copy	17th June 94	16th Sept 94	16th Dec 94
Distribution	July 94	October 94	January 95

Everyone is Raving About APL★PLUS III For Windows



"I have been playing with APL★PLUS® III for a while now and it is a joy to use!"
Morten Kromberg, Insight Systems, Denmark

"The way APL★PLUS III implements control structures is as non-invasive as anything I have yet seen, so please will everyone else copy this. All credit to Manugistics® for taking the plunge and pioneering this." **Adrian Smith, Vector, UK**

"APL★PLUS III lets us develop competitive applications that are gorgeously GUI on the outside and pure APL on the inside." **Eric Baelen, MEWS, Cherry Hill, New Jersey**

"APL★PLUS III is a breakthrough product for APL-based Windows development. From menu controls to DDE integration — all aspects of GUI development are now much easier due to a true Windows look and feel."

Rick Butterworth, APL Consultant, Arlington, Virginia

Don't miss out on the most important APL release ever! Fast, simple, GUI development; free run-time; innovative new control structures; support for DDE and DLL; and much, much more — now available with APL★PLUS III for Windows. To get your copy, contact your local APL★PLUS representative.

Cocking & Drury Ltd., 180 Tottenham Court Road, London W1P 9LE
(071) 436-9481 Fax (071) 436-0524

UPDATE ON APL94

APL94 will be held in Antwerp,
BELGIUM, 11-15 September 1994

APL — the Language and its Applications

The Annual International Conference on APL (co-organised by the Belgian Royal Association of the Flemish Engineers and the ACM/SIGAPL in collaboration with most of the European APL Associations/Societies) is focusing on the applications of APL but does not drop all the work on the development of the language itself.

At the time of writing this update the final programme is not yet fixed. But we can already give some idea of what it will be. We have received papers from 14 countries making the Conference really International indeed. But the distribution of the origin of the papers has greatly changed compared to previous Conferences, with the countries of the CEI being the leaders; the American centre of the APL world is now in balance with the East European one. There is also a modification in the distribution of the subjects. The language itself is still a major subject but with a clear shift towards new concepts like Windows and Object Oriented Programming.

The traditional applications of APL — Economics, Business, Management and Statistics — are also represented. Interestingly we have more articles on Artificial Intelligence, Biology and also Psychology and Geography. Besides the papers there will be posters. The posters — there are more and more posters at APL Conferences — should follow the trends of the papers. The posters are a way of presenting ongoing research work and establishing a scientific priority; they also facilitate cooperative work, on the basis of discussions with the author. *There is still time to present a poster at APL94.* There is no deadline! The posters, abstract and a full text, will be published in one of the leading APL Journals and will possibly be distributed at the Conference.

Other important scientific and technical events will take place during the Conference; remember that APL94 is the annual gathering and showcase of the APL World. There is little to say about the social events, but remember that

O.R. SYSTEMS SPECIALIST

York Based

Attractive + benefits

In the competitive and dynamic market of food manufacture, the scale, complexity and variety of logistical issues – particularly with regard to the supply chain – are immense. No more so than here at Nestlé.

Recognising this, we need to build on our success to date and supplement our efforts with the appointment of a professional to assist with group wide activity.

Operating within the IT function as a member of a specialist group, you will provide on a project basis, those essential OR skills of forecasting, modelling, prototyping and design, to develop and support the systems necessary to improve user productivity.

To succeed, you'll have to be a numerate graduate with 2/3 years' experience, preferably in a decision support environment. The majority of our

OR systems have been developed in APL running on DOS (APL *PLUS II), Windows (Dyalog) and Mainframe (APL *PLUS) platforms and previous experience of support and development in at least one of these is essential.

This is a critical role which will call for excellent communication skills, and in return can offer significant experience and progressive opportunities to match. This is also combined with an attractive rewards package normally associated with a 'blue-chip' employer.

In the first instance please forward a full CV including salary details, to: Carole Knight, Personnel Officer, Nestlé (UK) Ltd, Rowntree Division, Haxby Road, York YO1 1XY.



Antwerp was European Artistic Capital of Europe for 1993, and that was a real success. We are sure you will find great pleasure in the city. Please contact us:

for the Program (poster)

Alain Delmotte
Avenue du Marathon, 6
B1348 Louvain-la-Neuve
BELGIUM
tel: (32)-10-45 11 92
fax: (32)-10-45 23 26

for the Conference itself (invitation)

Joseph DeKerf
Rooienberg, 72
B2570 Duffel
BELGIUM
tel: (32)-15-31 47 24
fax: (32)-3-216 06 89 (attn. J. DeKerf)

e-mail:

APL94PGM@IRES.UCL.AC.BE

TIAPL94@AUTOCTRL.RUG.AC.BE

APL94 POSTERS APL94 POSTERS APL94 POSTERS

The submission of papers for APL94 is now over but you can still present your work to the Conference. If you have ongoing research work but want to share your preliminary results; if you want to establish your priority, and/or if you want to find cooperative work, do not hesitate to present a poster to the APL94 Conference.

Posters were only recently introduced at APL Conferences, but their importance is growing every year. The time allocated to traditional papers is fixed; the time for posters is the whole Conference and there is no limit to the time for discussion.

There is no deadline for the submission of a poster, *but*:

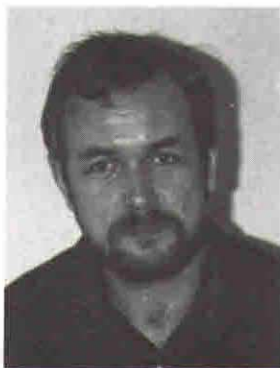
1. If you want your poster to be announced in the final invitation and the Conference program you have to send your proposal and abstract by the 31st of March.
2. The posters will be published in one of the leading APL Journals and — possibly — distributed at the Conference. So we will need your poster (what you want to be published, not only the abstract) by the 31st of May.

You can submit posters to me by mail, fax, or e-mail. I am able to process Word2 and WordPerfect5.2 for Windows — *ISIAPL* and *VectorAPL* TrueType fonts — and Word5 for the Mac — *APL68000* font; I am also starting TeX/LaTeX with the *cmapl* font.

Alain Delmotte
Program Chairman

OBITUARY: Mike Gregory

by John Searle



It is my sad duty to inform you of the passing of one of our members, Mike Gregory. Mike died on December 3rd, 1993, following a recurrence of his battle against melanoma, at the age of 42.

Mike was one of those people who could do anything with computers. His CV reveals that he worked on almost all modern operating systems and, apart from his APL skills (in all dialects) he was proficient in languages ranging from C to REXX, SAS and SQL.

Mike's career began at British Oxygen Corporation (BOC) in 1972 and over 10 years moved from programmer in the Gases Division to Technical Manager of Interactive Services at BOC Datasolve. His projects ranged from the development of APLB for ICL mainframe hardware, to MIS systems, to a simulation of the Grand National horserace for the Sun newspaper.

Mike then helped to form IBSL as Technical Director, a company which serviced various clients in the leisure industry, before forming his own company PC Associates in 1985. The variety of his clients was again considerable. After a brief period working for SimCorp, the Danish software house, he returned to his freelance business, specialising in operations research and airline applications.

I met Mike whilst consulting to a well-known favourite British airline, and he helped me understand the complexity of existing APL code (and the maze of freelancing paperwork!) with great patience and humour. What Mike didn't know wasn't worth knowing, and what he did know would fill a database of textbooks. John Scholes from Dyadic Systems told me that in their client contact system, Mike was always listed as "nice bloke".

Mike's passions in life, away from computers, were Jaguar motorcars, motorbikes and good wines. The joys of classic British motoring adorned his conversation, his bookshelves and his driveway. Mike was also very concerned for the environment, and was a keen supporter of wildlife preservation.

Mike is survived by his wife Judith and their children Bill and Claire. Mike did request that if you wish to remember him, please plant a tree in his name.

News from Sustaining Members

Compiled by Gill Smith

Dyadic Systems Ltd

Dyadic is pleased to announce Version 7 of Dyalog APL/W. This is a major release packed with many new and exciting features.

One of the most important additions is the ability to use `WC`, `WS`, `WG` etc. to access Visual Basic (.VBX) Controls alongside regular Dyalog APL/W ones with exactly the same syntax. This facility gives the Dyalog APL/W user access to hundreds of commercially available Visual Basic add-ons including various Graph, Grid and Spreadsheet controls; even an OLE client control. In addition, users can add controls of their own design written to the .VBX standard using ordinary Microsoft development tools. Access to Visual Basic Controls is implemented using a general mechanism for defining a new Class of object and will be extended to other standards (e.g. OLE 2.0) and user-defined Widgets under OSF/Motif in the future.

Dyadic believes that with the new VBX interface, coupled with the enhanced DDE and Clipboard support and the facility to access any Windows DLL with `MNA`, it alone can truly claim to offer an "open" APL for Windows.

A new `MDICLIENT` object is provided to allow users to develop applications that use the *Multiple Document Interface* (MDI) style. All of the standard MDI behaviour is supported. In addition, `FORM` class objects can now be children of other `FORMS`, and so can be used as general-purpose containers for a wide variety of uses and effects.

Version 7 now provides a variety of `EDIT` field types that perform built-in validation and formatting of numbers, dates etc.

Other new objects include `TOOLBAR`, `STATUSBAR` and `STATUSFIELD`. The `TOOLBAR` is normally displayed at the top of a `FORM` and is used to manage a scrollable set of tool buttons. The `STATUSBAR` is normally displayed at the bottom of a `FORM` and contains one or more `STATUSFIELDS`. These can have pre-defined meanings, such as monitoring the state of the Caps Lock key, in which case they are updated automatically by the interpreter. Alternatively, their contents can be set directly by your application.

All controls can now have associated *TIP* and *HINT* properties. A *TIP* is a (usually) short description which gets displayed as a pop-up message when the mouse pointer dwells over a control, for example over a *BUTTON* in a *TOOLBAR*. A *HINT* is typically a longer help message which gets displayed when an object (e.g. a *MENUITEM*) is selected. A *HINT* would normally be displayed in a *STATUSFIELD*, but you can in fact use any object capable of displaying text.

The *TAB_BAR* manages a scrollable set of *TAB_BTN* objects which allow you to provide a notebook-style interface similar to that used by the latest versions of Excel, Paradox, Lotus 1-2-3 and Organiser.

These new objects have all been designed to allow Dyalog APL/W users to build APL applications that provide the bang up-to-date user-interface features of other popular Windows products; and, because all the work is done for you by the interpreter, with the same snappy performance.

All *FORMS* and Controls now have a *3D* property that can be used to achieve a variety of 3D effects. Furthermore, as the value of this property is by default inherited from the parent, setting *3D* on the *SYSTEM* object '.' automatically gives a 3D appearance throughout your entire application.

Version 7 now provides support for *Windows Metafiles* so that you can incorporate fully scalable graphical pictures into your application. Not only can you import Metafiles, but you can also create them from primitive graphical objects and export them (via file or the Clipboard) to other Windows applications. A final touch is a facility to disassemble an imported Metafile so that you can edit or rearrange its components within APL.

The *APL Session* has not escaped our attention and a *StatusBar* and *ToolBar* have been added. Furthermore, the components of the Session are directly accessible as named objects whose properties can be set by `⌈WS` and read by `⌈WG`. New menus, menuitems, tool buttons, status fields etc. can easily be added using `⌈WC`, and unwanted items can be removed with `⌈EX`. Menuitems and tool buttons can action predefined "system" operations (such as [load] and [edit]) or can execute any APL statement. This allows you to configure the Session exactly as you want and you can dynamically change it (manually or under program control) to suit particular workspaces. For example, your `⌈LX` could add special menuitems and/or tool buttons, fully equipped with *TIPS* and *HINTS*, to run particular functions in a statistics workspace.

The names of your functions, and variables may be displayed in a separate dialog box and are therefore only a click away for editing, copying to the clipboard, etc. A number of improvements have been made to the *Editor* and

Tracer and fully CUA-compliant select/cut/paste operations have also been implemented.

Finally, Dyadic has introduced *Namespaces*, a major new enhancement to Dyalog APL. A Namespace provides the means to gather together a collection of objects and acts as a separate "context" in which functions (including callback functions) can be run. This means that local functions and variables can be hidden from the rest of the workspace. The implementation of Namespaces, which encapsulate code and data, is a further step towards Object Oriented technology and is intended to provide major benefits for APL developers. Further information on this exciting new feature will be available by the time you read this announcement. Please call Dyadic for details.

To coincide with Version 7, Dyadic is publishing "*Outer Products*" a catalogue of third-party tools and add-ons for Dyalog APL. Some "Outer Product" software, such as the *RAIN* graphics workspace and *CAUSEWAY* GUI design tool from APL ♦ 385, are also included in the Version 7 package.

For details concerning price and availability, please contact Dyadic or your local Dyalog APL supplier.

Insight Systems

On the Product Front, we are concentrating our efforts on the design and implementation of a new version of SQAPL which will support Microsoft's Open Database Connectivity (ODBC) specification. The primary design goal is to provide an interface which is completely compatible with our existing product — so that our customers will be able to take advantage of ODBC drivers to access dBase, Btrieve, ACCESS, Paradox (or even Excel and Comma Separated Variable text files) — plus any remote databases for which they have an ODBC Driver — and still be able to use SequeLink without changing the application. SequeLink is still a preferable solution for many Windows environments, and the only solution under Unix and OS/2 or if an APL Server is required.

An ODBC Driver is now available for SequeLink, so SQAPL users will have the option of using SequeLink "direct" or via the ODBC driver. TechGnosis have also announced an ODBC version of their Open Server. Once we have an ODBC SQAPL Client out the door, we will start work on an ODBC version of the SQAPL Server. With an ODBC SQAPL Server, developers will be able to make APL-Coded Databases or Analytical Tools running under Unix available to any development environment which supports ODBC.

We would also like to intentionally leak an official rumour that our ODBC interface will probably be bundled with an upcoming version of Manugistics' APL*PLUS III product. Due to the relatively long lead time for Vector, we have agreed to leak the news despite the fact that the agreement is still under negotiation. By the time you read this, you should be able to contact Morten Kromberg at Insight Systems or Paul Clements at Manugistics to hear whether we succeeded!

On the APL Consulting and Education Front, the Danish market is suddenly full of renewed interest in Graphical APL systems. It seems that many organizations who still use APL on a Mainframe are finally ready to "Downsize" — to the extent that APL courses have become a profitable activity! There was a waiting list for our first "Introduktion til Windows Programmering med Dyalog APL/W", so we will be scheduling several courses during 1994 and will also be "porting" the course to APL*PLUS III.

We would like to take the opportunity to announce that Insight Systems is now an IBM Software Partner, so we now market and support IBM APL2 and APL2-based products (like AGSS) under OS/2 and Unix. We are also authorised RS/6000 resellers, and thus capable of delivering complete "turnkey" systems based on the RS/6000 and any one of APL2/6000, APL*PLUS II/UNIX and Dyalog APL/X. We look forward to cooperating with IBM Denmark in providing APL and Client/Server expertise to their APL2 customers!

Manugistics Inc

Manugistics Inc is very pleased to announce the commercial release of APL*PLUS III for Windows. An Advance Release (version 1.0) began shipping on February 25, and the general availability release (version 1.1) will be ready by the time this issue of Vector appears. Please contact Cocking & Drury in the UK, your local APL*PLUS dealer, or Manugistics for specific information.

APL*PLUS III is a 32-bit Windows system that runs on Windows 3.1, Windows for Workgroups, and Windows NT. It is a completely Windows-hosted program, with a graphical development environment that has been designed to be a pleasure to use, whether for personal APL computing or for building Windows applications to be used by others.

APL*PLUS III includes a graphical development environment, event-driven user command processor, a built in graphical debugger, and a visual form editor that allows the programmer to build a graphical user interface by drawing with the mouse. The graphical user interface facilities are supported by fast, built-in

system functions, and include regular and 3-dimensional versions of forms, frames, buttons, checkboxes, scrollbars, etc. Graphics support is also built in to the user interface, as is support for Dynamic Data Exchange. System functions provide access to Windows libraries and any other software packaged in a Windows DLL.

The design of APL*PLUS III reflects our views of how APL needs to evolve to become a competitive alternative to other languages in this decade and beyond. The GUI development facility follows a strictly object-oriented structure, using "classes", "events", "methods", and "properties". The APL interpreter is based on the well-proven technology of APL*PLUS II, but the heart of the system has been replaced with an event-driven architecture. Control structures, in the spirit of C or Pascal, provide an alternative to the "spaghetti code" that afflicts too many APL programs.

APL*PLUS III will continue to evolve rapidly both in terms of new tools and in enhancements to the language itself. We welcome the comments and suggestions of Vector readers.

MicroAPL Ltd

In the previous issue of Vector, MicroAPL announced APL Level II for the new PowerPC Macintosh computer. APL Level II was one of the first dozen applications to be seen running native on the new PowerPC Macintosh. To coincide with the launch of this important new computer, we have been upgrading our entire family of Macintosh APLs. The details of the upgrade are being finalised as this is written, but we have a fairly clear idea of what the upgrade will include.

The APL programming environment has been enhanced, combining ideas from our APL.68000/X for the RISC System/6000 and features found in Mac System 7 as well as improving our conformance to Apple's System 7 user-interface guidelines. We have added support for Apple Events — multiple workspaces may be launched simultaneously at startup. New Print menu items allow prints of workspace listings as well as contents of the APL window. The new Windows menu identifies multiple APL sessions by workspace name and we have added `)IN` and `)OUT` menu items for workspace exchange. The number of simultaneous APL tasks supported has increased as has the number of edit windows and user-defined windows. Improved user-preferences allow control over such items as default workspace sizes, window positions and sizes and default libraries.

APL programmers can now easily access hierarchical menus and popup menus and we have further improved event handling facilities with new event types. Amongst the new support workspaces is the PLOTMAC workspace for high-level business graphics. The interpreters themselves have been upgraded in line with our latest versions for the RISC System/6000 and PowerPC and a number of small bugs in the Level II interpreters have been fixed.

MicroAPL was instrumental in forcing the APL world into issuing free runtime licences and we have now included with our Mac APLs a packager to make stand-alone applications which may be distributed royalty-free.

Pricing details for the upgrades will be announced in April but we can confirm that we will be offering substantial incentives to upgrade to our Level II versions of APL as well as maintaining our policy of realistic pricing across the board.

Soliton Associates Limited

Version 21 of SHARP APL for VMS, shipping in 2nd quarter 1994, offers improved performance and operability, in particular of the SHARP APL File System. For SHARP APL MVS applications requiring 24-hour 7-day availability, Version 21 offers new functionality supporting continuous on-line operation of databases stored in the SHARP APL File System.

Version 4.03 of SHARP APL for UNIX, shipping in 2nd quarter 1994, provides a powerful new Intrinsic Function facility, allowing secure and efficient access from SHARP APL for UNIX to software implemented in other languages.

Version 4.04 of SHARP APL for UNIX, shipping in 3rd quarter 1994, will support Sun's Solaris 2 operating system, allowing major multi-user SHARP APL applications to execute on large Symmetrical Multi-Processor Sun Unix platforms.

NSVP-API, an application programming interface for Soliton's Network Shared Variable Processor technology, will provide access from non-SHARP APL software to local or remote Network Shared Variable Processors, allowing, for example, non-SHARP APL client software to access local or remote SHARP APL server software, including SHARP APL for MVS Auxiliary Processors. Initially, NSVP-API will be available under UNIX and Microsoft Windows, and will support access to Network Shared Variable Processors operating under UNIX and MVS.

THE EDUCATION VECTOR

April 1994

Editor Alan Mayer

This Education Vector has been reprinted from VECTOR Vol.10 No.4. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Rd, Redland, BRISTOL, BS6 6LS Tel: 0272-730036.

Contents

Editorial	Alan Mayer	18
J: A First Lesson	Donald B. McIntyre	18
A Note on Primes	Ted Emms	30

Dr. Alan Mayer
European Business Management School
University College of Swansea
Singleton Park
Swansea SA2 8PP Wales, UK

Ysgol Rheolaeth Busnes Ewropeaidd
Parc Singleton,
Abertawe SA2 8PP

Tel: 0792-205678 Ext.4274 Fax: 0792-295626 Email: A.D.MAYER@SWANSEA.AC.UK

Editorial

Welcome to an Education Vector with a difference! The difference is that most of this issue is devoted to a language which is not APL. It will come as no surprise to our readers to learn that the "intruder" is J. Indeed I am delighted to publish the promised article by Donald McIntyre "J: A First Lesson".

I can say without hesitation that everyone who studies this article will find new insights and a new understanding of the language — I certainly did. Of course it helps to have the J interpreter on your computer, and for those looking for a low-cost introductory package we hope soon to have available the interpreter, together with papers by Donald McIntyre, on one disk.

I am also pleased to include in this issue a delightful excursion into the world of Prime Numbers by Ted Emms.

J: A First Lesson

by Donald B. McIntyre

It is not surprising if on your first encounter with J you should think it difficult; this is true of most new things. As A.N. Whitehead said: "Of course nothing is more incomprehensible than a symbolism we do not understand". The Editor of *The Education Vector* says: "I find J very difficult. ... Even Donald McIntyre admitted to coming close to despair before he could write J expressions which did what he wanted". It is true that I met formidable difficulties; but then I was one of J's first guinea pigs (August 1990) and had few examples to follow. Nearly four years later things are different: bugs are eliminated; new features and modified old ones make J easier; and, thanks in great part to *Vector*, the literature on J is very much expanded.

The Editor went on to say that "the articles in *Vector* are imperfect as the early ones were written in early versions of J which have been superseded". That is not wholly true; at J workshops at APL93 I distributed disks bringing all eight of my papers on J up to date through Version 6.2 (available from I-APL Ltd). The script files also execute in Version 7, though not at this time taking advantage of V7's new features. The disk includes other scripts used in my J workshops. Together

they constitute a book-length manual supplementing the publications of Iverson Software, Inc. Because the scripts are extensively annotated, they execute as dynamic tutorials. All necessary changes from the published texts are explained, and my paper on *Amendment* (Vector, Vol.9, No.3) explains the last major changes to J syntax.

No reader of *Education Vector* should think that undue difficulty is any longer a reason for putting off the study of J. Even more than APL, this is an executable mathematical notation of great elegance and power. You will enjoy it.

Let's take a concrete example: the computation of a *similarity table*, or the classification of objects by how close they are in a space spanned by their measured properties. This is the basis of Cluster Analysis and is Herbert Hellerman's first example in his classic texts of 1967 (using Iverson Notation) and 1973 (using executable APL). (Herbert Hellerman, *Digital Computer System Principles*, McGraw-Hill Book Company, New York (1967) p.55 and (1973) p.53.) His data are in a table:

```
1 4 3 1
2 1 3 5
1 1 4 1
3 8 2 4
2 3 3 2
0 2 4 2
```

Each row represents an *item*; e.g., a student, a patient, or some object. The columns represent measured properties; e.g., the results of tests. In this case six items are interpreted as points in 4-dimensional space, and the geometrical distance between two points is taken as a measure of dis-similarity between two items. Following Pythagoras, the square of the distance between two points is obtained by taking the sum of the squares of the differences between corresponding co-ordinates.

In J we begin by entering the data by linking the rows (with ;), which gives a string of six boxes. When these boxes are opened (by >) the result is the required table, which can be thought of as a collective noun. It can be given a name (using =.) for later reference, and it is then displayed simply by entering the name. As you read, enter what follows into J (Version 7). What you should enter will be indented and J's response will be flushed left.

```
1 4 3 1; 2 1 3 5; 1 1 4 1; 3 8 2 4; 2 3 3 2; 0 2 4 2
1 4 3 1 | 2 1 3 5 | 1 1 4 1 | 3 8 2 4 | 2 3 3 2 | 0 2 4 2
hh=. >1 4 3 1;2 1 3 5;1 1 4 1;3 8 2 4;2 3 3 2;0 2 4 2
```

Space does not allow us to give Hellerman's distance functions in both Iverson notation and APL, or mine using APL's Direct Definition, but here are three alternative versions in J. Spaces are for readability only; none is necessary.

```
DSQE=. ' +/"1 *: -"1/~ y.' : ''
dsq=. [: +/"1 [: *: -"1/~
dsqt=. +/"1@*:@(-"1/~)
```

The distance-squared matrix for Hellerman's data is:

```
      dsq hh
0 26 10 30 3 7
26 0 18 52 13 15
10 18 0 66 7 3
30 52 66 0 31 53
3 13 7 31 0 6
7 15 3 53 6 0
```

To those not already familiar with J these expressions will appear strange and perhaps even forbidding. My object is to help dispel the strangeness by showing how to read and write this much J.

We can consider the expression `2+3` to be a sentence in which 2 and 3 are nouns and + is a verb (function) that operates on its noun arguments to give a noun result. Each of the verbs `DSQE`, `dsq`, and `dsqt` operates on the noun `hh` to give the similarity table. Each says: *sum the squares of the differences*. If you can guess that `+/ "1` is *sums*, `*` is *squares*, and `- "1/~` is *differences*, then the symbolism is already less obscure.

`DSQE` is an explicit definition of the distance-squared verb in which `y.` is the explicit place-holder of the argument. The definition for the monadic case (one argument) is in quotes. It is followed by a colon and an empty string. Had a dyadic (two argument) case been needed, its definition would have replaced the empty string and `x.` would have been the place-holder for the left argument.

`dsq` and `dsqt` are tacit definitions, with no explicit reference to arguments.

Before addressing these directly we need to consider some fundamentals. The sentence `2+3` includes two kinds of objects; namely, a dyadic verb (+) and its two noun arguments (2 and 3). Verbs like + apply also to lists and tables; i.e., to collective nouns:

```
      0 1 2 3 + 3 2 1 0
3 3 3 3
```

Atoms (single numbers) are extended for compatibility. Here are examples of the dyads minus, times, divide, power, and the monads double, halve, square, and square root:

```

      0 1 2 3 - 2
-2 -1 0 1
      0 1 2 3 * 2
0 2 4 6
      0 1 2 3 % 2
0 0.5 1 1.5
      0 1 2 3 ^ 2
0 1 4 9
      +: 0 1 2 3
0 2 4 6
      -: 0 1 2 3
0 0.5 1 1.5
      *: 0 1 2 3
0 1 4 9
      %: 0 1 4 9
0 1 2 3

```

J uses ASCII symbols, either alone or as digraphs consisting of an ASCII character followed *immediately* by either a period (.) or a colon (:). Negative numbers are preceded with an underbar, and numbers less than 1 are preceded by zero as well as a decimal point. An adverb modifies the verb to its left; e.g. the passive adverb (~) commutes or interchanges the arguments:

```

      0 1 2 3 ^~ 2
1 2 4 8

```

A monadic verb has a single argument, to its right:

```

      - 0 1 2 3
0 -1 -2 -3

```

The reflexive adverb (~) supplies a copy of the right argument as the verb's left argument:

```

      +~ 0 1 2 3
0 2 4 6

```

The verb `1.` generates lists and tables of integers starting at zero, and the tally is the number of items.

```

      y=. 1.4
      y
0 1 2 3
      #y
4

```

The insert adverb (/) inserts its verb between the items:

```
0+1+2+3
6
+ / 0 1 2 3
6
```

Verbs, like nouns, can be named and displayed:

```
sum=. + /
sum
+ /
sum y
6
(sum % #) y
1.5
```

Because parentheses control the order of execution, the enclosed expression must first be evaluated. It is an entity — a train of three verbs defining a new (in this case, unnamed) verb. How to interpret it is not difficult to guess if you know that the arithmetic mean is defined as the sum divided-by the number of items. Name it for future use:

```
mean=. sum % #
mean y
1.5
```

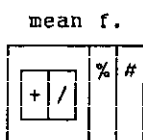
It is essential to note that a very different result is obtained if we enter the expression for immediate execution:

```
sum % # y
0.25
```

The reason is that no new verb is produced. Three monadic verbs are applied in succession and the result is the sum of the reciprocal of the tally of y, which is of no obvious use.

Display the boxed representation, or use the fix adverb (f.) to display the verb in terms of J primitives:

```
mean
sum % #
```



Define the verb tree (its explanation is beyond the scope of this first lesson) and use it for a tree display:

```
tree=. 5!:4@<
tree 'mean'
```

The display shows why trains of 3 elements are called tridents. A train of 3 verbs is a special case, called a fork. If f , g , h are verbs and x and y are nouns, this is how J interprets $(f\ g\ h)$:

```
(f g h) y <--> (f y) g (h y)    Monadic fork
x (f g h) y <--> (x f y) g (x h y) Dyadic fork
```

Note that the root verb (here g) is always dyadic.

J has other ways of building tables:

```
      0 1 2 +/ 0 1 2 3
0 1 2 3
1 2 3 4
2 3 4 5
```

A train of adverbs defines an adverb

```
      +/~ 0 1 2 3
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5 6
      ]y3=. 1.2 3 4
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
16 17 18 19
20 21 22 23
```

Right (J) is a verb that returns its (right) argument. It is used here as a convenient way of displaying a named noun. When working with collective nouns (multi-dimensional arrays) we need verbs that give the shape (\$), the tally or number of *Items* (#), and the rank (the tally of the shape):

```

    $y3
2 3 4
    #y3
2
    #y$
3
    
```

The verb I call *rank* combines two verbs with the atop (@) conjunction:

```

    rank=. #@ $
    rank y3
3
    
```

The verb *head* (t.) gives the first item:

```

    Jy2=. {t. y3
0 1 2 3
4 5 6 7
8 9 10 11
    rank y2
2
    Jy1=. {t. y2
0 1 2 3
    rank y1
1
    
```

Because *sum* and *tally* give the sums over the items and the number of items, the mean of *y3* is a 3 by 4 table of means across the two 3 by 4 tables. If *y3* is a budget report covering 2 years with 3 line-items and 4 quarters, then the means over two years (3 line-items and 4 quarters) are:

```

    mean y3
6 7 8 9
10 11 12 13
14 15 16 17
    
```

Without changing the definition of *mean*, we can use the *rank* conjunction (") to apply *mean* to the rank-2 cells; i.e., over the 3 line-items for each of the 2 years and 4 quarters (down the columns):

```

    mean"2 y3
4 5 6 7
16 17 18 19
    
```

Similarly the means of the rank-1 cells are over the 4 quarters for each of the 2 years and 3 line items (along the rows):

```
      mean"1 y3
1.5  5.5  9.5
13.5 17.5 21.5
```

Use a fork to catenate the means to the report:

```
      f=. ] , mean
      f y3
0  1  2  3
4  5  6  7
8  9 10 11
12 13 14 15
16 17 18 19
20 21 22 23
6  7  8  9
10 11 12 13
14 15 16 17
```

Apply this to the rank-2 cells:

```
      f"2 y3
0  1  2  3
4  5  6  7
8  9 10 11
4  5  6  7
12 13 14 15
16 17 18 19
20 21 22 23
16 17 18 19
```

Apply it to rank-1 cells:

```
      f"1 y3
0  1  2  3  1.5
4  5  6  7  5.5
8  9 10 11  9.5
12 13 14 15 13.5
16 17 18 19 17.5
20 21 22 23 21.5
```

We can reproduce these results by a *hook*, i.e. a train of 2 verbs. A hook modifies a set of data by a function of the data (the monadic case), or it modifies one set of data by a function of another set:

```
      (g h) y <--> y g (h y)
x (g h) y <--> x g (h y)
```

Note that g is always dyadic and h is always monadic. The required hook is:

$h = . , \text{mean}$

Using `match (-:)`, a fork can be used to show that f and h give the same result; i.e., execute `(f y3) -: (h y3)`

```
(f -: h) y3
1
(f -: h)"2 y3
1 1
(f -: h)"1 y3
1 1 1
1 1 1
```

Although the arguments of minus must agree, compatibility is not necessary for difference tables:

```
2 4 -/ 1 2 3
1 0 _1
3 2 1
```

The first step in creating a similarity table is applying minus to the rows (rank-1 cells). Parentheses are used here for readability but are not needed for execution.

```
1 2 (-"1) i.3 2
1 1
_1 _1
_3 _3
```

The rank conjunction bonds minus and 1, creating a new verb that applies to the rank-1 cells. Test this with a simple set of data; 4 points in 2-space, the columns being the x and y co-ordinates:

```
]y=. 1 1 2 3 ,. 1 2 1 3
1 1
1 2
2 1
3 3
-"1/~ y
0 0
0 _1
_1 0
_2 _2
...
2 2
2 1
1 2
0 0
```

Now use a fork to square (*) these differences. Because cap ([:]) caps a branch of a fork, ([: g h) makes g monadic.

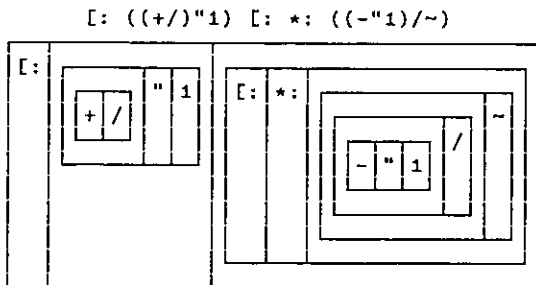
```
([: *: -"1/~) y
0 0
0 1
1 0
4 4
...
4 4
4 1
1 4
0 0
```

The squares of the distances are the sums of the squares of the differences. This is true also in multi-dimensional space. Take the sum over the rank-1 cells (rows), using cap to complete another fork. The result can be checked by hand.

```
dsq=. [: +/"1 [: *: -"1/~
dsq y
0 1 1 8
1 0 2 5
1 2 0 5
8 5 5 0
```

To read the definition of dsq, first identify the classes of the elements: there are four verbs ([: + *: -); two adverbs (/ ~); and one conjunction.

The parsing rules specify that: (a) adverbs and conjunctions are executed before verbs; (b) an adverb modifies the verb (or verb phrase) to its left; (c) and a conjunction has long left-scope and short right-scope. Consequently the parser sees the elements grouped as a train of 5 verbs:

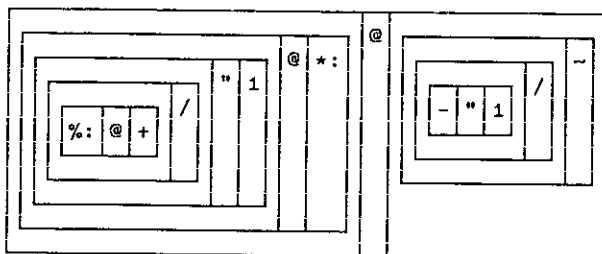


Unnecessary parentheses can be dropped without affecting the result:

```
dist2=. %:@+/"1@*:@{"-1/~)
```

Though `dist`, `dist1`, and `dist2` give the same results, the parser treats `dist2` as if it were written:

```
((((%:@+)/)"1 )@*:)@((-1)/~)
dist2
```



Read the expression from left to right. Because `@` has short right-scope we first have `(%:@+)`. But the conjunction `"` has long left-scope and therefore takes the verb phrase `((%:@+)/)` as its left argument; its right argument is `1`. Everything to the left of the next `@` is now a unified verb phrase, which is the left argument of `@`, while `*` is its right argument. Everything to the left of the last `@` is its left argument, while the parenthesised expression `(-1)/~` is its right argument. Lastly the verb `-1` is modified by two adverbs `(/~)`. Parentheses prevent `(-1)` being taken as the right argument of `@` and the two adverbs being left to apply to the entire expression.

Concluding Remarks

Omitting essential parentheses is a common error. My advice is: (a) use forks where possible; (b) scan from *left to right* looking carefully at the right argument of each conjunction (especially any atops) and remembering that conjunctions have short right-scope; (c) write fully parenthesised sentences and use boxed displays to make sure that the grouping is what you intend; (d) only then cut down the number of parentheses, always verifying that the result remains unchanged. With practice you will find that you are beginning to think like the parser! If you check all points in this paper by reference to Iverson's *J Introduction and Dictionary*, it will become easier to use the *Dictionary* as the essential tool it is.

All J expressions in this paper execute with J Version 7.

A Note on Primes

by Ted Emms

"My barmie noddle's working prime."

Robert Burns (1759-1796)

A prime number is an integer that is only divisible by 1 and itself. There are several clever APL "one-liners" which give the first N prime numbers. For example

$$\square \leftarrow (2 = + / [1] 0 = A \circ . | A) / A \leftarrow 1 N \quad \text{F1}$$

The idea here is to divide the range $1-N$ by all the numbers in the range and then identify those numbers which are only divisible by two numbers (1 and the number itself). If one is only interested in the start of the series of prime numbers then this formula will suffice. For example $N=120$ gives the series 2 3 5 ... 109 113 but $N=121$ gives (I-APL on my computer) the dreaded *WS FULL* error message. Another "one-liner" is

$$\square \leftarrow (\sim A \in A \circ . \times A) / A \leftarrow 1 + 1 N \quad \text{F2}$$

and this works well up to $N=120$ but fails for $N=121$.

We can take advantage of the fact that the only prime which is even is 2. Thus we can have the formula deal with odd values of N and then prefix 2 on the list of primes produced. Formula F1 becomes

$$\square \leftarrow 2, (1 = + / 0 = A \circ . | A) / A \leftarrow 1 + 2 \times 1 N \quad \text{F3}$$

and F2 becomes

$$\square \leftarrow 2, (\sim A \in A \circ . \times A) / A \leftarrow 1 + 2 \times 1 N \quad \text{F4}$$

and these are a great improvement. For example with $N=115$ formula F3 gives 2 3 5 7 227 229 but $N=116$ fails. Similar remarks apply to F4.

If we want the series of primes to be longer we must avoid the memory-gobbling behaviour of the outer product (using a two-dimensional array). The first approach is to create a vector V of the first N integers and drop the first. The first term is now 2 and we enter this in an empty vector T . Then we cancel all the terms in V divisible by 2. The first term in V is now 3 and we add this to T and cancel all terms in V divisible by 3. We continue in this way until the vector V is

empty. T then contains the series of primes. This program is a straightforward application of the "Sieve of Eratosthenes".

```
[0]  T←F5 N;V;B
[1]  V←1+1N
[2]  T←0ρ0
[3]  LP:B←V[1]
[4]  T←T,B
[5]  V←(¬0=B|V)/V
[6]  →(0<ρV)/LP
```

You might try this routine with $F5\ 500$. It gives the primes up to 500 but is very slow. We can speed it up by only dealing with the odd numbers and prefixing the list with the prime 2.

```
[0]  T←F6 N;V;B
[1]  V←1+2×1⌊(N-1)÷2
[2]  T←2
[3]  LP:B←V[1]
[4]  T←T,B
[5]  V←(¬0=B|V)/V
[6]  →(0<ρV)/LP
```

This is faster but we can do better. Rather than terminate the loop when the V vector is empty we can terminate when the length of the T vector equals or exceeds the length of the V vector. Then we obtain the final list by appending the V vector to the T vector.

```
[0]  T←F7 N;V;B
[1]  V←1+2×1⌊(N-1)÷2
[2]  T←2
[3]  LP:B←V[1]
[4]  T←T,B
[5]  V←(¬0=B|V)/V
[6]  →((ρT)≤ρV)/LP
[7]  T←T,V
```

But we can still do better. The largest divisor of the biggest prime cannot exceed the square root of that prime. Therefore we need only carry on with the loop until the leading term of T equals or exceeds that square root. This small but significant change now gives the fastest solution.

```
[0]  T←F8 N;V;B;K
[1]  K←⌊N*.5
[2]  V←1+2×1⌊(N-1)÷2
[3]  T←2
[4]  LP:B←V[1]
```

```

[5]  T←T,B
[6]  V←(¬0=B|V)/V
[7]  →(B<K)/LP
[8]  T←T,V

```

So we have arrived at a routine which will give a long list of primes and yet is reasonably fast. Once having reassured yourself that the routine works (e.g. with $F8\ 500$) you can move to $F8\ 10000$. This takes a few minutes but the reward is the list of primes up to and including 9973. You may care to save this vector as VP (vector of primes) for later use.

For example if you want a list ($P3$) of all three digit primes we can first eliminate the primes that are too big by $P3←(VP<1000)/VP$ and then the primes that are too small by $P3←(P3>99)/P3$. We can use VP to find out whether individual numbers larger than 10000 are prime or not. Say we wish to find if N is prime. We first find the square root of N and then the next largest integer:

$$K←\lceil N^{.5} \rceil$$

We then create a reduced list of primes P by

$$P←(VP<K)/VP$$

and use this to determine if any of these are divisors of N by $+/P|N$. If this is greater than 0 then N is composite. We may put these ideas together to form the heavily-contrived direct definition:

$$AA:(2\ 9p\ 'COMPOSITEPRIME\ ')\{1+0=+/0=((VP<\lceil A^{.5} \rceil)/VP)|A\omega;\}$$

I'm not sure that this qualifies for a one-liner since it does require the presence of VP in the workspace. However the ideas contained may be employed in solving the following problems.

1. Take all the integers 0,1,...,9 and use them to form two 5-figure primes.
2. Take the 9 integers 1,2,...,9 and use them to form three primes, one of 2 digits, one of 3 digits and one of 4 digits.
3. Take the 9 integers 1,2,...,9 and form three 3-digit primes. The primes must be the smallest "anagrams". The numbers 137, 173 and 317 are anagrams (rearrangements of the same digits) and so only the smallest (137) is available for choice.

[Ted Emms has provided solutions to these problems, which I shall publish next issue – together with any other contributions readers may like to submit – Ed]

APL Product Guide — Updates

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Pressure on space occasionally prevents us from printing the complete guide, however updates will always be listed. We do depend on the alacrity of vendors to keep us informed about their products. Anyone who is not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 0439-788385, Email: 100331.644@Compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS/2 and Unix

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet Interface. "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs
	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequelLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dogon Research	Consultancy	poa	APL Systems consultancy, design, implementation, support, documentation and maintenance. All dialects with special emphasis on APL2 and Dyalog APL/W.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.

Overseas Associations and Vendor Addresses

Compiled by Gill Smith

Acknowledgement

Thanks to Marc Griffiths, who has done a great job collecting names and contact points for APL groups worldwide. We will do our best to keep this list current, but please let us know of any errors or pending changes as soon as possible.

Vendor Addresses

This is the complete address list, including Email where known. I will contact everyone on this list in good time, so that we can publish a fully up to date and comprehensive Product Guide and address list in the July Vector.

Please note that our phone number has changed! Vector production is now at:

0439-788385 (UK)
+44 439-788385 (International)

Our Compuserve number is:

100331.644@compuserve.com

Please use electronic mail where practical!

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM/SIGAPL	International	Quote Quad		
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM80
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Chicago SIG	Chicago			
Hartford Group	Hartford, Connecticut, USA			
CPC UG APL SIG (Capital PCUG)	Washington, D.C.	Capital PC Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	-	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc.	Tokyo, Japan			
Melbourne APL Grp	Melbourne, Australia			
New York SIG	New York, USA			
Potomac SIG	Washington DC, USA			
Rochester APL	Rochester, New York			
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter		
SOCAL	Los Angeles, CA	Newsletter		
SovAPL	Obninsk, Russia			
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qity SI-Info		SF60 (SI) + SF20 (SAUG)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings	
Toronto SIG	Toronto, Canada	Gimme Arrays!		

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
ACM/SIGAPL	Donna Baglio	ACM, 1515 Broadway, New York, NY 10036 USA Tel: (212) 626-0606 Email: baglio@acm.org
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245)-496647; Fax: 0245-496646.
Adaptable Systems	Lois & Richard Hill	Melbourne, Australia. Tel: +61 3 589 5578 Fax: +61 3 589 3220
Adfee	Bernard Smoor	Dorpestraat 50, 4128 BZ Lexmond, Netherlands. Tel: 31-3474-2337, Fax: 31-3474-2342
APL-385	Adrian Smith	Brook House, Gilling East, York. Tel: 0439-788385 Fax: 0439-788194 Email: 100331.644@compuserve.com
APL Bay Area Users Group	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: (415) 928-2058 Email: frgp21a@prodigy.com
APLBUG		
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattermann	Rheinstrasse 23, D-69190 Walldorf, Germany. Tel: (49) 6227-2003

The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: (203) 762-3933 Fax: (203) 762-2108
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: (33) 1 69 08 79 63
Atlantis Software	Arthur Whitney	1105 Harker Avenue, Palo Alto, CA 94301 USA
BACUS	Joseph de Kerf	Rooiberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-730036. email: acamacho@clx.compulink.co.uk Reuternet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS Tel: 0252-874697
Paul Chapman		41 Lambs Conduit Street, London WC1N 3NG. Tel: 071-831-3782.
Chicago SIG	Larry Mysz	836 Highland Drive, Chicago Heights, IL 60411 Compuserve:73040,3032
Cinerea AB	Rolf Kornemark	Skyttegatan 25, S-193 00 Sigtuna, Sweden. Tel & fax: +46 592 55 421.
Cocking & Drury Ltd.	Ronlily Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071436 9481 Fax: 071-436 0524
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: 011 885 168 Fax: 011 8122-652
CPC UG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: 301-762-9372.
David Crossley		187 Le Tour du Pont, Quartier Le Mourre, 64210 ST DIDIER, France Tel: 90-66-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: (46) 31-45 37 40. Fax: (46) 31-45 24 23.
Peter Cyriax Systems	Peter Cyriax	22 Hereford Road, London W2 4AA. Tel: 071-229-5344
Danish User Group	Per Gjerlof	Email: c03per@vm2.uni-c.dk
DataTrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX. Tel: 0604-750241
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP Tel: 081-520-6334 Email: bowman@apl.demon.co.uk
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: +31 03474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256-811125 Fax: 0256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-824741
FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Middx, UB5 4QS. Tel: 081-864-9537
Greymantle Associates	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY Tel: 0442-878065
Hartford CT Group	Bob Pomeroy	Mass Mutual Life, 1295 State St, Mallard F465, Springfield, MA 01111 Tel: +1 (413) 788 8411x2838
H.M.W. Trading Systems	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA. Tel: 071-353-4212; Fax: 071-353-3325
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 Tel: (703) 628 7824 Internet: dick.holt@acm.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics., LE18 8QL Tel: 0536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX Tel: 0368-527190. Email: clark.i@applelink.apple.com
I-APL Ltd	Anthony Camacho (for queries, order forms) J C Business Services (for pre-paid orders only)	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-760036 email: acamacho@clx.compulink.co.uk Reuternet (Sharp): ACAM 56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU
IBM (APL Products)	Nancy Wheeler	APL Products (M46V/D12), IBM Santa Teresa, PO Box 49023, 555 Bailey Avenue, San Jose CA 95161-9023, USA. Tel: 1-408-463-APL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812-3111611 Fax: +7 812-3153321 Email: aim@infostroy.spb.su

Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: (45) 42 10 70 22 Fax: (45) 42 10 75 74 Email: insimkr@uts.uni-c.dk
Interprocess Systems Inc.	Stella Chamberlain	11650 Alpharetta Highway, Suite 455, Roswell, Georgia 30076, USA Tel: (404) 410-1700, Fax: (404) 410-1773
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 071-265-1120
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel:(416) 925-6096 Fax: (416) 488-7559
Japan APL Association		23-2-302 Hiromichi, Adachi-ku, Tokyo 120, Japan
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessmer Drive, Stevenage, Herts SG1 2DX Tel:0438-310155 Fax:0438-310131 E:kestrel@apl.demon.co.uk
Lingo Allegro USA Inc.	Steven J. Halasz	203 North LaSalle St., Suite 2100, Chicago IL 60601 USA Tel: (312) 558-1342 Fax: (312) 346-9603
Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3195, Australia
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX, Tel: 021-359-5096, Fax: 021-359-0376
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866 Fax: 071-928 1006
Ellis Morgan		Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants. Tel: 0730-263843
New York SIG APL	Nestor Nelson	PO Box 138, NY 10185-0002, USA
Optima Systems Ltd	Paul Grosvenor	Alport House, Purley Way, Croydon, Surrey CR0 0XY Tel: 081 781-1812 Fax: 081 781-1999
Potomac APL SIG	John Martin	Computer Sciences Corp, 1100 West St, Laurel, Maryland 20707-3587 Tel: +1 (301) 497-2698 Fax: +1 (301) 498-8260 Email:jam@acm.org
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB. Tel: 0276-20789, Fax: 0276-683427, Mobile: 0831-307548
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A. Tel: (212)-864-3078
The Rochester Group Inc.	Robert Pullman	50 S Union St., Rochester NY 14607-1828, U.S.A. Tel: 716-454-4360, Fax: 716-454-5430
Rochester APL	Gary Dennis	Sollton Associates, 1100 University Avenue, Rochester, NY 14607 Email: gsd@lpslab.tor.sollton.com
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	991 Creekdale Drive, Clarkston, GA30021 USA
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027, Fax: 02407-3118
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91354-3812 Tel: +1 (818) 222-2759
Sollton Associates	Laurie Howard	Sollton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 570 8733 Fax: +31 20 570 8758
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region, Russia Email:askom@apl2.obninsk.su
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel: 203-868-0131
SWAPL	Stuart Yanus	PO Box 210367, Bedford, Texas 76095, USA Tel: (817) 577 0165 Compuserve: 73700,2545
SwedAPL	Glan Medri	Box 16181, S-103 24 Stockholm, Sweden Tel:+46 (8) 96 09 47
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@ifi.unizh.ch
Sydney APLUG	Rob Hodgkinson	PO Box 1511, Macquarie Centre, NSW 2113, Australia Tel:+61 2 257 5313
Toronto SIG	Marc Griffiths	PO Box 384, Adelaide St Post Office, Toronto Ontario M5C 2J5, Canada Tel: +1 (416) 532-0843 Email: marc@utcc.utoronto.ca
Unlware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France. Tel: (1) 45-27-20-81, Fax: (1) 45-27-20-71, Telex: 648348F UNIWARE
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel: 0946-692588
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523369
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: (203) 872-7806

"Calculus" by Kenneth E Iverson

reviewed by Keith Smillie

Iverson, Kenneth E., 1993. *Calculus*. Iverson Software Inc., 33 Major Street, Toronto M5S 2K9, Canada. 130 pp., \$25.00 + 7% GST. (US Dollars)

Anyone visiting a university bookstore cannot help but notice how large, expensive and even pretentious many of the mathematics texts, and indeed texts in a variety of other disciplines, have become. At the University of Alberta, for example, the main text in one of the introductory calculus courses has almost 1100 pages and costs \$71.75. Since there is also a 400-page solutions manual costing \$27.65 and a 150-page laboratory manual costing \$8.65, a student would have to spend over one hundred dollars for more than sixteen hundred pages of material for this one course which would be for many students only one of five courses being taken in a term. Whether the present-day student receives a fair return on such a large investment of money, reading time and the physical effort in handling the texts is of course another matter. (By contrast this reviewer's first calculus text was an unassuming, even drab, little book of some four hundred pages costing less than four dollars although these were immediate postwar prices.)

The above thoughts have been prompted by an examination of *Calculus*, Kenneth Iverson's latest monograph from Iverson Software Inc., which because of its size, scope and price stands in sharp contrast to the present-day texts just mentioned. Its content might best be judged by the following paragraph in the Preface:

The scope is broader than is usual in an introduction, embracing not only the differential and integral calculus, but also the difference calculus so useful in approximations, and the partial derivatives and the fractional calculus usually met only in advanced courses. Such breadth is achievable in small compass not only because of the adoption of informality, but also because of the executable notation employed. In particular, the array character of the notation makes possible an elementary treatment of partial derivatives in the manner used in tensor analysis.

The Preface concludes with the statement that

The text is paced for a reader familiar with polynomials, matrix products, linear functions, and other notions of elementary algebra; nevertheless, full definitions of such matters are also provided. The text, then, consists of the following eight chapters: Introduction, Differential calculus, Vector calculus, Difference calculus, Fractional calculus, Properties of functions, Interpretations & applications, and Analysis.

There is one page of references, and a one-page Language Summary given on the last page of the text rather than more conveniently on the outside back cover as is usual with most ISI publications.

The author has written *Calculus* in his characteristic terse style which is described very well by the following sentence beginning at the bottom of the first page of Chapter 1:

To avoid distractions from the central topic of the calculus, we will introduce the necessary notation with a minimum of comment, assuming the reader can grasp the meaning of new notation from context, from simple experiments on the computer, from the language summary in Appendix A, from the Introduction and Dictionary, or from the study of more elementary texts such as *Arithmetic*.

This is followed by two pages of simple examples illustrating some of the main features of J and a further two pages of exercises to reinforce these ideas. The remainder of the chapter consists of an overview of some of the topics to be developed in the remainder of the text, together with the introduction of a number of verbs for handling script files, constructing graphs, and defining some basic mathematical concepts such as circular and hyperbolic functions, matrix products, polynomials and complex numbers.

As would be expected the text makes considerable use of the recently introduced general derivative adverb *D*. where $u \ D \ n$ is the n th derivative of u , and the scalar derivative adverb $D=.$ (" ∂ ")(D.1) such that the expression $f \ D \ x$ gives the derivative of the function f evaluated at x . Iverson also uses the integral or anti-derivative adverb *I* such that $f \ I \ x$ gives the area under the graph of f from 0 to x . Whereas the adverb *D* is a primitive in Version 7 of J, the adverb *I* is defined in terms of a sequence of verbs which, for the parameters used for the examples and exercises, is equivalent to approximating the function by four groups of a polynomial of order ten. Persons wishing to obtain some idea of the style and presentation might read the discussion of these two adverbs in the first eight pages of Chapter 2. As an example of the use of *D* and *I*, consider the verb

$N=. (\%:@0.@2:)(\%~) \wedge @-@-:@*:$

for the probability density function of the standard normal distribution. Then $N \ D \ 1$ which has the value $-.0.241971$ is the value of the first derivative of the probability density function evaluated one standard unit from the origin, and $N \ I \ 1$ which is 0.341345 is the area under the standard normal distribution from the origin out to one standard unit. Also the expression

$5.1 \ 8.5 \ 8.5 \ ":(],.N,"0 (N \ I)) \ x$

courses often succeed only in turning students away from mathematics, and from the many subjects in which the calculus plays a major role" may well apply to the present monograph. At the APL93 conference Iverson gave a tutorial "Teaching Calculus" in which he introduced the ideas and many of the details given in *Calculus*. The diskette distributed at that time contained a number of scripts used during the presentation. A careful study of these scripts could be a most helpful introduction to the present work. After a further study of *Calculus*, the reader could then, as the present reviewer has done, have the pleasure of approaching some old problems in a more general and satisfying manner than has been previously possible.

References

- Iverson, K. E., 1991. *Arithmetic*. Iverson Software Inc., Toronto.
- Iverson, K. E., 1993. *J Introduction & Dictionary*. Iverson Software Inc., Toronto.
- Randell, Brian, 1975. *The Origins of Digital Computers. Selected Papers*. Second edition. Springer-Verlag, New York.
- Standard Mathematical Tables. Nineteenth Edition*, 1971. The Chemical Rubber Company, Cleveland, Ohio.

RECENT MEETINGS

This section of Vector documents all British APL Association meetings, and any other events of interest to the APL world. If you have recently attended any gathering which you feel would be interesting to Vector readers, please let the Editor have a brief note, and we will include it here.

In this issue, we document Donald McIntyre's J Workshop, Gary Littlejohn's visit to the APL centres in Russia, and the February British APL Association meeting.

J at Greenock

reported by Norman Thomson

On the 1st of March, Donald McIntyre gave the latest of his justly acclaimed workshops at the IBM plant education centre in Greenock. The emphasis from the start was on the various forms of function composition about which more appears elsewhere in these pages. Version 7 of J was used, in which caps ([:]) used in a fork provides an alternative to @.

Donald has now assembled a vast library of scripts which illustrate the many facets of J. These he selects with considerable flair and enthusiasm to construct individual sessions based on single problems, each of which highlights a generous variety of what most needs to be understood by the beginner. Construction of similarity matrices and of Markov chains are two such examples.

Donald also described the evolution and history of J, and the astounding capacity of the J interpreter to produce interesting results beyond what even its designer and author conceived — a new interpretation perhaps of artificial intelligence.

Visit to APL Centres in Russia November 1993

reported by Gary Littlejohn

Department of Social and Economic Studies, University of Bradford

I joined the BAA in 1992, and soon discovered through Vector that APL was also being used in Russia, and that I had just missed a conference in St. Petersburg. By emailing Erkki Juvonen, I was able to get email addresses for contacts in Russia. I was interested in this for two reasons: firstly I had a long-standing interest in what was then the USSR, having written various books and articles about it; secondly my colleague at Bradford, Gavan Duffy, who had introduced me to APL, was working for 6 months of each year in South Africa, contributing to the construction of a family of models of the South African economy using Dyalog APL. In Bradford we were also working in J to build an elementary demographic model of South Africa, displaying the results in a Geographical

Information System. I was very keen to find out what the Russians were doing in the field of economic modelling, as well as in other areas.

I received a grant from the British Council for Gavan and me to visit Moscow and Obninsk to explore the possibilities for collaboration between Bradford and the APL communities in these cities. However, at the last minute, Gavan was recalled to South Africa and was unable to accompany me.

Moscow

The Computer Centre of the Russian Academy of Sciences (RAS) is well known as the place where the original Sputnik calculations were done. It is also where the game Tetris was invented. I was received extremely warmly, and was delighted to discover that the approach to economic modelling was complementary to that being used in South Africa. There is a very real basis for cooperation, in my opinion. We drew up a draft agreement for funders. I was told that Professor Igor Pospelov and Dr. Andrei Buzin had been prevented from attending Toronto because of delays in receiving visas, and that they expected to be at Antwerp.

The experience of modelling goes back about 20 years, and APL seems to have been used since the late 1970s. The approach to modelling is quite original, compared to British techniques which tend to rely on a lot of data. In Moscow, the technique is to do thousands of iterations before deciding on the parameters of the model, which can then be run with a fair degree of confidence on a relatively small amount of data. They have a similar philosophy with regard to the transparency of the model to that held in South Africa, namely that the assumptions should be as clear as possible and that the model should be used as a basis for debate on economic policy, rather than function as a black box from which unchallengeable conclusions are drawn.

They have been modelling structural changes in the economy since long before Perestroika became a term known in the West. The behaviour of economic agents is described explicitly, and the macro-economic model is built up from the "microdescription" of the behaviour of economic agents. The development of models was done in three stages, beginning rather surprisingly with three models of a market economy. Then a model of a centrally planned economy was developed. The features of this model were extremely interesting to me since they indicated problems which Western economists were beginning to detect in the Soviet economy in the late 1970s. Clearly these problems were being addressed at that time, at least by these modellers. Was anybody listening? A model of an economy undergoing radical reform was then constructed,

apparently late in the period of Perestroika. This predicted the hyperinflation which occurred two years later under the Gaidar reforms. Such a prediction was also being made in May 1990 on analytical grounds, without the benefit of a model, by Professor Hanson of Birmingham, surely one of the best British economists in the field of the former Soviet Union. I understand that now the RAS Computer Centre team has the chance to obtain more recent data from the Ministry of the Economy in Russia.

In addition to the models, the other most interesting work consisted of GRAN, a GRaphical ANalyser, which is an interactive graphical manipulation package, unlike anything that I had encountered. It means that many statistical methods and well known economic models can be graphically interpreted in an interactive mode. I see it as a useful complement to the *APL Statistics Library* work based at Swansea. Most interesting of all to most APL users is the work being done on a compiler. I know that they have had what they call a pre-compiler for some years, but they have been working on a full compiler. Realising the significance of this I did my best to make sure that we were not losing something in translation. We may get a paper on this at Antwerp. I have since received an email communication saying that further progress has been made on the compiler.

I hope to return to Russia to explore these matters in more detail in future. I also sincerely hope that they give a paper in Antwerp, since the Vector discussion of their work at the St. Petersburg conference does not really show how interesting it is. A final point is that a commercial company is using Dyalog APL to construct data bases. A discussion of the relative merits of such data bases and Paradox for Windows was held on the train to Obninsk.

Obninsk

Obninsk was a closed city, with a closed Zone inside the city, until December 1991. The reason is that it was and remains the heart of the Russian nuclear industry. The world's first nuclear power station started generating on 27th June 1954. I was taken right into the reactor core by Sasha Skomorokhov, who will be well known to participants at the St. Petersburg and Toronto conferences. I had read his Toronto paper on adaptive learning networks, as well as those of other Russian Toronto participants, most of whom I met in Obninsk. Apart from such work which can be found in Vector and APL Quote Quad, there is also very interesting work going on in APL in the fields of Computer Assisted Learning (CAL), statistics and personality profiling. I found this out when I met staff from the Obninsk Training Centre.

This is the main training centre for the Russian nuclear industry, with its own hotel (where I stayed) and dedicated training building. The Centre is suffering from the fact that outside agencies now have to purchase their training, and many are now unable to do so in a situation of hyperinflation. Nevertheless, they are continuing to work on, for example, CAL material for training nuclear engineers. I had never seen APL used in this manner before, with moving diagrams of the flows within a power station. Having just received a detailed lecture on the design of the first such power station, I was able to follow the logic of the lessons. The statistics work is unlikely to be discussed at Antwerp, because the person concerned does not feel sufficiently confident in English. We discussed the relative merits of SPSS and APL for exploratory data analysis.

The personality profiling for nuclear power station engineers was very interesting, using a panel method for assessment by colleagues and a member of the management, which is then analysed using APL statistics programmes and discussed with the subject. Again, unfortunately, the language barrier makes it very unlikely that a paper will be presented at Antwerp, despite my attempts to encourage those concerned to do so. I feel that the work would be of interest to management in the West, being of quite general applicability, but with interesting differences from some Western staff appraisal methods, because of its emphasis on safety. One might see this work and that of Sasha Skomorokhov as part of a post-Chernobyl attempt to improve safety in the nuclear industry. While I discussed Chernobyl with Sasha, I did not at the time think to link these aspects of the work in Ozninsk.

I also gave a lecture in English at the Atomic Energy Institute (with Sasha translating, except when I forgot at one point and spoke in Russian, which he translated into English!) on current problems in the British economy. I spoke to various economists and received papers on monetary theory. I also met two Vice Rectors of the Atomic Energy Institute and signed an agreement on research collaboration, not only in the area of APL, between it and Bradford.

Conclusion

I am now seeking funds for further research collaboration with both the RAS Computer Centre and the Ozninsk Atomic Energy Institute. Apart from the extremely interesting work which I encountered, I had an unforgettable time with a warmth of hospitality that is hard to describe. For that reason alone, I want to return, but I also feel that the work going on there is very important for the APL world community. The hyperinflation is having very adverse effects on the APL work there, and financial support is needed. I hope that a way can be found to augment the existing fund.

British APL Association Meeting 4th February 1994

David Eastwood and Peter Cyriax, reported by Adrian Smith

Introduction

Two contrasting talks, both interesting! David Eastwood gave us an overview of the Toronto APL Toolkit (now 11 years old, and still going strong) and Peter Cyriax took us carefully through the steps needed to improve the performance of large APL systems.

As a reporter (and a member of the Vector working group) I would be very interested to hear readers' reactions to the following proposition: I feel that we may be rather neglecting the mainframe, because it is old, boring and just generally part of the scenery. However, what Peter made very clear in his talk is that, on the mainframe, *APL is still unique*. It is one of the very few interactive environments available, it is pretty good at spreadsheets, it does very competent graphics, it can handle reasonably large amounts of data very quickly, and it is better than COBOL at doing hard sums (particularly if the sums involve floating point). If you are stuck on the mainframe (because that is where your data is), and you want to do any of the above, you will very likely use APL.

Does the mainframe readership of Vector feel alienated by all this Windows and Gui stuff? I think that there's life in the old dog yet, and we should probably make a positive effort to get better mainframe coverage. Please let any of us know if you agree.

The Toronto Toolkit

outlined by David Eastwood

Essentially, this consists of generic VS APL code, freely distributed in electronic form for more or less every APL you can name. It has been running for 11 years, originally in book form, and has been available on disk since 1992.

Why use a Toolkit?

Sometimes you just need to cover a gap in the interpreter. If you are used to `⎕DBR` (delimited blank removal) on APL68000, or `⎕SS` (string search) on

APL*PLUS/PC, and you move to Dyalog APL (which has neither) you need functions to fill the holes.

Sometimes you want to do something hard, like subtracting two calendar dates, which is a problem that has been solved many times before. An old (=reliable) utility will save you plenty of thinking and testing.

Platform independence has its attractions! Bulletin boards are well clogged with "how do I get this code from here → here?" questions.

It helps to teach you good APL. The people who wrote this stuff knew what they were at, and knew that it was going into the public domain. They cared about quality, and clarity, and readability. In answer to the question "where do I learn about good APL" you can reply "look at the toolkit functions, then copy and modify".

Observations

Setup is easy. Everything is $\square IO$ independent, so all you need to do is add your own error trapping and set up one global ΔCR to tell the functions what your 'end of line' character is.

From this point, everything worked, and some of the functions are already out earning their keep in real systems. Two small points to note (for users of more modern APLs) are that there are no 'default' left arguments (i.e. no ambivalent functions), and that your particular APL may support many of the 158 supplied utilities already, so don't just copy them all!

Some Examples

There is a lot of stuff, so why not use its own 'workspace browser' code to explore ...

```

p⊂NL 3
158 15

5 matacross ⊂NL 3      ⍝ to see a columnar list
'ba*' nl 3             ⍝ wildcard list (3 across)

explain 'nl'           ⍝ documents function 'nl'

```

You will find lots of handy stuff:

- a function lister which extends comments and labels to order
- a global reference finder which tracks down rogue variables which should be local and aren't

- something to put your locals into alphabetical order, and remake your line labels (if you like the L10, L20 variety)
- something to browse the entire workspace for a string or proper APL name. It will also replace (but unfortunately it does it within quoted strings) on a proper name basis.
- something to show the bracket balance within complex lines of code.

That is just one section of the Toolkit. There is plenty of other good stuff, for example the coverage of date conversion is quite comprehensive, allowing you to work on Julian dates internally (the number of days since 4712BC), find out when Easter will be in 2025, and so on. It is definitely worth having a copy on your system, and making sure someone knows about it and can publicise it.

Performance of Large APL Systems

by Peter Cyriax (working at Commercial Union)

What do I mean by a "large system"?

- many workspaces, many users, so ...
- no informal method of finding out what is going on any more!

Where do you look to improve performance? Most people can optimise a piece of code, but what should you optimise? A tool like the APL*PLUS compiler is useless on its own; you need a large range of tools, of which the compiler is only one.

Background to Commercial Union's Installation

There are two coupled mainframes, each of which consists of 6 CPUs, each rated at 45 MIPS (approximately equal to a 300MHz PC). Provided the entire cluster is run below 84% load (which it always is) each APL user has a 300MHz PC available for his sole use! What he sees is a 3Mbyte workspace, a 4Gbyte local disk, and an enormous ADABAS database accessed via 'hot batch' using the "Natural" query language.

Performance, as the user sees it, is dominated by line-speed. Everyone has enough CPU, so local users (like Peter) see better than ½ second response, while remote users out in the branches can be down to 3-10sec when the network is busy. However in general terms:

"It's not as pretty as a PC, but it is as fast, and for some jobs it's better"

Why Care about Performance?

Two reasons: (1) Cost, (2) Response time. In general, cost is what people on mainframes care about; PC people are more concerned about response time. So — the DP charge to the CU Life Department is the thing to attack! Moving out to PCs is not an option, because the mainframe is where the data is, and the system is in constant use out in the branches answering real questions like "... if I terminate this policy, how much is it worth to me now?".

Roughly 10% of the total machine load is APL, so there is plenty there to attack! In round terms, it costs the guy who holds the budget for APL £5M p.a. We look for 20% improvements in code efficiency — I only wish they'd paid me what I'd saved them!

Aside — "I believed that it was not possible to write code so badly that it would affect response time — I was wrong, it can be done!"

The Tools

There are four subjects you need to address if you want code to go faster:

1. Attitude. You have to CARE!!
2. Analysis. The workspaces are too big to browse by eye.
3. Design. If the logic is sufficiently bad, no coding efficiency will help.
4. Coding Techniques. For things which are computationally subtle. For example to find `'hi*ho'` in a big text database. The routine we have will search the Bible in 1 CPU second — but it took a week to THINK about it!

First, the correct **attitude** to optimisation:

- Don't do it
- Do it tomorrow
- Go for speed
- Correct results? Worry later!

Do one job at a time!

As soon as you start mixing "getting things to go quickly" with "getting them to run right" — you fail. Keep trying algorithms as long as they come out somewhere near.

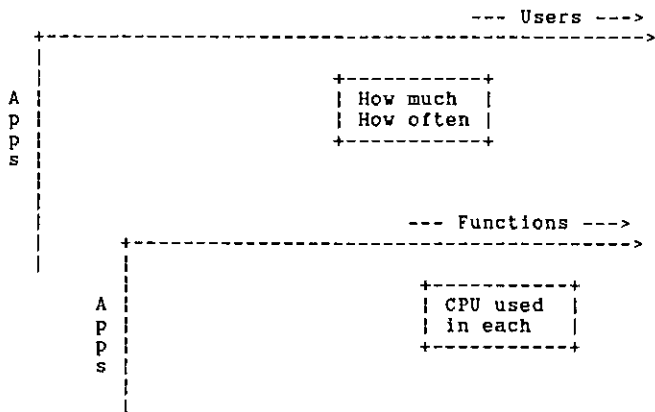
Aside — the rate at which volumes of data are increasing is still outrunning the increases in CPU power. Typically, when you process data you compare one set

with another, so the processing time goes up as the square of the data volume. The true price of resources is still going up.

Next, the analysis tools you will need. In particular we would like to know:

- which users use the CPU (easy)
- which applications use the CPU. This is much harder, (unless you organise some Trojan Horses in advance)
- which functions use the CPU (across 200 applications)
- what functions exist in any application
- which lines are doing the damage!

What you are trying to do is construct a pair of graphs:



Remember — if you can hit 1% of the CPU, that's an ongoing £5,000 ongoing cost-reduction every year.

Aside — in an insurance business your central operation requires data lifetimes of 20-30-40 years. You must process where the data is!

How do you figure out what to hit (i.e. how do you make those two charts)? It is impossible without a degree of system and code management which will:

- record cumulative CPU on entry to and exit from each workspace
- switch on `MONITOR` within applications on demand
- record which utility functions are present in each workspace

- maintain master copies of utilities, which are shipped automatically when updated. There is an appalling function called *GO* in every workspace — it has nasty names and contains lots of horrid code, but it does the business!

Unless you do these things, you might be able to optimise a workspace; you will never optimise the whole system.

What can you do about **Design**? A huge portion of all applications is utility code, so if we optimise that we can write highly optimal applications without having to think. Don't optimise application code, it is too subject to changes in the specification.

Optimising Code — the last stage.

- you do need a reliable CPU monitor.
- learn by experience.
- THINK! You want the biggest element is a big numeric vector; $1 \uparrow A [\Delta A]$ may look neat, but it will not be fast (why?).
- when you need scalar looping code (which is sometimes unavoidable) get into some language which is tuned to do this. This is where the APL compiler fits.
- use assembler *FASTFNS* if you have them, but hide them in your own utilities and make sure everyone is using them.

Some Examples

Here are five systems which have been attacked at various times over the years:

1. *PTP* — which writes a few letters. Finding out what to attack!
2. "unnamed" to protect the guilty. The design was so bad that it had to be rebuilt.
3. *IPSZ*, a utility to take a long list of +ve numbers and break them into clumps such than none exceeds a given value. A classic use of this is wrapping text into a given page width. Just what you need a compiler for.
4. Making a useful *FI* for column matrices.
5. Various methods of reading files, particularly the variable-length kind.

First, let's look at *PTP*. The first thing to do was to uncompile it and run it again. The system as a whole remained balanced, but the CPU increased by 65%. This was interesting because the data volumes were small, and typically the compiler roughly halves CPU. Was it *GDDM*? This came out at 30ms per screen access, and *GDDM* calls accounted for only 25% of this — not a major issue. Now let's look at print generation — aha! 3 seconds of CPU to run 3 pages — appalling!

With some serious work this should improve at least 10 times, so now we know where we should concentrate our efforts.

Now let's take a look at "unnamed". This runs around 900 times a year. Up to June it averaged 13cpu sec per run; from August it is down to 2.5 sec and disk I/O is also down by a factor of 4 — *so what was it doing!*

```

+-----+
| Big    | >>>> <=====>
| Flat   | <=====> Component file
| File   | <=====>
+-----+

```

Basically, it submitted a batch extract and split out 50 fields into 50 separate file components, with some selection and ordering as it went. As originally written it read the flat file 50 times, compressing and indexing using the same criteria at each pass. With a little cunning and a scratch file, you compress and index once only, and (more to the point) read the original data once only!

Aside — you really would not believe what end-users get up to!! There was no spec (OK, we're used to that), and the logic was uniformly distributed across the functions! In the end we synthesised a new structure from the code, and when we had finished over 90% of the code had simply vanished!

Text wrapping with *IPSZ*. This is quite a simple loop: chop off what fits, write a line, chop off what fits, write a line ... etc. All the sensible APL algorithms are of the n^2 variety, so here is a great opportunity for the compiler. However you do need to do lots of horrid things to your code to keep the compiler happy:

```

N←'ppL'      a Force N to be (clearly) scalar
S←~1         a Unnecessary initialisation outside the main loop
R←Np ...     a Always prebuild result

Loop:
  R[+]← ...   a Use indexed assignment into result

End:
  R←I+R       a ... and clip it at the end!

```

However, the resulting function does still look quite like APL, and it is a damn sight easier for an APL person to do this than to rewrite functions in C (and then work out how to call them!).

Data validation with *QUADFI*. This executes character matrices on the assumption that there is one number per row. It is a little more expensive than

□FI, but does more work, as it correctly handles blank rows and other gunk in the data. A useful trick:

```
B←~'' MATIOTA CHMAT      a Finds blank rows fast
CHMAT←B÷CHMAT
```

... and only do the □VI where the data is legitimate.

Finally, **reading IBM files**. Some of the tricks are old chestnuts, but just in case you don't know about RECFM=U, I'd better tell you!

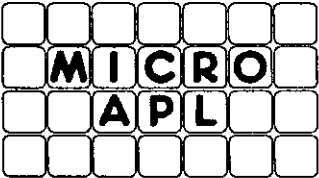
The obvious loop: "open □SV0, read, check, read, check ... close" is SLOW, horribly slow, so what can you do? Well, checking the control variable takes the same time as reading the data variable, so don't do it (after the first time). As far as anyone can tell, the time to read a record is totally independent of record size, so read blocks (up to 32K bytes) instead of 80-byte records and you get an immediate 400-fold reduction in CPU time. Oddly, this is what the TSO file AP does if you tell it you don't know what your record format is — so do that and just trim the tail and reshape afterwards ('cos you knew that they were really 80-bytes, didn't you!)

The fun starts if you have variable-length records! We gave up (wrongly) on these for a long time, then someone built a little sample file, pulled it into APL and examined it carefully. It turns out that there is a 4-byte length counter embedded at the start of each record, so you can just grab the whole file (as above) and chain down the records. Clearly, there is no way of doing this in parallel, but it works very well with compiled code — and saves us £1,000 per week in CPU costs.

Reprise

Attitude (you have to care), analysis (you need to know where to look), design (don't even try to optimise if the structure is too bad), and coding techniques (exploit the compiler if you have it).

If you do this intelligently and thoroughly, a year-on-year reduction of 20% in the CPU cost of current systems is quite within your grasp.

—		APL Preferences		▽	
APL.68000					
Platforms		Versions			
☒ RISC System/6000		☐ Level I			
☒ Apple Macintosh		☒ Level II			
☒ Commodore Amiga					
☒ Atari ST					
APL for GUIs					

MicroAPL Ltd., South Bank Technopark, 90 London Road,
LONDON SE1 6LN, UK

Tel: 071 922 8866 Fax: 071 928 1006

Applelink: microapl Internet: microapl@applelink.apple.com

GENERAL ARTICLES

This section of Vector is intended for articles about the applications of APL, and non-technical (i.e. having a minimal content of actual program code) material about the development of the language.

In this issue, we have a plea from Kromberg and Gfeller for standardisation in the APL environment, and a proposal from Smith and Pearson which at least offers some hope that access to the Windows GUI can be common across all major APLs.

We also have two applications: the paper from Sylvia Camacho addresses the tricky problem of finding matches in a database of names, given mis-spelled or confused information; the paper from Thomas Olsen is a complete example of a Windows application programmed in J.

Finally, George MacLeod explores the limitations of Windows as a high-quality graphics engine, and suggests some solutions.

Standardization Beyond the Language

*by Martin Gfeller and Morten Kromberg
mgf@rmx.risk.reuter.de / insimkr@uts.uni-c.dk*

Introduction

As APL professionals working with several APL and non-APL systems, we have become increasingly frustrated by the lack of standardization in some ancillary areas of the APL language and system. This lack of standardization leads to duplication of human effort and computer resources, sometimes to the point where more standard non-APL solutions become more feasible, despite APL being inherently easier for the task.

Standardization should increase the portability and inter-operability between different APLs, and between APL and non-APL software. The areas to standardize concern language pragmatics, rather than language fundamentals. Hence, they should be easier to standardize, since fewer design beliefs are involved.

While APL should adopt common industry standards established in its environment, there are two broad APL-specific areas that need special consideration: character set issues and array representation.

Character Set Issues

APL's use of special glyphs is an asset, but carries quite a high cost. Standardization attempts to minimize this cost.

Character Set Encoding

Different APL products use different ways of superimposing the APL character set onto the operating system's character set. A standard mapping would allow easy exchange of APL text files and fonts, as well as easier mixing and matching of language systems with session managers and user interfaces.

Three distinct mapping schemes are needed, according to the width of the underlying system's character representation. The most important character sets of different width are 7-bit ASCII, 8-bit ASCII, and Unicode.

- **7-bit ASCII.** 128 character positions cannot accommodate alphabets, numerals, punctuation, control characters, and the APL glyphs. APL uses overstriking to solve the problem, and represents the second alphabet by overstriking the first with the underbar character. It also discards some ASCII characters not used in APL. Fortunately, this encoding has been standardized, and hence most APL terminals or terminal emulators are usable with most ASCII based APL systems. The industry standard is called APL-ASCII Typewriter-Pairing [ISO 86, in annex].
- **8-bit ASCII.** With 256 characters, all APL glyphs could be represented, in addition to all standard 7-bit ASCII characters. However, 8-bit ASCII was not invented for the benefit of APL; and most of the 128 additional positions have been used for additional European language glyphs, and for various useful symbols (such as smiling faces).

APL vendors superimposed their character set on 8-bit ASCII by removing some ASCII glyphs. Some vendors occupied parts of the first 32 positions, which are defined to control devices. While this is acceptable on PCs, it makes handling of asynchronous devices difficult. Even more severely, most vendors implemented a different mapping scheme. This makes the transfer of APL text and the exchange of tools very difficult. For example, each APL product generally requires its own screen and printer font (plus different fonts for each printer type).

- **Unicode.** Multi-byte schemes such as Unicode (UCS-2 in ISO 10646) with more than 65000 positions allow the representation of all APL glyphs, including overstrikes used in any system, without affecting other character positions. The Unicode standard [Unicode 91] contains all the APL characters, but products must become available to support it. An implementation for APL2 is proposed by [Brown 93], which also discusses many considerations and trade-offs related to these issues. Basically, the 8-bit ASCII character set should be a defined subsequence of Unicode, including all ASCII characters and APL glyphs. Conversion between 8-bit (short) characters and 16-bit (Unicode) characters must be handled automatically by the interpreters.

Action Points

A standard for APL representation in 8-bit ASCII is most urgently needed. Vendors should incorporate the standard, and provide migration tools toward it. The standard should be defined by designating a subsequence of Unicode with 256 distinct elements.

APL products implementing Unicode are required for Unicode to unleash its potential benefit to APL.

Keyboard

The variety of APL keyboards is bewildering. Some differences stem from the requirements of the underlying operating system (such as Windows reserving the use of the Alt-key to invoke menus), but some are plainly unnecessary.

The keyboard issue is largely independent from the character set encoding, but is complicated by different national keyboard layouts. We suggest that standardization should begin with the US keyboard, and national APL user groups should transform the US APL keyboard to their national layout. To allow this, the APL keyboard handlers must be flexible enough to handle problems like "dead keys"; table driven finite state translators fit this requirement.

The primary requirement for APL keyboard design is to leave the ASCII keyboard undisturbed. This means that all keys issue their standard glyphs, and APL glyphs are only generated with otherwise unassigned key states (Ctrl, Shift+Ctrl under Windows). Since all keys retain their default behaviour in the unmodified and shifted state, the alphabetic map to minuscule and majuscules. Since APL names should be enterable easily, without using Caps-Lock, this leads naturally to the *union character set* convention [Gfeller 86].

Action Points

Design a standard US ASCII APL keyboard that works with Windows. Let national APL user groups adapt the keyboard to the layouts of their countries.

Human-readable Keywords for ASCII

There will always be devices that cannot be used to enter or display APL glyphs. While they might not be used for extensive APL programming, there is no reason for not making them available for APL use. The most common solution for this problem is keywords, but again, there is a large variety of keyword schemes, and none of them is popular across APL implementations.

Keyword schemes should be designed for human readability. They do not need to mimic the terseness of symbols, because the result is often cryptic. They should be easy to remember or discover, because many users will resort to them infrequently, when no APL device is available. Longer words can be more descriptive and may be easier to remember; on input, they could be abbreviated as long as they remain unique.

Keywords should map one-to-one to glyphs; adoption of different keywords for monadic and dyadic use involves syntax analysis and makes the transliteration too complex.

Action Points

Standardize on a simple, readable keyword scheme, and adopt it in all language implementations.

Array and Code Representation

APL workspaces, functions, and arrays are more structured than objects in many other languages in that they cannot be represented as simple text files.

Workspaces need a static representation for transfers between systems, whereas arrays need a representation for dynamic exchange between processes.

WSIS Transfer Standard

A *Workspace Interchange Standard* (WSIS1) exists as an annex of the ISO Standard APL [ISO 86]. It does not extend to derived functions and user defined operators (nor to APL2 or SHARP APL packages), and it is not widely supported.

An extended WSIS should be defined, encompassing at least nested arrays and user-defined operators. Precise rules on how the various dialects should import border-case nested arrays (transferring enclosed scalars from a grounded to a floating system) should be defined.

The WSIS standard should be based on the standardized 8-bit ASCII character set encoding, so a complex translate table mechanism will not be necessary (since the representation is static, it can always be passed through suitable reversible utilities such as uuencode to convert to a 7-bit or any other desired encoding).

Action Points

Create an updated WSIS. All vendors should support it. A test workspace in WSIS should be provided for vendor conformance testing.

DDE/OLE Interchange Formats

DDE (*Dynamic Data Exchange*) and OLE 2.0 (*Object Linking and Embedding*) are Windows standards for inter-application communication. Other standards exist or are emerging in other operating system frameworks, but many representation issues are common amongst them. (XDR is an important representation description standard in the UNIX environment; it is used to describe RPC parameters. OMG's CORBA IDL is a new standard to describe objects.)

There are DDE and OLE format conventions for common structures such as bit maps, text strings and spreadsheet cells, but not for multi-dimensional nested arrays. Such a format would be useful for communication between APLs of different vendors, between APL and ancillary services (e.g. session managers, network shared variable processors, or array editors), or between APL and other applications using array structures.

There are two kinds of formats, a native format and a presentation format. The presentation format is a text representation of the information, and is equivalent to APL's display form. The native format is useful for further internal processing, and can be used to exchange arrays between processes that understand their structure.

Both formats should be standardized. The descriptor part of the format (the "header") must not make use of APL glyphs. The array contents may be character data based on standard 8-bit ASCII or Unicode, as indicated by a flag in the descriptor.

Action Points

APL array DDE/OLE native and presentation formats should be defined. APL implementations under Windows should make them available through their respective interprocess communication mechanisms (Shared Variables, native function calls).

Foreign Function Descriptor Format

Many APL interpreters are able to call functions written in another language. Often, this facility is implemented by an APL system function (`⌈na`) that associates an APL function with a description of a foreign function. However, the format used to describe the foreign function (its signature; e.g. parameters and their types) varies between dialects. The descriptor format should be standardized, and a format that can be automatically generated from C and C++ header files should be preferred.

Foreign functions may need to access APL workspace structures; however, this is an area that realistically cannot be standardized due to the different internal formats of workspaces for different interpreters and architectures. Portable applications should instead use the inter-application communication methods described above to access data in foreign APL workspaces.

Action Points

Create a standard descriptor format for foreign functions. Build tools to transform C and C++ header files into this format.

Martin Gfeller
Reuters SA
Risk Management Software
Kleinstrasse 6
CH-8008 Zürich
Switzerland

Morten Kromberg
Insight Systems ApS
Nordre Strandvej 119a
DK-3150 Hellebæk
Denmark

References

- [Brown 93] J. Brown et al. "*Extending the APL Character Set*", APL 93 Conference Proceedings, p. 41, Toronto, August 1993.
- [ISO 86] IS8485, "*Programming Language APL*", International Standards Organization, Geneva, 1986
- [Gfeller 86] Martin Gfeller, "*Adding Lowercase to APL*", APL News, Springer Verlag, June 1986.
- [Unicode 91] The Unicode Consortium, "*The Unicode Standard*", Worldwide Character Encoding, Version 1.0, Vol. 1, Addison-Wesley, Menlo Park, CA, October 1991.

A Common Approach to the Windows GUI

proposed by Adrian Smith and Duncan Pearson

Abstract

This is the first of a set of three papers which together outline a common approach to programming the Windows User Interface from APL. The base utility set is now complete for Dyalog APL, and will quickly be ported to APL*PLUS III now that we have a beta-release of the interpreter. This will be freely available at no charge (other than handling and postage) from APL-385 or the British APL Association.

This paper covers the background, specifically:

- why we feel that a common approach to GUI development is a good (indeed necessary) thing for APL.
- the philosophy behind the design, particularly the idea that what you see and manipulate on the screen is simply the visible manifestation of some APL variable.
- some of the pretty things you can do, and some idea of how fast it is to do them in this environment.

The second paper will be a detailed technical walk-through of the code that forms this basis layer. All the major functions will be listed and discussed, and the commonality of approach across Dyalog and PLUS III will be highlighted.

The third paper will focus on two particular features of the environment: printing and high-quality graphics. It will also introduce the 'developers' kit' for use by any skilled APLer in producing (and maybe even selling) objects which go far beyond the base functions shown in today's proposal.

Why a Common Approach?

The basic answer is very simple – because it greatly increases the life-expectancy of your APL code. At Rowntree, we have always built our mainframe systems on a layered set of utilities; screen handling went through FSM from the early days of AP124. This allowed us a simple migration to AP126 and GDDM (and hence we could exploit colour and graphics) while the Nestlé systems which we have recently inherited are still firmly stuck on AP124 and look old and cranky by

comparison. The FSM approach transferred easily to the APL*PLUS/PC environment, and allowed us to train new graduates on a single set of utilities, regardless of the platform they were working on. This not only reduces training costs, but it allows the OR manager far greater flexibility in deciding who can go on holiday when! In the end, it is things like this which determine whether an old system lives on, or whether the business cannot face the cost of a rewrite, and reluctantly abandons something useful.

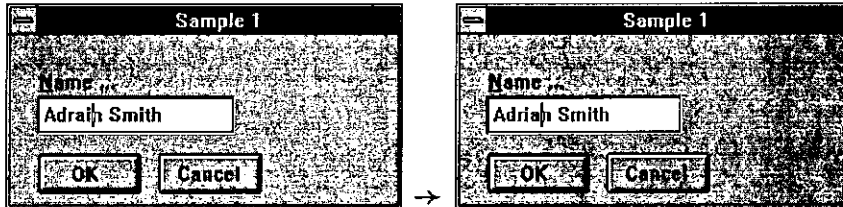
There is an important, indeed crucial, difference between the simplistic 'cover function' approach taken by the consultants who built the Nestlé systems, and a true platform-independent layer which is designed from the outset to be portable. Simply hiding all your `SV0` calls under 1-line utilities may make you feel better, but it takes you nowhere. You are still structurally tied to one particular way of (in this example) writing data to the screen. You may just move your code to another mainframe system (AP124 to AP126, say) which is closely related, but you will never move it to a Vax or a PC which handles the display in a fundamentally different way.

Of course there is another (usually unspoken) reason to make sure your code is portable: the vendor from whom you bought your interpreter might either go out of business, or might just decide not to bother with APL any more. If I was dependant on Vax APL, or even APL2/PC I would be seriously concerned that both Digital and IBM have bigger fish to fry, and I might without any warning get 'stabilised' and eventually de-supported. Both Manugistics and Dyadic are small companies, and one or both might simply curl up and die sometime over the next 10-20 years. I am proud of the fact that I still have code running with 1982 timestamps on it – can I rely on both of the above still to be there in 2006? The odds of at least one of them being there are very considerably higher, but which one?!

However, portability isn't just about paranoia! If you build your utility set from the perspective of what the application needs, rather than what the subsystem provides, you just get a much better utility set. The Visual Basic design toolkit [1] is a classic example of how not to do it – it simply offers you everything Windows has thought to provide, and it is your problem to work out which of the multifarious elements you must combine to make a sensible application.

The first thing I found I was doing in APL was making something which looked like this:

```
name←'Adrain Smith'
Gui_call sample1      A A utility fn to call 'sample1'
```



```
name
Adrian Smith
```

The text field is just the visible manifestation of a variable! If that variable should change for any reason, the visible text will automatically update to show the new value. If I make a change to the visible text, as soon as I quit the field, the variable is assigned to match what I can see on the screen, and any other objects which might happen to be linked to it get updated automatically and immediately.

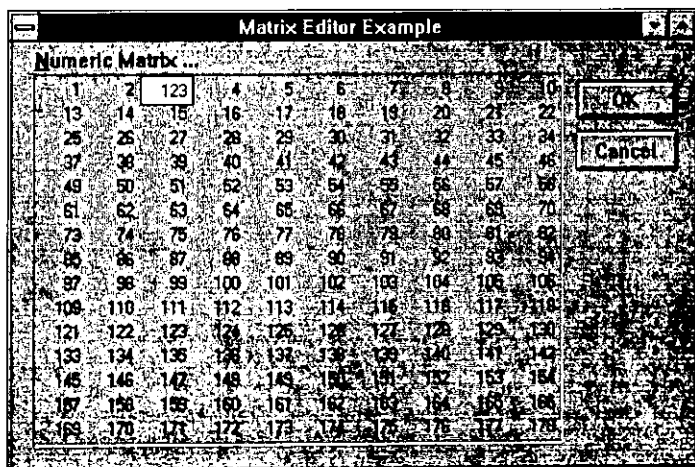
How many objects can you see? Including the form itself, I can see four:

- a simple form with a caption of "Sample 1"
- an OK thing (no caption needed here) which will close it
- a Cancel thing (ditto) which will abandon it
- a text thing, with a caption of "Name..." where I can type something. In this example, it has been linked to the APL variable *name*.

More or less every dialog box will have some combination of OK, Cancel, Close, so I should be able to add one of these to my form, and it should already know how to behave (e.g. OK should close its form, and return 1 to whatever called it up). Virtually every edit field has a caption, and I generally want it just above, left aligned. I have spent enough frustrated minutes watching our PowerBuilder developers trying to line up their labels with their edits to realise that there had to be a better way!

I want a matrix object, which will take an array of numbers, sling it on the screen, let me type all over it, and ensure I get something sensible back! And so on, and so on.

```
numbers←23 123 1000
Gui_call testmat
```



```
numbers
1 2 123 5
13 14 15 16 ... etc.
```

This is APL after all! Windows offers you one or two quite good vector controls (like the list box) but is quite hopeless at arrays.

To summarise: a common approach should give you two things: insurance against future change, and a development platform that started life with the APL application developer in mind. How well Duncan and I have achieved that goal, it is up to you to judge.

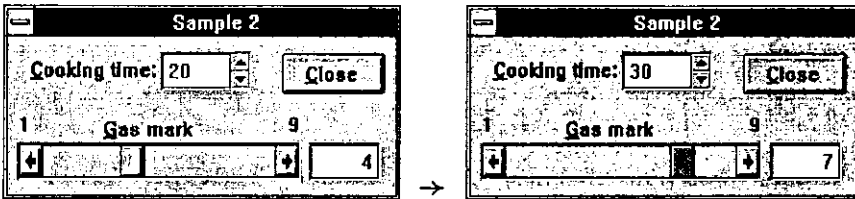
Design Philosophy: 1 – Direct Manipulation

The first thing we must do here is to acknowledge our joint debt to Ian Clark. We first saw APLomb[2] at an APL vendor forum just a year ago, and independently came to the conclusion that, if there was a right way to do it, this was it.

The key concept to grasp is that what Ian offers is direct manipulation of APL variables by the Windows objects. By this, I don't mean fancy stuff like dragging lorries across the map to move goods from factory to depot (although it does this too), but much more mundane things like editing text and setting numbers to values.

Now look at this:

```
cooktime←20
gasmark←4
Gui_call sample2
```



... dragging the scroll bar changes the number to its right; typing over the number shifts the scroll. I can spin the cooking time (in 5-minutes units) or just type it in. When I close the form:

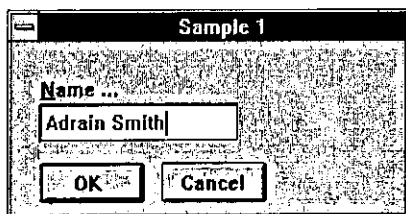
```
cooktime gasmark
30 7
```

The example above nicely illustrates our first design objective: it requires absolutely no APL code, and it took me less than 3 minutes to make it, test it, and copy it from APL to the printed page. I simply assigned the variables (as above), drew the form, added a 'Close' thing, a 'Spin' thing, a 'Parameter Bar' thing and a 'Number' thing, linked the spinner with *cooktime* and the other two with *gasmark*, and ran it. Naturally it worked first time, because there was almost nothing to go wrong!

Design Philosophy: 2 – Forms follow Functions

What we mean is that we like the way APL functions call other functions (with arguments) and get results back. We also like the way functions can have local variables. In short, a function can behave strictly as a black box, and execute with no side effects in the workspace. This makes for reliable applications, easy maintenance, and facilitates the evolutionary design process that is one of the great strengths of APL. Something a bit like this ...

```
⍺←'Adrain Smith' Gui_call sample1
Adrain Smith
```



The form definition is held in matrix *sample1*. It is called with 'Adrain Smith' as its argument by utility function *Gui_call*, which returns the corrected value ... assuming I typed correctly and hit the OK button.

The form keeps a local copy of the original text so that if I type incorrectly and hit <Cancel> it can return the original value. In the old days, I would have made a simple function such as:

```
⍵ new←fix name
[1] new←name
[2] name←input 'Enter new name'
[3] →(pname)+0
[4] new←name
⍵
```

My form is really just another way of achieving the same ends: it takes the old name as argument, copies it (so we can cancel), and returns what you did. To do something as simple as this, I should not require a function as well as a form! All normal APL rules apply, so that if you have a button on a form which makes another form, it can pass arguments and get results in just the same way. Code executing from the child form can see (and modify) local variables defined by its parent, and so on down to the globals in the workspace.

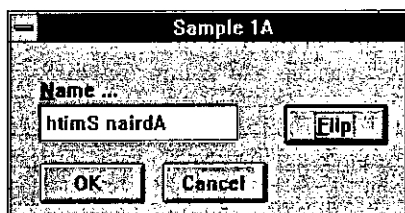
To summarise: we like the APL approach to functions, arguments and results because it lets us build complex systems from simple (tested) components with no unexpected side effects. We want the same behaviour from our Windows systems, which may be a lot more complex than anything we have yet built.

Design Philosophy: 3 – Events Trigger Actions

This idea follows on quite logically from the way we always used to program our old mainframe systems! How many of you have written code like this ...

```
Wait:PF+GDDMΔWAIT
→(Add,Delete,Save,Error)[1 2 3:PF]
Add: do this ◊ →Wait
Delete: do that ◊ →Wait
Save: Saveit ◊ →Done
Error:GDDMΔBEEP ◊ →Wait
```

The only “events” you needed to handle were 12 Pfkeys, <Clear> and <Enter>, so a simple multi-way branch was fine. Under Windows, life is far richer, but potentially far more complex. Trying to write similar code which unscrambles the results of a ‘wait’ (in Dyalog this is a `[DQ ' ']`) results in an enormous and unreadable main function, which could easily run to several hundred lines [3]. The alternative is to specify, for each object in your dialogue box, a table of events that interest you, and the APL code to be executed when each of them occurs. Here is *sample1* again, slightly modified to illustrate the idea:



You can guess what happened when I hit the ‘Flip’ button! Well, buttons only report a couple of interesting events, and in this case all we want to hear about is a ‘Select’. The dialogue-box defines an entry which flips *name*, and also lets the edit field know that its linked variable has changed.

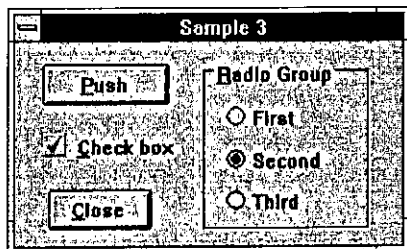
show sample1a

Ty	Caption	Pos	Size	Sty	Ref	Data	Listen	Event-Action Table
FM	Sample 1A	320 156	120 272	{Dbx}				
TX	&Name ...	44 20	25 136			name	name	
AC	&Flip	44 188	28 72	{ }				SL name+⊕name {name}
EN	OK	84 20	28 72					
QT	Cancel	84 104	28 72					

Once again, no programming is needed, or at least you could say that the dialogue-box is the program!

Design Philosophy: 4 – We Define the Objects that WE Need

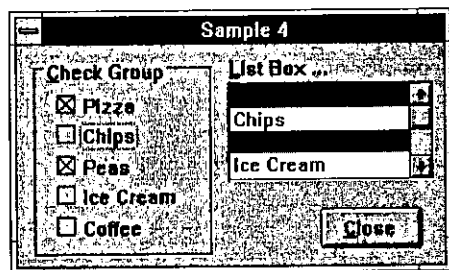
When I say “we define” what I really mean is “Microsoft did not get everything right”. Nearly every Gui-builder package that we have seen slavishly follows the Microsoft lead and has three sorts of buttons, but functionally there is almost no similarity between ‘Push’ buttons, ‘Check’ buttons and ‘Radio’ buttons. Here is what might be useful to an application builder:



The ‘Push’ button is the same as the Windows kind ... it reports a ‘Select’ event when pressed, and I should execute a piece of code. Easy.

The ‘Check’ is typically linked to a boolean scalar. They rarely trigger actions, so have very little to do with the first kind.

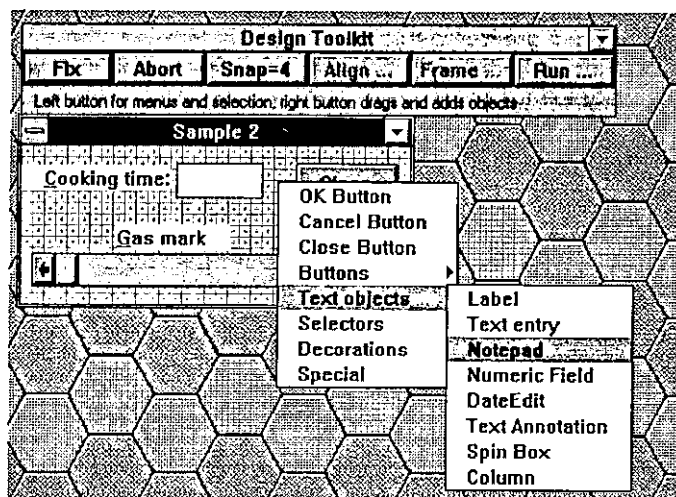
Radio buttons only make sense in groups, so why not just make a ‘Radio Group’ object which is linked to two variables: an option vector (‘First’ ‘Second’ ‘Third’ in this case) and an integer which tells it which one is selected (2 in this case). Quite soon, you run out of screen space, but this is no problem, as you simply switch from a radio-button group to a ‘drop-list’. In the same way, you quickly find that the closest relative of a check-box group (like a multi-select radio-group) is actually a list:



Both of these objects naturally link to the same type of data: a vector of names to choose from and a vector of integers (1 3 in this case) which marks the picked elements. It is simple for the programmer to choose the most appropriate way of representing the data – without making any changes to his APL code.

We could go on – Windows offers you an apparently innocuous choice between ‘single’ and ‘multiline’ edit fields, yet there is a difference in kind between simple text entry (such as I used a few pages back) and a ‘notepad’ which is a full scale word-wrapping editor, and naturally watches a vector of text vectors! Let’s make a collection of objects sensibly classified by what they do, and the kind of data they watch:

Just to give you a flavour (the appendix lists the full range of objects) here is a screen-shot of our designer, offering to add an item from its range of 'texts':



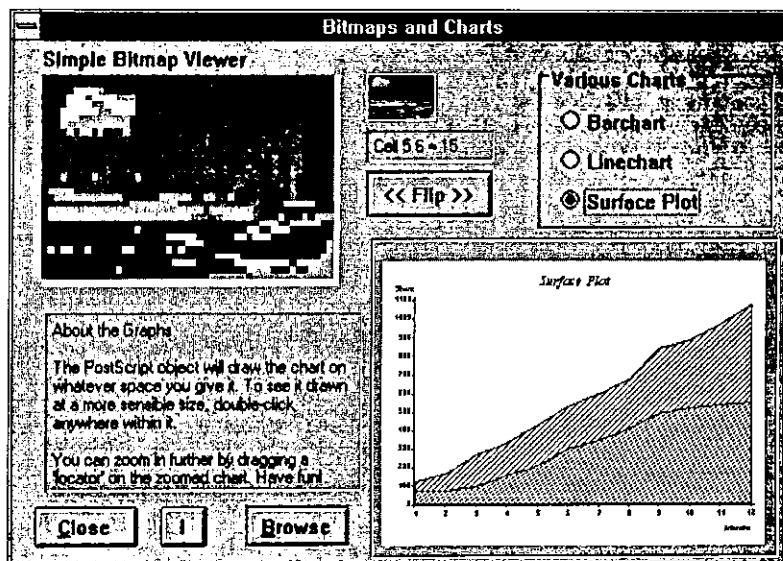
The designer does the obvious things – you can add and delete objects, drag them around, stretch them and shrink them and snap them to the grid of your choosing. It also does some less obvious things, like anchoring objects to a given edge (so you can stick your 'Close' button to the bottom right corner) or to the height/width of the form (so your notepads grow when the user resizes the form). It is good at aligning, cloning, framing, and just generally making the finished product look professional, all with the minimum of effort!

Here is the dialogue box it drew:

show sample2

Ty	Caption	Pos	Size	Sty	Ref	Data	Listen	E.A.T.	Id
FW	Sample 2	320 156	112 272	{Dbx}					
SP	&Cooking time:	12 108	25 61	{side}		cooktime			
PA	&Gas mark	76 8	24 196	{ }	1 9	gasmark	gasmark 0		
NM		76 212	24 48			gasmark	gasmark 0		
CL	&Close	12 192	28 72						

One final example, which is really just showing off:



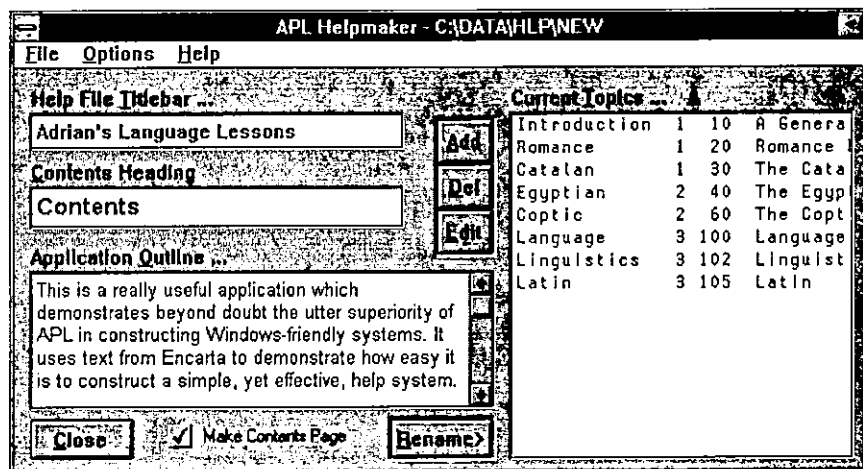
Have a look at the object table in the Appendix, and see how many of these you can identify. I suppose I should repeat (because I still find it hard to believe) that this entire screen requires almost no code, other than schoolboy APL such as $bits \leftarrow \phi bits$ which is what the << Flip >> button does when you hit it. Fancy stuff (like the recessed label which tracks the colour value as the mouse moves over the bitmap) was almost too easy; note the 'Mouse-Move' action which sets a local variable to the row/column position of the mouse (in a co-ordinate system which matches the bits in the bitmap). All the label needs to do is link to a simple expression which reports these numbers, and the $(coord) \triangleright \Delta beach$.

DISP dbx_psbk[sel;1 2 3] * Selected objects only:

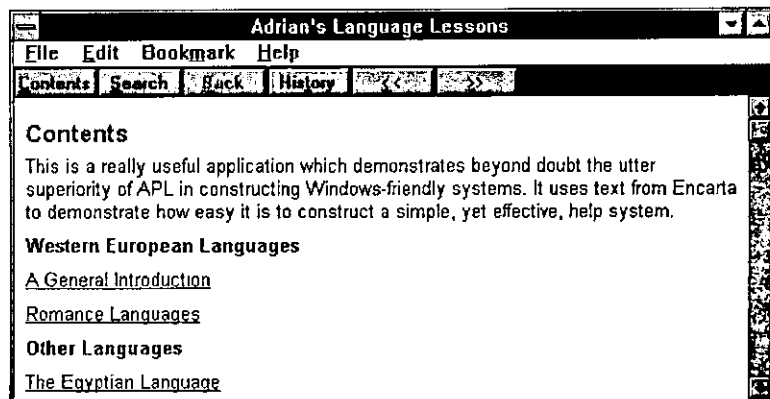
FM	Bitmaps and Charts;cht;cell	CR cht+1 o cell+1 1 {}
BK	Simple Bitmap Viewer	NW cell-[w[3 4] *w is the message {cell}
AC	<< Flip >>	SL abeach- ϕ abeach {abeach}
AC	xBrowse	SL tmp-[TC[3] Δ csvton Δ pg*,cht o ted Δ tmp {}

A Practical Example: Building Windows Help Files

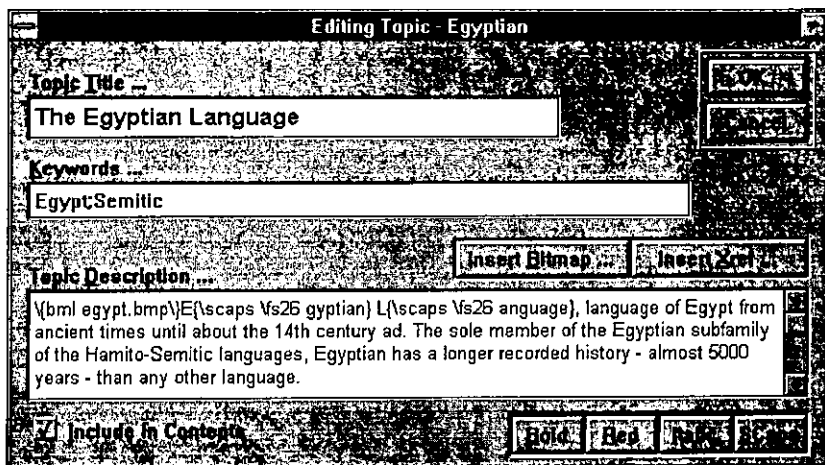
Just as a way of rounding this off, here are a couple of screens from a genuinely useful application that took all of a Saturday morning to build:



I finally got sick of trying to make help files in Winword (keeping track of the jumps quickly covers your screen in Post-It notes) and spent a couple of hours understanding what it actually wrote to that .RTF file. Then I made a little function to convert APL data to .RTF (easy) and I thought a little about how I wanted my help files organised. Then I got hacking. The screen above shows the overall structure of the help system ... you can add, remove, rename and group topics. Here is the Contents page it made for me:

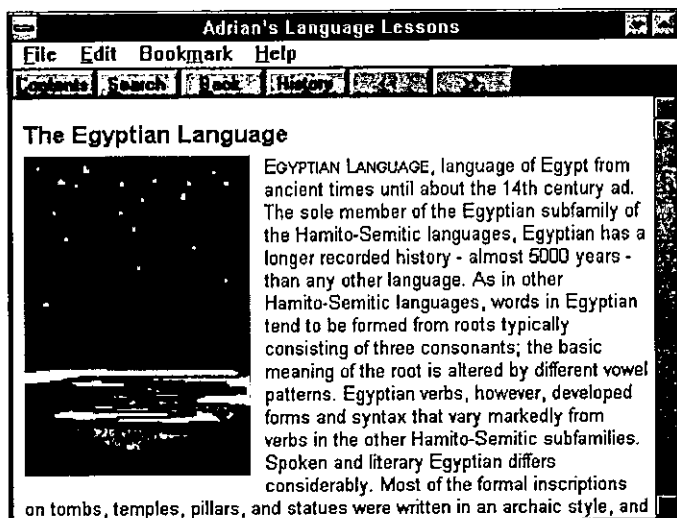


Double-click on a topic in the APL Helpmaker and:



... the funny stuff in the text is what you need to put there to make your RTF turn into a nice help file! It was all inserted for me by the buttons above and below.

Run the output from this through the SDK help compiler, hit Egypt, or search for 'Egypt' or 'Semitic' and hey presto:



What does this tell us about GUI design? The fact that I built the guts of it in a day (and I am slowing down, unlike Duncan) says a lot for the ergonomics of the designer, and for the choice of objects in that list a few pages on. The fact that it virtually all worked first time (once the typos were cleared) and has been utterly reliable during some pretty tough use, suggests that the basic idea of objects which watch data and trigger actions is the right way to build things.

What else might be important? Perhaps it runs a touch slower than the same screen built in raw $\square WC$, but I would never have built something that looked that good in a week, never mind a morning. For simple dialogues, I can honestly say that speed is not an issue, and that is running on my dear old PS/2-70 at work, not my super-whizzo Elonex at home!

Coming in Next Issue

How it all works. The dialogue-box matrix defined in detail, and every nut and bolt of the object-prototype matrix laid out before you! The mechanics of localisation, and a blow-by-blow account of exactly what happens when you change a value on a form over here, and the same variable is linked to three other things on another form over there!

Yes, it was hard going, but it feels like the best APL environment we have ever had, and it may just give us a fair run against the Visual Basics and Power-Builders which all claim to do this sort of thing, then let you down badly when the going gets tough.

References

- [1] Adams, Martyn. "*APL Experiences and Microsoft's Visual Basic for Windows*" Vector Vol.9 No.4 pp.100-112
- [2] Clark, Ian. "*APLomb: the view through QUAD-shaped spectacles*", Vector Vol.10 No.3 pp.41-50
- [3] Eastwood, David. "*Working with Windows*", Vector Vol.8 No.1 pp.102-103
- [4] Microsoft Encarta (1994 edition) "*Languages*"

Appendix: the Object Catalogue at 18th March 1994

Id	Description	Primary Data	Reference Data
FM	Parent Form	Icon (bits array or 'file')	Menu structure
RU	Roll-up toolbox	Bitmap backcloth ('file name')	(unused)
EN	OK Button	(unused)	(unused)
QT	Cancel Button	(unused)	(unused)
AC	Action Button	Bitmap to place on button	(unused)
CL	Close Button	(unused)	(unused)
VC	VCR Buttons	Current Index	Reference list to step through
SB	Scroll Bar	Thumb (integer 1..max)	Range, step, large step
PB	Picture Button	Bits array or 'bitmap file'	(unused)
LB	Label	Label value	(unused)
AN	Text Annotation	Panel contents	(unused)
TX	Text entry	Text variable	Format ('XXXX' is w/c 4 long)
NM	Numeric Field	Data (single number)	Format Picture (e.g. 'ZZ9.99')
DT	DateEdit	Data variable (integer date)	(unused)
SP	Spin Box	Data (number, or index to list)	Increment or Reference List
NP	Notepad	Data (vector of text vectors)	(unused)
YN	Yes/No choice	Data (boolean singleton)	(unused)
GR	Radio Button Group	Data (selected indices)	Reference List (button labels)
CK	Check Box Group	Data (picked values)	List (button labels)
LS	List Box	Data (selected values)	List (listbox contents)
SL	Drop-down Selection	Data (Index)	List (drop-down contents)
CB	Combo selector	Data (text)	Reference List (alphabetic)
DC	Drop-edit Combo	Data (text)	Reference List (alphabetic)
PA	Parameter Bar	Data (number)	Reference (min,max)...
IC	Icon Holder	Bits or 'file name'	(unused)
BM	Bitmap Holder	Bits or 'file name'	(unused)
AT	Artistic Text	Text to show	Font Specification
PS	PostScript Graphics	Data to draw (PostScript code)	(unused)
BK	Backcloth	Bits array	Placement (Tile,Stretch)
PI	Progress Bar	Progress to date (current,max)	(unused)
GV	Grooved Rectangle	Depth (negatives give ridges)	(unused)
PL	Plinth or Recess	Depth (negatives give plinths)	(unused)
MA	Matrix	Numeric matrix	Format (as APL thorn, default is 8 2)

This is the most readable part (columns 1,2,10) of our class table, which is used by the designer to prompt for data, make Cue-cards and offer the choice of objects shown on the previous page. The rest of this table has the code to make them, update what you see on the screen, and retrieve what you typed, or clicked or dragged. This is what we shall be looking at in the next issue!

What's in a Name?

by Sylvia Camacho

The problem of finding a record by name is familiar to anyone who has set up a system to manage a membership or a customer list. Names are not unique and are often misheard and mis-spelled. Finding the first match and then presenting successive records to allow a choice to be made on the basis of the other details is an easy way to handle common names but the onus is on the keyboard operator to get the name exactly right and to cross-check with other details until completely satisfied that a match has been found. I am sure mine is not the only family with several members having accounts at the same branch of a bank to have found we are paying each others' standing orders.

A more challenging case is to find a record which is believed to be in the database but an exact match for the supposed name cannot be found. The member or customer may not be on hand, may not speak good English or may not have legible handwriting. In such circumstances the operator could make successive guesses or try to set up a search on another field for which some details are known like address or post code. Even better would be to be given a list of near matches.

I found myself in need of some such retrieval functions recently when setting up a customer database for a shop selling TV and white goods on hire purchase or rental. Many customers do not know their account number and a facility to enter the first few characters of their surname and retrieve a list of near matches allows a quick response. The database holds the list of names as a matrix and uses it for sorting before printing. I decided, therefore, to use the sort matrix to control the sequence in which records would be put to the screen according to the 'goodness of fit' of the matrix name to the name input, which might be only the first few characters.

My first attempt was the function *match* below which compares the key input with the same number of columns down the matrix of names and returns the 2 decode of the resulting booleans. This gives more weight to a match closer to the beginning of the names but, of course, the sequence of the key must be an exact match over those columns selected by the length of the key. It is probably faster to confine the decode to those names having the correct initial letter as shown. One would normally hold the names matrix as upper case to save conversion on retrieval. The result is a numeric vector which is used to re-order the database index before the records are displayed. By this function Smith is more closely

matched by Smythe than it is by Smillie but the column positions of characters must be the same for a match to contribute to the weighting, and most spelling variations do not have this property.

```

vmatch[0]v
  v r+names match key;boo;s
[1]  a returns weighted match
[2]  names+ucase names
[3]  s+pkey+clip ucase key
[4]  boo+names[:0]io]=itkey
[5]  r+names[boo;1pboo;1s]=((+/boo),s)pkey
[6]  r+boo\21&r
  v
    names[!names match 'camacho'];]
Camacho
Camaccio
Comacho
Comanche
the remaining names, having a zero weight, are not shown

    names[!names match 'mahomet'];]
Mahon
Mahmoud
Macleod
MacIntyre
Mohamet
Mohammed
McCloud
McIntire
Moon
and after this zero weighted

```

Then I remembered reading in a Zark Newsletter about a coding system based on the sound rather than the spelling of names. It was devised to resolve problems with early US Census returns where the large immigrant population Anglicised their names with very variable results. It has also proved useful for doing genealogical research where spelling is less standardised and varies over the generations. The principle is to group the letters of the alphabet according to similarity of sound. Those familiar with Pitmans shorthand will recognize this idea. The set of vowels and near-vowels is assigned to Group 0 and are only taken into account as spacers during the assessment of consonants, being omitted from the final code. The grouping is as follows:

0	A E H I O U W Y
1	B F P V
2	C G J K Q S X Z
3	D T
4	L
5	M N
6	R

To obtain the code for a name, case is ignored and spaces and non-alphabetic characters are compressed out. Then the group number is substituted for each character. Contiguous duplicate numbers are compressed out and the first number is dropped and the first character of the name is restored in its place. Finally all zeros are compressed out and the code truncated to the alpha plus three digits with trailing zero fill if necessary. The result is called the *Soundex Code* for the name. Applying this algorithm to my list of names and using the upgrade of the result to order them should bring names close together which are similar in sound although not in spelling. Thus *MacLeod* will have the same Soundex code as *McCloud*. However the initial alpha has an overriding effect as shown by the second of the namelists below. If as is seen in the third column the initial digit is retained instead of substituting the first character then *Feezy* can be matched with *Pheasey*.

Of course one has to accept that many more possible matches will be thrown up by the use of Soundex and this may be difficult to manage for a large database. Zark suggests that the return from the *soundex* function should be made numeric by using the index of the initial character in the alphabet as the start of the Soundex code. The Zark article was presented as a competition to find the fastest version which my attempt below will certainly not be. Has anyone seen the winning one?

```

▽soundex[[]]▽
  ▽ codes←soundex names;vector;alph;nums;s;ab
[1]  a see Zark News 1993 Quarter 2
[2]  names←ucase leftjustify(1 0 [-2+pnames])pnames
[3]  vector+,'*',names
[4]  alph+'AEHIOUWYBFPVCGJKQSZDTLMNR*'
[5]  nums+'0000000011112222222334556*'
[6]  vector+(vectorαalph)/vector
[7]  codes←nums[alph\vector]
[8]  codes←(codes*-1φcodes)/codes
[9]  codes+,'*', 0 1 + '0' vim codes
[10] s←1+φcodes+'0' vim(codes*'0')/codes
[11] codes←names[;[]io],(s,3)+codes
  ▽

```

vim is a slightly amended version of the Sharp *vtom*

alphabetic	Soundex	Soundex, no alpha	
Barclay	Barclay	B624	Feezy 1200
Berkeley	Berkeley	B624	Pheasey 1200
Braun	Braun	B650	Barclay 1624
Brown	Brown	B650	Berkeley 1624
Camaccio	Camaccio	C520	Braun 1650
Camacho	Camacho	C520	Brown 1650
Comacho	Comacho	C520	Shaksper 2216
Comanche	Comanche	C552	Shakespeare 2221
Feezy	Feezy	F200	Camaccio 2520
Graeme	Graeme	G650	Camacho 2520
Graham	Graham	G650	Comacho 2520
Macleod	Macleod	M243	Smith 2530
MacIntyre	McCloud	M243	Smythe 2530
Mahmoud	MacIntyre	M253	Smiley 2540
Mahon	McIntire	M253	Smillie 2540
McCloud	Mahon	M500	Comanche 2552
McIntire	Moon	M500	Graeme 2650
Mohamet	Mahmoud	M530	Graham 2650
Mohammed	Mohamet	M530	Yates 3200
Moon	Mohammed	M530	Yeats 3200
Newell	Newell	N400	Macleod 5243
Newhall	Newhall	N400	McCloud 5243
Pheasey	Pheasey	P200	MacIntyre 5253
Reiter	Reiter	R360	McIntire 5253
Rider	Rider	R360	Newell 5400
Ryder	Ryder	R360	Newhall 5400
Shakespeare	Shaksper	S216	Mahon 5500
Shaksper	Shakespeare	S221	Moon 5500
Smiley	Smith	S530	Mahmoud 5530
Smillie	Smythe	S530	Mohamet 5530
Smith	Smiley	S540	Mohammed 5530
Smythe	Smillie	S540	Reiter 6360
Yates	Yates	Y320	Rider 6360
Yeats	Yeats	Y320	Ryder 6360

Wildcard Matching

The method of searching against an incomplete key which is included in DOS uses * and ? as wild cards. This technique is available as an ISIAPL toolkit function called wildcard which returns a boolean vector showing the rows of the matrix for which a match is found. The search argument is summarised below.

- x Select exact match on x.
- x* Select names starting with the sequence x.
- *x Select names ending with the sequence x.
- x*y Select names starting with x and ending with y.
- *x* Select names containing the sequence x.
- * Select all names.

In summary, '*' matches an arbitrary sequence of any length, including 0. Also, the special character '?' is permitted within <x> or <y>, and has the significance of matching any single non-blank character.

```

(names wildcard 'C*cho')/names
Camacho
Comacho
(names wildcard 'Cam*')/names
Camaccio
Camacho
(names wildcard '*Int*')/names
MacIntyre
McIntire

```

There is a very similar and somewhat more flexible facility in *Microsoft Access*: see the specification below. Of course this only allows one record at a time to be found so one has to step through the file using the **Find Next** button. In order to pull out all matches one would have to write some SQL and that involves reading the manual!

The asterisk (*), question mark (?), number sign (#), exclamation point (!), hyphen (-), and brackets ([]) are wildcard characters. You can use these characters in queries, commands, and expressions to include all records, file names, or other items that begin with specific characters or match a certain pattern. You can also use wildcard characters and matching characters to further refine a search.

- * wh* finds what, white, and why.
 *at finds cat, bat, and what.
 Like the MS-DOS asterisk (*) wildcard character, this asterisk matches any number of characters. But unlike MS-DOS, it can be used as the first or last character in the character string.
- ? b?ll finds ball, bell, and bill.
 Like the MS-DOS ? wildcard character, this symbol matches any single character.
- # 1#3 finds 103, 113, 123.
 Matches any single digit.
- [] b[ae]ll finds ball and bell but not bill.
 Matches any single character within the brackets.
- ! b[!ae]ll finds bill and bull but not bell.
 Matches any character not in the list.
- b[a-c]d finds bad, bbd, and bcd.
 Matches any one of a range of characters.

Is anyone prepared to amend the Sharp wildcard function to meet the MS Access specification? Or maybe, of course, to better it? Happy hunting!

Drawing the Line

by George MacLeod

As I proceed with the developments of my business graphics system which is now called *Entropy Business Graphics* (the Trade Mark process is in motion) I realise the serious limitations on drawing lines using the Windows graphics tools. There are essentially two problems:

1 – Limitation on Line Styles

A Windows polyline is drawn with a "pen" and a pen has one of six different styles. These are solid, dash, dot, dot-dash, dot-dot-dash and null (or invisible). A pen also has a width in pixels. Unfortunately if the width of the pen is greater than one pixel then only the solid line style is available. This is an annoying limitation on the screen but colour can be used to give line variety.

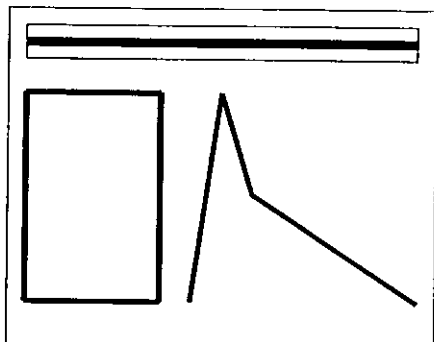
Printing is really the problem. Most people use monochrome printers and so colour is not available for printed output. A line that is only one pixel wide at 300 dpi is very fine and when dotted can be lost during any photocopying of the output. Different thicknesses of solid lines can be drawn to differentiate lines but this is rarely satisfactory.

I therefore set out to create APL functions that would calculate and draw the filled circles and polygons necessary to produce the five visible styles at any width. This requires some rudimentary trigonometry and a little coding effort. Very soon I came to the question of what do you do at the end of a line segment? Sometimes the segment will finish with part of a dash. Windows is quite clever in that it starts the next segment (if there is one) with the rest of the dash giving the effect of the dash being bent at the data point. This is effective with a one pixel wide line but would be an added complication with a wider line.

2 – Line Caps

The problem of segment joins exists even with solid lines. When you draw a number of Windows polylines of different thicknesses you find that their ends suddenly become crudely rounded for lines greater than five pixels wide. This is because all Windows polylines have *Line Caps*. For lines whose width is five pixels or less the line caps are square and above five pixels they are round or semi-circular. The purpose of line caps can be seen in the following three squares.

Solid Lines 1.



LSTYLE 0

CAPS 0

SQUARE Ignored

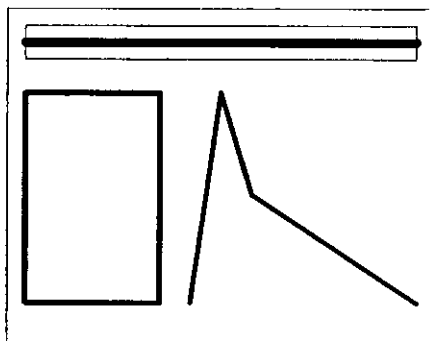
DASHENDS Ignored

DASHSIZE Ignored

GAPSIZE Ignored

This combination is usually unsatisfactory because of the notches where the line segments join. It is useful for single segments where it is necessary for the segment to be the specified length.

Solid Lines 3.



LSTYLE 0

CAPS 2

SQUARE Ignored

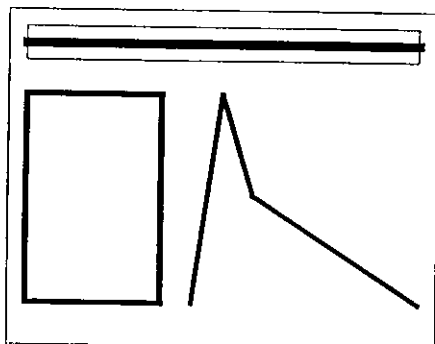
DASHENDS Ignored

DASHSIZE Ignored

GAPSIZE Ignored

This combination has round caps which are good for all joins.

Solid Lines 2.



LSTYLE 0

CAPS 1

SQUARE Ignored

DASHENDS Ignored

DASHSIZE Ignored

GAPSIZE Ignored

This combination has square caps which are very good for 90 degree joins but are less satisfactory for joins at other angles.

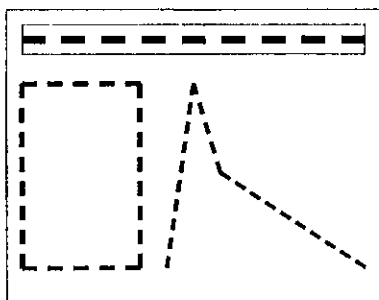
Each square is drawn with lines of the same length and width. The first square is drawn with lines with no line caps, resulting in the notches at each corner. The second square has round line caps. The line is drawn to the specified length but a semi-circular cap has been added to each end of the line producing a square with rounded corners. This kind of cap produces a good join regardless of the angle between adjacent segments. The third square has square line caps. Each line is extended by half the width of the line at each end and the result is a very good square. This technique is less effective if the join between line segments is other than at right angles.

The type of cap required and whether one is required at all depends on the object being drawn, and the Windows approach is very limited. Joining solid lines requires control of line caps but joining broken lines has other possibilities.

Changing the Gap Size.

If a dashed line is used to join two points it seemed to me initially desirable that the line should start and end with a complete dash. This can be engineered by slightly adjusting the size of the gaps between dashes.

Dashed Lines 3.

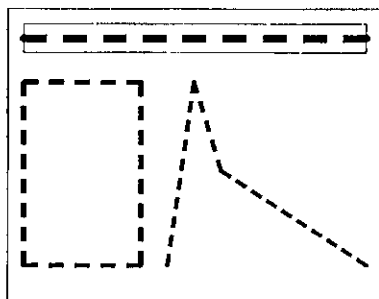


LSTYLE 1	DASHENDS 1
CAPS 0	DASHSIZE 3
SQUARE Ignored	GAPSIZE 2

Setting DASHENDS to 1 causes there to be a complete dash at each end of a line segment. This means that round and square caps are possible.

This combination has no caps which causes notches at segment joins and means that it is usually unsatisfactory.

Dashed Lines 5.



LSTYLE 1	DASHENDS 1
CAPS 2	DASHSIZE 3
SQUARE Ignored	GAPSIZE 2

Setting DASHENDS to 1 causes there to be a complete dash at each end of a line segment. This means that round and square caps are possible.

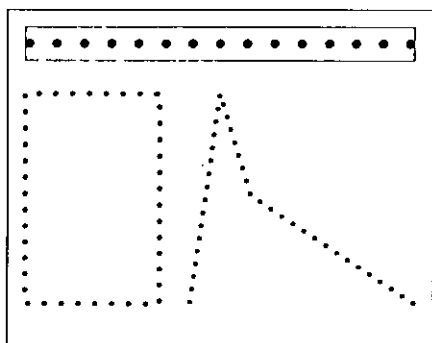
This combination has round caps which makes it generally acceptable.

If the segment is to have square line caps then the line is drawn half a width longer at each end. This may mean an additional dash and possibly a shrinking of the gap size, or alternatively the gap size can be expanded to extend the line. The better alternative each time is the one that requires the least tampering with the gap size.

A Realisation

Dashes are easy to comprehend but what about dotted lines? A dot can be a circle or a square. If the line is not very wide it does not matter too much which, but if the line is wider then circular dots look better. Windows graphics tools include an ellipse but no circle. Happily Dyadic have provided a circle amongst their graphical objects. (A circle differs from an ellipse in that its shape is preserved when the object containing it is re-sized.)

Dotted Lines 1.



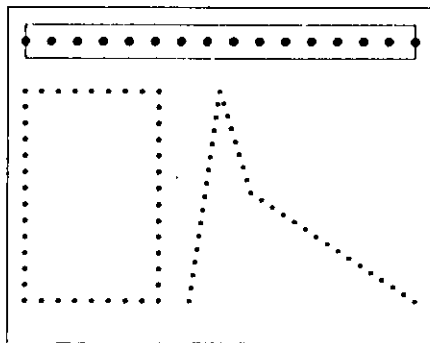
LSTYLE 2
CAPS 0
SQUARE 0

DASHENDS ignored
DASHSIZE ignored
GAPSIZE 2

In this example SQUARE is set to zero causing the dots to be round.

This combination has no caps which means that the dots at each end of the segment are not centred on the data points.

Dotted Lines 2.



LSTYLE 2
CAPS 1 or 2
SQUARE 0

DASHENDS ignored
DASHSIZE ignored
GAPSIZE 2

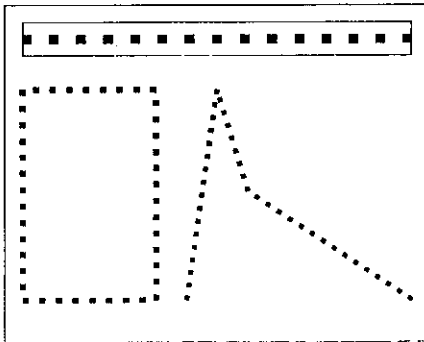
In this example SQUARE is set to zero causing the dots to be round.

This combination has caps which means that the dots at each end of the segment are centred on the data points.

There is a small problem with the circle and that is the centre is always on a pixel. This can sometimes have the effect of making a line consisting of dashes and round dots look as if the dashes and dots are not lined up properly. This only happens when the line is only a few pixels wide, in which case a square dot is more satisfactory.

The idea of a line of round dots having square caps is rather daunting until we realise that the definition of square caps is the extension of each line segment by half a width at each end and it does not matter what the line comprises. If we adjust the gap size of a dotted line to ensure that it has a complete dot at each end, and we extend the line by half a width at each end, then the dot at the end of a segment will be in exactly the same place as the first dot in the next segment. This is good if the dots are round but not if they are square (the squares will usually have a different orientation).

Dotted Lines 3.



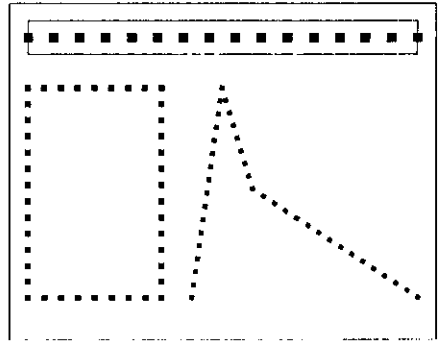
LSTYLE 2
CAPS 0
SQUARE 1

DASHENDS Ignored
DASHSIZE Ignored
GAPSIZE 2

In this example SQUARE is set to one causing the dots to be square.

This combination has no caps which means that the dots at each end of the segment are not centred on the data points.

Dotted Lines 4.



LSTYLE 2
CAPS 1 or 2
SQUARE 1

DASHENDS Ignored
DASHSIZE Ignored
GAPSIZE 2

In this example SQUARE is set to one causing the dots to be square.

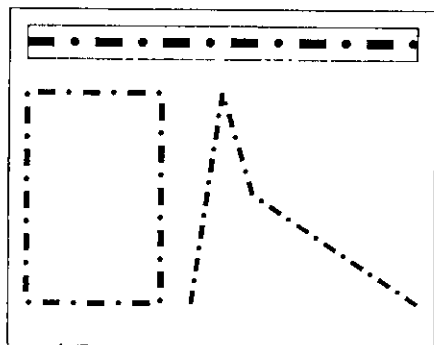
This combination has caps which means that the dots at each end of the segment are centred on the data points.

This problem is solved if we omit the first dot in the subsequent segment. This shows up another difficulty. If the polyline we are drawing is closed, i.e. it is a polygon, then the first segment can be thought of as subsequent to the last segment and it too needs to have its first dot omitted.

Dot-Dash and Dot-Dot-Dash Lines

These are interestingly different. We want a line to start and end with a complete element, either a dot or a dash, and perhaps to be extended to form square caps. In this circumstance where elements at the join may not be the same a different technique is required. We would also like the pattern of dots and dashes to continue unchanged into the next segment. We cannot therefore omit the first element of a subsequent segment but must instead indent the first element. I have tried different sizes of indent and the one that works the best seems to be an indent equal to a gap plus half a width.

Dot Dash Lines 1.



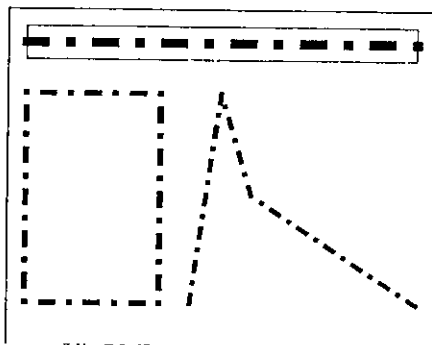
LSTYLE 3
CAPS 0
SQUARE 0

DASHENDS Ignored
DASHSIZE 3
GAPSIZE 2

In this example SQUARE is set to zero causing the dots to be round.

This combination has no caps which means that the segments appear not to be long enough.

Dot Dash Lines 4.



LSTYLE 3
CAPS 1 or 2
SQUARE 1

DASHENDS Ignored
DASHSIZE 3
GAPSIZE 2

In this example SQUARE is set to one causing the dots to be square.

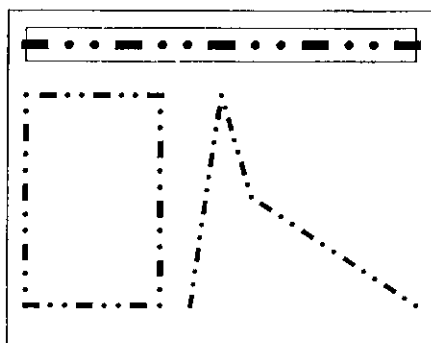
This combination has caps which means that the segment is extended by half the width at each end.

Dashes at the End

I have described this topic in the sequence in which I developed the code omitting the cul-de-sacs that I visited on the way, including for instance drawing part-dots at segment ends which looked very nasty. There was however one facility which I had before I solved the problem of joining broken lines, and that was the facility to force dashes at each end of a dashed line and apply traditional

square or round caps as if it were a solid line. This is good for drawing rectangles, for instance axis lines, so I made provision for this facility.

Dot Dot Dash Lines 2.

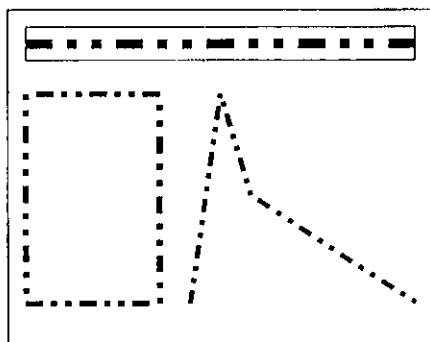


LSTYLE 4	DASHENDS Ignored
CAPS 1 or 2	DASHSIZE 3
SQUARE 0	GAPSIZE 2

In this example *SQUARE* is set to zero causing the dots to be round.

This combination has caps which means that the segment is extended by half the width at each end.

Dot Dot Dash Lines 3.



LSTYLE 4	DASHENDS Ignored
CAPS 0	DASHSIZE 3
SQUARE 1	GAPSIZE 2

In this example *SQUARE* is set to one causing the dots to be square.

This combination has no caps which means that the segments appear not to be long enough.

The Implementation

Having got all the lines working I then thought it would be interesting to make it look like a new graphical object in Dyalog APL/W. So I invented a compound graphical object consisting of up to four real objects and which has the properties of a *POLY* object plus five more. These are *CAPS* to control line capping behaviour, *GAPSIZE* to specify the size of the gaps between elements (prior to any adjustment), *DASHSIZE*, to specify the length of any dashes, *SQUARE* to control whether dots are round or square and *DASHENDS* which can be used to force a dashed line to have dashes at each end. *GAPSIZE* and *DASHSIZE* are specified as integer multiples of the line width property.

This object I called an *Enhanced Polyline* with a *TYPE* of *EPOLY*.

The four objects of which it can consist are as follows:

1. An invisible polyline which is used to contain any filling (solid colour, hatching or bitmaps) specified by the *FSTYLE* property. This object uses its *DATA* property to keep all of the specified properties of the *EPOLY* object together with any specified user data.
2. An optional solid line drawn without caps in the specified background colour of the *EPOLY* line. If none is specified then this line is not drawn.
3. An optional *POLY* object consisting of, sometimes many, rectangular polygons. These are solid lines, or the dashes and square dots which comprise the *EPOLY* line. If none of these is required the line is not drawn.
4. An optional *CIRCLE* object consisting of, sometimes many, filled circles. These are round dots and round caps. If these are not required the line is not drawn.

Performance

Performance can be a problem. Obviously a great deal more work is involved in computing all the rectangles and circles compared with just drawing a specified line. There is also a surprising amount of work involved in parsing the Dyalog-type syntax and this can be eighty percent of the effort in drawing a single segment line. It is less significant with disjoint lines with a number of segments.

The implementation consists of three user functions *QUAD_WC*, *QUAD_WG* and *QUAD_WS* which behave like their system counterparts plus *QUAD_EX* which can be used to expunge the constituents of an *EPOLY* object.

I am now working on a simpler and more limited syntax which speeds up the process. The situation is that the performance is OK on the screen using my 486 machine. I developed the *EPOLY* object for the purpose of producing high quality printed output and in this circumstance the time taken to send the bitmap output to the printer far outweighs the processing time and so performance is not a problem.

Resolution

The relatively coarse resolution of the screen can have some odd effects. Because of rounding problems segments of a line at different angles can have different thicknesses when drawn on the screen. This does not happen on a laser printer as the resolution is so much greater.

Dragging

As the *EPOLY* object consists of a number of real objects it cannot be dragged around the screen. The *LOCATOR* object can be used to provide similar facilities.

Using the Clipboard

We can use the Clipboard to transfer pictures using the *EPOLY* and other graphical objects into Windows-based word processors but the interface is not as good as it might be. The graphs have to be drawn on a *BITMAP* object (perhaps a very large one) and then placed on the Clipboard. It can then be pasted in by the word processor and will appear on the screen. The difficulty is that when the page is printed the word processor (or perhaps the printer driver) makes the bitmap have the appropriate resolution by replicating pixels. This results in a loss of quality in the printed output.

This problem could be resolved if APL/W were to have a *METAFILE* object and the *CLIPBOARD* were to have a *METAFILE* property. This would enable the word processor to produce the picture at the appropriate resolution without pixel replication. It would also remove the necessity of having sufficient workspace to hold large bitmaps.

Conclusion

This development was a good example of APL being used to build something where the design developed throughout the time the code was being written. When I started this task I had not realised the nature of the problems that would arise and how some of my ideas would look scruffy when seen on the screen or when printed.

Since writing this paper I have talked to Adrian Smith whose mind has also been dwelling on these matters. One of his concerns was dealing with very short dashed-line segments where, for example, the segment is shorter than a dash, or perhaps shorter than a dash plus a gap. The simple solution for the former situation is to either make the segment blank (which is what I do) or to make it a solid line. The trouble with leaving it blank is that if there is a series of short segments, then nothing appears on the page! Adrian's view, which I think is probably best, is to use part-dashes, so that the same dash may be split between two or more segments (which is what PostScript does). I will probably adopt this thinking, having worked out how it works with lines including both dots and dashes.

AMORTIZE: a Windows Application in J

by Thomas M. Olsen

Abstract

A simple application for Windows is described, assuming some previous exposure to J. The message loop verbs are explained and may serve as a departure point for further development. Execution speed is compared for tacit and explicit definition of the numerically-intensive part of the program.

Program Description

AMORTIZE was written to gain experience with the Windows version of J and to provide a useful but simple demonstration of utility: people often borrow money or consider an annuity. The program produces an amortization schedule for a specified present value and total number of fixed payments. Multiple values of compound interest are allowed. In Windows fashion, it has an *About Box*, appropriate *Message Boxes*, and *User Help*.

The program is implemented as script files, for easy inspection and editing by the reader, and can be downloaded from CompuServe (GO CODEPORT, Library 3 (C Language), filename AMORTIZE.ZIP; PKUNZIP d-option). At the expense of brevity and with administration overhead, but consistent with object orientation, defined vocabulary could instead be encapsulated by saving locally (`[x] 2! :2 y`) and copying in when needed. Where practical, J sentences appear both tacitly and explicitly.

Message Loop

The script in Fig. 1 is run silently (`0! :3 y`) from J to start the application. If the interpreter would accept a command at invocation, this could be attached to a custom icon, via the Windows Properties feature. A double click on the icon would then suffice to start the program.

To understand the verbs `WinExec` and `WinRet` in Fig. 1, consider a typical `n by 2` array returned by the `wait` command. See Fig. 2. Row zero indicates ENTER was pushed; row one the parent window, row two the menu item or child window. If the Windows program, or subprogram, has no wait command, the result is:

*type	2 nowait
-------	----------

Here and in Fig. 2, a * indicates a system result. When given the n by 2 array, the verb WinExec extracts the name of the menu item or child window and executes, with null argument dq (double quote), a verb by that name. Any remaining 1-cells (rows) of the n by 2 array from wait are ignored until later. See Fig. 3.

```

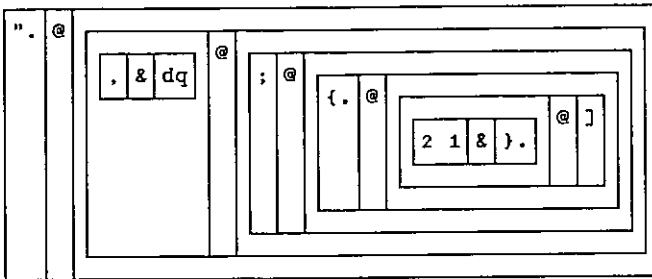
NB. c:\j\amortize\amortize.js
NB. Amortization Schedule. TMO 7 June 1993
NB. J PC386 Vers 6.1.0 was used for Windows 3.1 application
NB. Interpreter: ISI/33 Major Street/Toronto, ONT M5S 2K9/Canada
NB. Initialize to application vocabulary:
    dir=. 'amortize\'
01:3 <dir,'initial.js' NB. See Appendix A
NB. Execute "child name" from Wait:
    WinExec=. ".@(&dq@(;@({.(2 1&}.@J)))) NB. ".dq,;(. 2 1 }. y.
NB. Respond to menu select, ENTER, and ALT F4.
NB. '$.=. 0' is not equivalent to '$.=. start'
NB. A message box doesn't need a wait loop.
    m=. 'start) ". y. NB. Name parent window'
    m=. m;'loop) a=: wwait dq NB. Ignores 1st ENTER'
    m=. m,<'NB. wwait result: 4-menu; 6-close; 7-ENTER'
    m=. m,<'$.=.' ;('467'1. 1{;1{,a){menu;quit;menu;loop'
    m=. m,'menu) a=: WinExec a'; '$.=. loop'
    m=. m,<'quit) 01:55 dq NB. Immediate exit'
    WinRet=. m : ''
WinRet 'wpd 'main.wpd' ' NB. Open parent window

```

Fig. 1. Main script including message loop verbs.

*type	7 enter
*parent	amortization parameters
*id	inp3val
inp0val	1000
inp1val	6+1.6
inp2val	12
inp3val	360
inp4val	9

Fig. 2. Typical array returned by wait command.

Fig. 3. The J-verb *WinExec*.

WinRet (from Fig. 1) is the message loop:

```

start) ". y.          NB. Name parent window :
loop) a=: wwait dq    NB. Ignores 1st ENTER
NB. wwait result: 4-menu; 6-close; 7-ENTER
$.=. ;('467'i. 1{;1{,a){menu;quit;menu;loop
menu) a=: WinExec a
$.=. loop
quit) 0!;55 dq       NB. Immediate exit

```

Parent and Child Windows

When AMORTIZE is launched via the last line in Fig. 1, WinRet 'wpd 'main.wp' ', the start) sentence executes a Windows program:

```

rem c:\j\amortize\main.wp;
pc Amortize;
menupop "&File";
    menu main_file0 "&Enter Input"; menu main_file1 "&Quit";
    menupopz;
menupop "&Help";
    menu main_help0 "&About Amortize";
    menupopz;
pmove 6 15 300 200;
pshow;

```

where two menu commands map three menu items to corresponding J verbs in Appendix B; J locative names here and in Appendix B correspond to the three parent windows in AMORTIZE: **main**, **input**, and **output**. Here, **main** has the alias "Amortize". There is no wait command in **main.wp** and control returns to the J-interpreter; WinRet rests at the loop) sentence **wwait** command.

Assume the user now selects "Enter Input" from the popup menu, so the WinRet menu) sentence executes `main_file0`:

<code>wpd 'input.wp'</code>	<code>:</code>
<code>0 ShowInp 'PV'</code>	
<code>1 ShowInp 'APR'</code>	
<code>2 ShowInp 'PPA'</code>	
<code>3 ShowInp 'TNP'</code>	
<code>4 ShowInp 'PNTS'</code>	

where the first sentence opens the Input window, defined in Appendix C under the alias "Amortization Parameters", with its own popup menu items. Corresponding J-verbs for Input.wp are defined in Appendices A and B. This parent window, Input, uses two child windows for each input noun, one for a static descriptive label and the other for an edit box:

Fig. 4. Input window with data.

The first four default values in Fig. 4 would only appear if the user had entered them during the current session. J sentences can be used here for compact specification of input. Remaining sentences shown above for `main_file0` display these default nouns via the verb `ShowInp`. Appendix D defines it:

<code>:</code>	<code>\$.=. (2=4!0 <y.)#1</code>
<code>wd 'cset inp',('x.),'val;text "' ,("y.),'";'</code>	

where the suite `$.` is null unless the noun exists. The second sentence focuses on the corresponding child edit window and writes the formatted value. Finally, since `main_file0` has no wait command, control again returns to the loop) sentence of `WinRet`.

Input Validation and Error Messages

Pushing ENTER in the window shown in Fig. 4 returns the 8 by 2 array shown in Fig. 2. The cursor could have been in any of the five child windows, `inp0val` thru `inp4val`, but the corresponding J-verbs are all synonymous with `InpEval`. See Appendix B and the definition of `InpEval` in Appendix D. `WinRet` passes the 8 by 2 array to `WinExec`, which executes `InpEval`:

```
NB. ExecChk left arg sets n for err msg
start) b=. 2 1}.a
$.= (4*0 ExecChk 'PlusChk ',PV=: ,>1{b}).$.
$.= (3*1 ExecChk '0 _100 LimChk ',APR=: ,>2{b}).$.
$.= (2*2 ExecChk 'IntChk ',PPA=: ,>3{b}).$.
$.= ;(3 ExecChk 'IntChk ',TNP=: ,>4{b}).$.
$.= ;(4 ExecChk '3>',PNTS=: ,>5{b){ok;error
error) a=. WinRet 'FieldMsg ',q.('':n),q
$.= start
ok) WinRet ' 'calsched.js' ' Output 'SCHED' '
```

where the supporting verbs are again defined in Appendix D. In brief, the five sentences which conditionally redefine the suite `$.` each extract an input value and test it. The second sentence in `ExecChk` guarantees a result by returning the left argument of `".` on syntax errors:

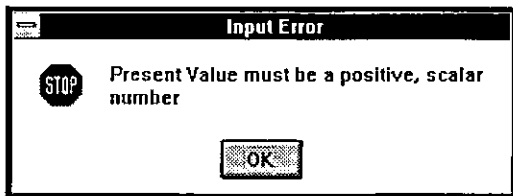
```
: n=: x.
'r=. 1' ". 'r=. ',y.
```

`FieldMsg` opens an appropriate error message box:

```
wd 'csel inp',y.,'val; cfocus; beep 3;'
wpd 'inpmg',y.,'wp'
```

where the two child commands select the offending field and move the cursor to it, respectively. The various message definitions are shown in Appendix E.

The user must respond to the message box:



Output Display

If the amortization parameters pass all tests, InpEval proceeds to calculate and display a schedule:

```
ok) WinRet ' 'calsched.js' Output 'SCHD' ' '
```

where the verb Output defined in Appendix D is:

```
: NB. Left refs script to make text named in right
01:3 (.a=. (<dir),&.> x.;'output.wp'
wp {:a
wd ('ctext "'&,@().@,@@(cr&,"1@(&,'"';')@(".@J)))y.
```

With the dir defined in Fig. 1, sentence one produces two file names,

amortize\calsched.js	amortize\output.wp
----------------------	--------------------

and silently executes the file named by the first atom, defining a pronoun, SCHED. See Appendix F. Sentence two produces the output parent window, with alias "Amortization Schedule":

```
rem c:\j\amortize\output.wp;
pc "Amortization Schedule";
menupop "&File";
    menu output_file0 "&Save";
    menu wclose "&Exit";
    menupopz;
menupop "&Help";
    menu output_help0 "&Items";
    menupopz;
xywh 0 0 267 96; cc results listbox ws_thickframe ws_hscroll
ws_vscroll; pas 0 0; pcenter; pshow;
csl results; cfoem; cfocus;
```

Here, the window style `ws_hscroll` is included but does not produce the horizontal scroll bar when needed. Sentence three of Output formats the result, `SCHED`, and writes it to the child window, "results":

Amortization Schedule						
File	Help					
	6%: 6.00	7%: 6.65	8%: 7.34	9%: 8.05	10%: 8.78	11%: 9.52
1	0.17	0.12	0.09	0.07	0.05	0.04
	1.00	0.82	0.67	0.55	0.44	0.36
	999.00	999.18	999.33	999.45	999.56	999.64
	5.00	5.83	6.67	7.50	8.33	9.17
52	0.19	0.14	0.11	0.08	0.06	0.05
	58.95	49.63	41.54	34.58	28.65	23.62
	941.05	950.37	958.46	965.42	971.35	976.38
	252.81	296.33	340.02	383.82	427.69	471.59
104	0.25	0.19	0.15	0.12	0.10	0.08
	135.36	116.78	100.22	85.59	72.75	61.58
	864.64	883.22	899.78	914.41	927.25	938.42
	489.17	575.12	662.98	751.32	840.92	930.64

Characteristically, the user is not limited to a single output window, but can focus on the input window (Fig. 4), change the data, and push ENTER again. Thus, multiple schedules can be compared side-by-side. However, no special code is included to detect such operation and, if `Exit` is commanded from any of the windows, including the input window, the last output window opened is the first window closed.

The amortization schedule algorithms in Appendix F are straightforward and can be illuminated by executing them individually. Students are encouraged to display the defined verbs.

Tacit vs. Explicit Execution

The tacitly defined algorithms for `SCHED` were interchanged with their explicit analogues, which appear as comments in Appendix F, yielding an alternate script: `xplisched.js`, shown in Appendix G. Recall that tacit definitions in J are pre-parsed. Using the sample input of Fig. 4, average times for `SCHED` only, on an IBM PS/2 77 (80486DX2, 66 MHz) are:

```
tacit definition: 286 ms (calsched.js)
explicit definition: 275 ms (xplisched.js)
```

The sentence `10 (6!2) '0!3 <dir,'__sched.js' ' ' was used to execute the two cases. The conclusion that it doesn't really matter here is qualified by the style of programming in Appendices F and G; only four of the 22 algorithms use the defined verbs payment, subset, and twofix.`

Thus, the sentence defining Interest Factor is, e.g.:

```
tacit definition (App. F): IF=. I (>:@[ ^/ -@]) K
explicit definition (App. G): IF=. (>:I)^/-K
```

Moreover, the advantage of pre-parsing is masked in the example, because of the array dimensions: nine rows by six columns; parsing occurs only once for each algorithm. A slight reduction of execution time would result from reworking the tacit-definition algorithms to avoid, in Appendix F, deeply-nested right compositions and unneeded use of @].

Implementation Notes

The Codewright (Premia Software, Inc.) text editor was convenient in developing the program. J includes Windows Visual Edit (foreign conjunction 11!:1), useful but limited to adjusting size and location of existing child windows. The multi-line message box text shown in Appendix H had to be centred left-and-right by trial and error. Hopefully, ISI or an enterprising user will eventually add a GUI development environment with toolbar icons for Windows widgets, to put J on a par with Dyalog APL/W, Visual Basic, and Visual C++.

Documentation Notes

The editable manuscript file, not printout, for this paper was composed with Windows Write, using the Arial TrueType font. On an IBM PC, the boxed items used here for illustration were cut-and-pasted from the J window and mapped directly to the MS Line Draw font using WRITE Character. Microsoft scrupulously removed all other ASCII characters from MS Line Draw and these appear as dots. But they can be highlighted and mapped immediately into True Type Courier New font. On a given boxed item, Character must be applied to each contiguous string of common-font characters.

Inclusion of illustrations requires only the Windows Clipboard. For best contrast, select the Monochrome colour scheme for the Desktop.

Acknowledgments

This paper resulted from the October 1992 J-Workshop given at Karsten Mfg. by Donald B. McIntyre, who through his boundless energy provided continuing inspiration and advice. Louis Solheim and Hadi Polynin enabled the project and the staff of Information Systems encouraged it. The J interpreter is a product of Iverson Software Inc.

Appendix A – Initialization

```

NB. c:\j\amortize\initial.js
NB. Initialize to application vocabulary:
    cr=: {. crlf=. 13 10{a.
    eof=: 26{a.
    dq=: 2#q=. '""'
    read=: 1!:1
    vd=: 11!:0
    vp=: vd@(-. &eof)@read
    vpd=: vp@(<@(&dir&,@))
    vwait=: vd@('wait; ' &[)
    vclose=: vd@('pclose; ' &[)
    vreset=: vd@('reset; ' &[)
NB. Define child name returned by wait command:
0!:3 <dir,'children.js'
NB. Define verbs referenced in child name from wait command.
0!:3 <dir,'chdverbs.js'
```

Appendix B – Verbs Mapped from Menu Items

```

NB. c:\j\amortize\children.js
NB. Child name returned by wait command points to verb:
    m=. <'vpd 'input.vp' '
    m=. m,'0 ShowInp 'PV' ' ';1 ShowInp 'APR' '
    m=. m,'2 ShowInp 'PPA' ' ';3 ShowInp 'TNP' '
    m=. m,<'4 ShowInp 'PNTS' '
    main_file0=: m : ''
    main_file1=: ('vclose dq';'0!:55 dq') : ''
    main_help0=: vpd'about.vp'&,@]
    input_help0=: vpd'parmdfns.vp'&,@]
    input_help1=: vpd'parmnavi.vp'&,@]
    inp0val=: InpEval : ''
    inp1val=: InpEval : ''
    inp2val=: InpEval : ''
    inp3val=: InpEval : ''
    inp4val=: InpEval : ''
    m=. 'fn=. <dir,'amortize.txt' '
    output_file0=: (m;'FILE_TXT 1!:2 fn') : '' NB. Replaces old file
    output_help0=: vpd'outitems.vp'&,@]
```

Appendix C – Input-Window Definition

```

rem c:\j\amortize\input.vp;
pc "Amortization Parameters";
menupop "&File"; menu vclose "&Exit";
    menupopz;
menupop "&Help"; menu input_help0 "&Definitions";
    menu input_help1 "&Navigation";
    menupopz;
pmove 60 1 198 90;
pshow;
rem Vertical size of 10 units allows space for _ underscore;;
xywh 0 0 194 70;cc inp_background static ws_border;cn " ";
```

```

xywh 6 3 77 10;cc inp0 static ss_right;cn "Present Value: ";
xywh 87 1 38 13;cc inp0val edit es_autohscroll ws_thickframe;cfocus;
xywh 6 16 77 10;cc inp1 static ss_right;cn "Interest Rates (%): ";
xywh 87 14 100 13;cc inp1val edit es_autohscroll ws_thickframe;
xywh 6 29 77 10;cc inp2 static ss_right;cn "Periods per Year: ";
xywh 87 27 17 13;cc inp2val edit ws_thickframe;
xywh 4 42 77 10;cc inp3 static ss_right;cn "Total No. of Payments: ";
xywh 87 40 17 13;cc inp3val edit ws_thickframe;
xywh 7 55 77 10;cc inp4 static ss_right;cn "Displayed-Cell Limit: ";
xywh 87 53 17 13;cc inp4val edit ws_thickframe;

```

Appendix D — Verbs for Input/Output Windows

```

NB. c:\j\amortize\chdverbs.js
NB. Define verbs referenced in child name from wait command.
NB. Display input noun if it exists:
    PNTS=: '9' NB. default fills output page
    m=. '$.=. (2=41:0 <y.)#1'
    m=. m;'vd 'csel inp',('x.),'val;ctext "'('y.),'";' '
    ShowInp=: ' ' : m
NB. Report error condition:
    m=. 'vd 'csel inp',y.,'val; cfocus; beep 3;' '
    m=. m;'wpd 'inpmgs',y.,'wp' '
    FieldMsg=: m : ' '
NB. Boolean zero if positive scalar:
    Sclar=: 1&=@#&@.
    PlusChk=: Scalar *: (*./@{0&<}) NB. Monadic fork
NB. NB. (1=#,y.)*:*./0<y.
NB. Boolean zero if atoms bounded by left arg:
    LimChk=: (+./@.)@(>"1(,"0-)) NB. Verb @ dyadic hook w/monadic hook
NB. NB. +./, x. >"1 y., "0 -y.
NB. Boolean zero if positive, scalar integer:
    PosInt=: *./@{(>=.)*. (0&<}) NB. Monadic hook in monadic fork
    IntChk=: Scalar *: PosInt NB. (1=#,y.)*:*./{y.=>y.}*0<y.
NB. Validate content of an input field:
    ExecChk=: ' ' : ('n=: x. ; 'r=: 1' " . 'r=: ' ,y.')
NB. Validate all input fields:
    m=. 'NB. ExecChk left arg sets n for err msg';'start) b=. 2 1}.a'
    m=. m, '<$.=. (4*0 ExecChk 'PlusChk ' ,PV=: ,>1{b}).$. '
    m=. m, '<$.=. (3*1 ExecChk '0 _100 LimChk ' ,APR=: ,>2{b}).$. '
    m=. m, '<$.=. (2*2 ExecChk 'IntChk ' ,PPA=: ,>3{b}).$. '
    m=. m, '<$.=. ;(3 ExecChk 'IntChk ' ,TNP=: ,>4{b}).$. '
    m=. m, '<$.=. ;(4 ExecChk '3>' ,PNTS=: ,>5{b}{ok;error'
    m=. m, 'error) a=. WinRet 'FieldMsg ' ,q,('n),q;'$.= start'
    m=. m, '<ok) WinRet ' ' 'calsched.js' ' ' Output ' 'SCHED' ' ' '
    ,
    InpEval=: m : ' '
NB. Open parent window for output and format the text:
    m=. '<'NB. Left refs script to make text named in right'
    m=. m, '01:3 {a=. (<dir),&.> x.;'output.wp' ' ;wp {a'
    m=. m, '<'vd ('ctext "'&,@{).@,@(cr&,"1@{&,"'");'')@('.@1)))y.'
    Output=: ' ' : m

```

Appendix E — Error Message Programs

```
rem c:\j\amortize\inpmmsg0.wp;
mb "Input Error" "Present Value must be a positive, scalar number"
mb_iconstop;
rem c:\j\amortize\inpmmsg1.wp;
mb "Input Error" "Rates must lie between zero and 100"
mb_iconstop;
rem c:\j\amortize\inpmmsg2.wp;
mb "Input Error" "Periods per Year must be a positive, scalar integer"
mb_iconstop;
rem c:\j\amortize\inpmmsg3.wp;
mb "Input Error" "No. of Payments must be a positive, scalar integer"
mb_iconstop;
rem c:\j\amortize\inpmmsg4.wp;
mb "Input Error" "Displayed-Cell Limit must exceed two"
mb_iconstop;
```

Appendix F — Mostly Tacit Algorithms for Amortization Schedule

```
NB. c:\j\amortize\calsched.js
NB. Numerical values:
A=. ". 2.> PV;APR;PPA;INP;PNTS
NB. Interest each period:
I=. 0.012*.(%2>/@{(1 2%{0}))A NB. 0.012*.(%2>/1 2{y.
NB. Fixed payment from present value, interest per period, and periods:
payment=. ({0}) *2> [ % -.@(>:@[ ^ -2>@{(3%{0}))
PMT=. 1 payment A NB. ({y.)*2>x.%-.(>x.)^-.2>3{y.
NB. First, last, and sufficient intermediate periods for display:
subset=. /:-@-@(<:@[ , 1%{0}[ <. ([ >.%{0} ] * (>:@{(1.0})))
NB. NB.
/:-@-@(<:x.),1,x.<.(x.>.%{y.)*>:i.y.
NB. Right arg limits number of amortization points:
K=. (>@{(3%{0} A) subset _2%+>(4%{A) NB. (>3{A) subset _2+>4{A
NB. Interest factor:
IF=. 1 (>:@[ ^/ -@{0}) K NB. (>:1)^/-K
NB. Balance after K payments:
BAL=. |:IF%-(A)+>PMT*IF%<:IF
NB. Payment between amortization points K:
twofix=. 2%{+}\@{0}
PAY=. A (-/"2@twofix@(>@{0,})) BAL NB. -/"2 twofix(>@{A),BAL
NB. Cumulative payment against principal:
PAID=. +/"2@(+)\@{0} PAY NB. +/"2 +\PAY
NB. Cumulative interest on principal:
INT=. PAID--K*/PMT
NB. Fraction of payment to principal:
FPTP=. PAY%PMT ([ */- 1%{0}-@(-/"1@twofix@{0})) K
NB. NB. PAY%PMT*/-1,--/"1 twofix K
NB. Width of formatted result columns:
B=. FPTP,. PAID,. BAL,:"1 INT
w1=. 6.2%+@(>.@{(1%{0}^@(>./@{0}))}) B NB. 6.2+>.10%>./;B
NB. Width of formatted payment number:
C=. 4%{0}2%{0}("@{0}."1_10{0})) K NB. 4{0}2 "1_1. K
w2=. 2%{0}3%{0} C NB. 2{0}3y.
NB. Interest rate formatted for column headings:
D=. '%; '2{0}1~)@{0}:"2,2%{0}(>@{(1%{0}))A NB. '%; "1~;.,>1{A
NB. Payment formatted for column headings:
E=. 0.2%{0}2%{0} PMT NB. 0.2%{0}2 "0 PMT
```

```

NB. Width of column headings:
w3=. 1{$ F=. ' ', "1 D , "1 E
w1=. w1+0>.w3-<.w1

NB. Payment number and payment cells:
G=. C, "1 w1":B
NB. Table header, right-justified:
H=. (-2{$G){.%, (-w3>.<.w1){. "1 F
NB. Table for list box:
SCHD=. H ([, ' %,@(./@{5%{"_1@}}))G NB. H, ' ', ./5%{"_1 G
NB. Text ready for file save:
T=. 'Schedule for Amortizing ', PV, ' under Compound Interest', crlf
T=. T, 'At payment on left: fftp, cum paid, balance, interest', 4%$crlf
FILE_TXT=: T, 2%}.@, crlf, "1 SCHD NB. T, 2%}.crlf, "1 SCHD

```

Appendix G — Explicit Algorithms for Amortization Schedule

```

NB. c:\amortize\explsched.js
NB. Numerical values:
A=. ". %> PV;APR;PPA;TNP;PNTS
NB. Interest each period:
I=. 0.01*%>/1 2(A NB. 0.01%*%>/@{1 2%{A})
NB. Fixed payment from present value, interest per period, and periods:
payment=. ' ' : '({.y.)*>x.%-(>x.)^>3{y.'
PMT=. I payment A NB. ({.A})*>[%-@(>:@{^>@{3%{A})}
NB. First, last, and sufficient intermediate periods for display:
subset=. ' ' : '/:~.(<x.), 1, x.<.(x.>.%y.)*>:1.y.'
NB. /:-@~.@(<:@[1%,@{[<.(%)])*(>:@{i.@})}
NB. Right arg limits number of amortization points:
K=. (>3(A) subset _2>+4(A NB. (>@{3%{A) subset _2%+>{4%{A)
NB. Interest factor:
IF=. (>:1)^/-K NB. 1 (>:@{^/-@}) K
NB. Balance after K payments:
BAL=. |:IF*-(A)>+PMT*I%<:IF
NB. Payment between amortization points K:
twofix=. '2(+\y.' : ' ' NB. 2%{+\\})@]
PAY=. -/"2 twofix(>@{A), BAL NB. A (-/"2@twofix@(>@{A,]) BAL
NB. Cumulative payment against principal:
PAID=. +/"2 +PAY NB. +/"2@{+\\}) PAY

NB. Cumulative interest on principal:
INT=. PAID-K*PMT
NB. Fraction of payment to principal:
FPTP=. PAY%PMT*/~1,-/"1 twofix K NB. PAY%PMT([*/~1%,@{(-@{(-/"1@twofix@)])})K
NB. Width of formatted result columns:
B=. FPTP,. PAID,. BAL, "1 INT NB. 6.2%+@(>.@{10%^.@(>./@{;@})}) B
w1=. 6.2+>.10^.>./;B
NB. Width of formatted payment number:
C=. 4{"2 "., "1, K NB. 4%{("2)@(".:@{("1@{;@})}) K
w2=. 2{$C NB. 2%{($@) C
NB. Interest rate formatted for column headings:
D=. '%; ', "1~":., >1(A NB. '%;
' %{"1~}@{":%>@{1%{A})}A
NB. Payment formatted for column headings:
E=. 0.2": "0 PMT NB. (0.2%{"0)@) PMT
NB. Width of column headings:

```

```

w3=. 1{$ F=. ' ', "1 D , "1 E
w1=. w1+0>.w3-<.w1

NB. Payment number and payment cells:
G=. C, "1 w1":B
NB. Table header, right-justified:
H=. (-2{$G){.%, (-w3>.w1){. "1 F
NB. Table for list box:
SCHED=: H, ' ', /5%{."_1 G      NB. H ([, ' %,@(/@5%{."_1@])) G
NB. Text ready for file save:
T=. 'Schedule for Amortizing ', PV, ' under Compound Interest', crlf
T=. T, 'At payment on left: fptp, cum paid, balance, interest', 4%$crlf
FILE_TXT=: T, 2)., crlf, "1 SCHED NB. T, 2%}.@, crlf, "1 SCHED

```

Appendix H — Help Text

```

rem c:\j\amortize\about.wp;
mb "About Amortize" " A schedule is displayed from your input.
Not copyrighted; not liable for your results.
      Thomas M. Olsen
      71303.3615@compuserve.com
      Karsten Mfg. Corp.
      12 July 1993"

mb_iconinformation;

rem c:\j\amortize\parmdefs.wp;
mb "Parameter Definitions" "      Present Value: amount of
loan/annuity
      Interest Rates: annual percentage rate
                        of compound interest;
                        multiple values allowed;
                        e.g., 6+1.7
      Periods per Year: payments per annum
      Total No. Payments: payments to retire
                        loan/annuity
Displayed-Cell Limit: default of 9 fills one page
                        of saved output; includes
                        first, last, and penultimate"

mb_iconinformation;
rem c:\j\amortize\parmnavi.wp;
mb "Navigation" "Push ENTER to display amortization schedule"
mb_iconinformation;
rem c:\j\amortize\outitems.wp;
mb "Items" "      Heading: compound interest,
                  fixed payment
Left margin: payment number
      Cell items: fraction to principal,
                  cumulative paid,
                  balance,
                  cumulative interest"

mb_iconinformation;

```

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Technical Editorial: in Search of Contexts	Adrian Smith	106
Hackers' Corner: Flying Windows	Jonathan Barman	108
Finding the 10 to the 10th Prime	Gene McDonnell	110
APL - a Client-Server Language?	Morten Kromberg	114
A Note on Probabilities in the F-Distribution	Keith Smillie	124
Defined Operator Simulation for APL*PLUS II	Allan Gay	126
Inner Products for Range Tests	Jonathan Barman	133
Another Use of Not-Equals-Scan	Anthony Camacho	136

Technical Editorial: in Search of Contexts

by Adrian Smith

The Workspace is one of the great joys of APL, and it has served us remarkably well from the early days of 28K, through the VSPC days, when we had a whole 120K (or maybe 450K if we pleaded) to romp in, through to today's multi-megabyte machines. It gives you the freedom to call things by any name you choose, and to assemble a handy collection of working tools in one convenient place, where you know where they all are, and where no-one can change them under your nose.

For the sake of this freedom, we have forced ourselves to adopt a whole set of annoying conventions, or have risked losing vital functions and data with a single misguided `)COPY`. Typically we name our utilities with unnecessary prefixes (like my own `FSMΔthis`, `DRAWΔthat` set from AP126 days), or just hope that our names are sufficiently improbable that no-one else will have chosen any of them (the APL Statistics Library approach).

This approach was just about tenable when workspace sizes kept us down to a reasonable working set of functions; typically an application was made up of several workspaces, maybe one or more to maintain the base data, one to do the clever stuff (like building a production schedule), one to do the reports, and one to manage the import/export to mainstream systems. On the front of all this would be a tiny `MENU` workspace to switch between them. If an application broke, almost anyone could hack their way through to the point of failure with a quick check of `)FNS` and a look at the `□LX`.

Consider what happens as the workspace size increases, not by the odd factor of two or three, but by an order of magnitude. The universe of names may be large, but the odds of a clash shorten dramatically in a world where there are 10 times as many names!

Consider how you would react when faced with someone else's broken system where you type `)FNS` and the listing goes on for page after page after page. 90% of that stuff is ancient, rock-solid utility code — you want to see the 10% that makes this application what it is. It probably fits on 4 or 5 lines.

Consider the awful possibility that there really is a bug deep in the core of that ancient utility code! There may be hundreds of workspaces out there, all falling over just because Easter is in March this year. Mainframe folk know well enough

the necessity of infecting each and every workspace with a little piece of virus code that will go and check its utility timestamps against some central library, and automatically upgrade everyone with the latest versions of everything. But hang on a minute ... this is *my* application, running in *my* workspace! I don't want it falling over just because some cretin in the back office has 'fixed' a utility with the latest whizzo assembler code. I want to check it all out first, then ship that upgrade when I know it works. Anyway, viruses have got rather a bad name of late.

Under Windows, things get a whole lot worse. You *have* to program with a raft of small functions, because you have lots of objects out there, and the user may interact with any of them at any time in many different ways. Each combination of object/event usually needs a function to handle it, often a one-liner. Many objects really need little functions of their own, ranging from a simple upper-case mapper (for a text entry field), through date conversion (a date box), to a significant subset of PostScript (the *RAIN* graphics viewer).

Now, I want to call the APL function that does the PostScript 'moveto' operation *moveto*, and similarly for *lineto*, *show*, *currentpoint*, and so on. These are not particularly improbable names, and I can't imagine anyone would use my workspace more than once if they did a `)COPY RAIN` and their *show* function was promptly obliterated!

To a limited extent, clever programmers have found ways of fudging round the problem. APL*PLUS/PC does some amazing things with local `⌈FX` to implement `]UCMD`, and with Dyalog you can do a bit of fancy footwork with `⌈SHADOW` and `⌈OR`. This is how Duncan & I implement local variables on forms — we just stick them in the object's *DATA* property, and shadow and execute them when required. Yes, it works, but it is a monumental kludge we would be better off without.

What we need to do now ...

J has "locales", Manugistics have promised us "capsules", Dyadic are working on "namespaces" and APL2 has `⌈NA`. Please let them all know that we need this stuff, and that we need them to get it right. It must take away for ever the need for pointless prefixes on function names, it must offer us a sensible way of managing utility upgrades, and it must above all cut `)FNS` down to something that fits easily on one screen. If it does all that, and includes the Windows objects in a tolerably consistent way, then I, for one, will be a lot happier about the future of APL.

Hackers' Corner: Flying Windows

by Jonathan Barman

A little while ago I was shown how to edit the file that produces the screen saver *Flying Windows*. Windows is supplied with *Flying Windows* displaying the Windows logo which is actually defined as the very last character in the WingDings font (`^1+[]av`).

The screen saver program file SSFLYWIN.SCR includes this character just before the word DISPLAY. It is very easy to edit SSFLYWIN.SCR using Write with the *no conversion* option, and to paste the character of your choice from Character Map. I got fed up doing it this way because there was always the chance that I might muck up the file, and I found it difficult to visualise what the WingDings characters looked like at large font sizes.

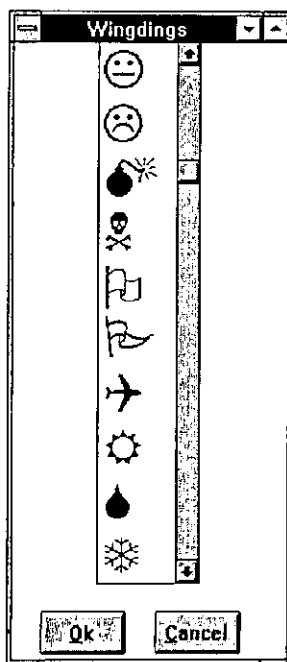
For a bit of fun I wrote an APLIWIN function to allow me to select the character from a list of big WingDings, and eventually came up with the following:

```

V wingdings;a;s;C;V;[]io
[1] []io+0
[2] C+[]av[156] ♦ V+[]av[255]
[3] a+'pc menu;pn Wingdings;'
[4] a+a,'xywh 30 0 35 200;'
[5] a+a,'cc 11 listbox ws_border ws_vscroll lbs_notify;'
[6] a+a,'cfont -35 0 0 0 400 0 0 0 2 3 2 1 2 "Wingdings";'
[7] a+a,'ctext ',V,',([]toapl 33+^1+[]av),[0.5] C),V,','
[8] a+a,'xywh 10 200 30 15;'
[9] a+a,'cc Ok Button;cn "&Ok";'
[10] a+a,'adjw 10;cci Cancel;cn "&Cancel";'
[11] a+a,'csel 11;cfocus;'
[12] a+a,'pmove 0 0 100 230;'
[13] a+a,'pcenter;pshow;ptop;wait;'
[14] s+^1+[]wd a
[15] -[]wd 'pclose;'
[16] +(s[6 7]# 7')+0
[17] +(0=([]toascii ^1+s) []hf writex '\windows\ssflywin.scr 11762')/0
[18] 'File write problem'
V

```

I wanted this to be a completely self-contained function, so there are no sub-functions, nor any global variables. Also, it is tailored for my screen size of 800 by 600 pixels. I had to muck about a bit with the numbers on lines [4], [8] and



[12] to get the list box about the right size on the screen. To get the numbers for the argument for *cfont*, on line [6], I ran '**data' wdg wd 'mbfont;*', and selected a suitably sized WingDing font. I let the results splat out onto the APLWIN session manager, and then just cut and pasted them into my function.

I found the magic number on line [17] by reading in the entire file and searching for the word DISPLAY:

```

io←0
a←hfread '\windows\ssflywin.scr'
-1+('DISPLAY'⌵toapl a)/⌵a
11762

```

I quite like the snowflakes, but there are lots of other possibilities.

At Work and Play with J

by Eugene McDonnell

What is the 10,000,000,000th prime number? This column tells a story, and it has a moral. It does not concern itself directly with J, the ostensible reason for these columns, but it can be justified because one of the direct antecedents of J is the language A, developed by Arthur Whitney while he worked at the investment banking firm of Morgan Stanley.

It was one page of C code for an A-like interpreter, written one afternoon by Whitney at Ken Iverson's Kiln Farm in Ontario, that gave Roger Hui the direction he needed to start work on what was to become J. Roger exhibits this code in Appendix A of *"An Implementation of J"*, published by Iverson Software Inc. Whitney no longer works at Morgan Stanley: he has set out as a freelance and is developing a language called K, which has some affinity with A and J. Hui now works at Morgan Stanley, and it is his adventure hunting down a large prime that the story is about, and Hui used A as the weapon with which he targeted the large prime.

The story begins when Hui's boss at Morgan Stanley challenged him by saying, "You think you're smart, but you don't even know what the 10-to-the-10-th prime is." Hui's immediate response was, "Do you start counting from 0 or 1?" The boss was so taken aback that for a minute or so he didn't understand Roger's question.

The boss's challenge seems to have been meant as an example of a theoretically attainable but practically impossible computational task. This article tells how Roger went about achieving the impossible. I look on it as a triumph of the client-server technology. This column is not so much an article about programming as it is about computer logistics. The programming aspects, while important, are secondary to the story of how Roger went about organizing the solution.

The germ of Hui's solution was to envision a Boolean vector p of length k such that the i th element of p is 1 if i is prime, and 0 otherwise. Just sum-scan this very long vector and look for the index of 1e10 in it. How long should such a vector be? The *Prime Number Theorem* says that the number of primes less than k is roughly $k\%(\log k)$. Solving for k in the equation $1e10 = k\%(\log k)$ gives a value for k about $2.63e11$. Roger, out of prudence, used the value $2.7e11$. A vector of $2.7e11$ elements is unrealizable in the present state of computer

memories, especially since A doesn't have a Boolean type: Boolean vectors require the same space as integer vectors. A vector of $4 \times 2.7 \times 10^{11}$, or 1×10^{12} bytes long is simply not on the cards. Even a Boolean vector taking just 1 bit per element would have to be more than 3×10^{10} bytes long, so it was clear that the problem had to be partitioned.

Hui's central program computes the primes between m and n , using the sieve method, eliminating multiples of 2, 3, 5, 7, 11, 13, 17, etc. This can be done independently in parallel on many small intervals that make up the larger interval of interest, and, if portioned out to computers that can communicate with a common central file, will permit the problem to be solved in a shorter time than if only one computer were to tackle it.

The method of partitioning was suggested by the presence of more than 150 workstations on the floor in Hui's part of Morgan Stanley. They are all interconnected Unix machines, and any machine can be used from any other machine. With relatively small effort a multi-processor solution could be set up, using these machines in parallel. The machines are not heavily used at weekends, and it was on a weekend that the experiment took place.

The strategy was to let the machines tackle the problem a billion integers at a time. Hui's colleague at Morgan Stanley, Seth Breitbart, suggested creating 271 files, named 0, 1, 2, ..., 270, each denoting the named interval of 1×10^9 consecutive numbers, and each one empty.

What a machine would do once it was set going was to look at the list of files, pick one at random (named m), erase it, work on the interval $(1 \times 10^9 \times m) + 1$ to $1 \times 10^9 + m$, a million numbers at a time, and after finishing, write a file containing a record giving the number of primes in each of the million-number intervals within that 1×10^9 (there are a thousand of them). After finishing, it repeats that process, stopping only when the list of files/intervals is empty. The machines were set up to process a million numbers at a time since the smallest machine available had enough memory to handle that many numbers at once. Roger notes that there's no great harm if two machines accidentally happen to pick the same file/interval. In the flexible Unix universe additional machines could be brought on stream at any time.

If one is a Unix system superuser it is possible to take all sorts of liberties with these machines, like finding the ids of all other machines, but Hui prefers (wisely, I think) not to be tainted by such capabilities, so to get the machines' names he went about the floor reading the names of the machines from strips of paper affixed to each one, then sat down at his machine and made inquiries about the state of each machine on his list.

If it was idle, he set it going on the problem. As he was doing this, there was a nice dilemma to resolve. Should his time be spent improving the algorithm before launching more machines, or should he spend time looking for additional machines? He favoured the latter approach for the novelty of it, and ended up using about 15 IBM RS/6000s and 60 Sun Sparc 2s and Sparc 10s.

After 20 hours, he had 271 files, each with 1,000 records. From these he made a 271,000-element vector of the number of primes in the intervals $1e6 + i \cdot 271000$. By sum-scanning this he knew the interval containing the 10^{10} th prime. His function `psieve` returns a Boolean list selecting the primes between m and n . Applying this to the magic interval gets the actual 10^{10} th prime.

Some details about his program "`psieve`":

If a number q is not divisible by any number less than or equal to $\sqrt{q}$, then q is prime. Therefore, to test a number less than $2.7e11$ for primality one need only use trial divisors less than $\sqrt{2.7e11}$ or roughly $6e5$.

In practice, Hui precomputed a list of all the 78,498 primes less than $1e6$ recursively, bootstrapping up from 2 3 5 7. (This only takes a few seconds.)

It was then a routine matter to determine for any number less than $1e12$ whether or not it was prime: just see whether its residues with respect to each of the primes less than a million was nonzero; if so, it was a prime.

For the curious, here is a condensed list of the first 10,000,000,002 primes, with their 0-origin ordinal numbers.

0	2
1	3
2	5
3	7
4	11

...

$1e10-1$	252,097,800,623
$1e10$	252,097,800,629
$1e10+1$	252,097,800,637

After doing this, Hui found that there is a table in William Judson LeVeque's "*Fundamentals of Number Theory*", section 1.1, giving the number of primes less than $10^3 + 1.8$. Hui's table agrees with LeVeque's for $10^3 + 1.7$. For 10^{10} , however, LeVeque says 455,052,512 and Hui says 455,052,511. It turns out that LeVeque is wrong, Hui having checked his results with some help from Lee Dickey at Waterloo University. Dickey tells Hui that his colleagues speculate that

LeVeque may have gotten his numbers from lists that D.N. Lehmer compiled, which included 1 as a prime, and LeVeque may have slipped in not subtracting 1 from that particular count. (1 isn't a prime since it doesn't satisfy the definition of a prime: a positive integer n with exactly two distinct factors, 1 and n .)

Now for the moral of the story: Hui tells me he has also since found some work that would have made it much easier to discover the n th prime, for any n . E.D.F. Meissel, a German astronomer, found in the 1870s a method for computing individual values of $\pi(x)$, the counting function for the number of primes $\leq x$. His method was based on recurrences for partial sieving functions, and he used it to compute $\pi(1e9)$, where π is a function that computes the number of primes less than or equal to its argument.

D.H. Lehmer simplified and extended Meissel's method. Recently, further refinements of the Meissel-Lehmer method which incorporate some new sieving techniques have been reported by Lagaria et al [1]. In this article the authors give an asymptotic running time analysis of the resulting algorithm, showing that for every $\epsilon > 0$ it computes $\pi(x)$ using at most $O(x^{(2/3)+\epsilon})$ arithmetic operations and using at most $O(x^{(1/3)+\epsilon})$ storage locations on a computer using words of length $1 + \epsilon \cdot 2^{\wedge} x$ bits.

The algorithm can be further speeded up using parallel processors. They show that there is an algorithm which, when given M parallel processors, computes $\pi(x)$ in time at most $O((M) \cdot x^{(2/3)+\epsilon})$ using at most $O(x^{(1/3)+\epsilon})$ storage locations on each parallel processor, provided $M \leq x^{1/3}$. A variant of the algorithm was implemented and used to compute $\pi(4e16)$.

They report that $\pi(4e16)$ took them 1730 minutes on an IBM 3081K; $\pi(2e12)$ took 3 minutes; $\pi(3e12)$ took 4 minutes. Had he known about this method, Hui could have used a binary search technique to find the $1e10$ th prime, using an $O(2 \log n)$ technique, after narrowing the interval to be searched in by a reasonably generous use of the Prime Number Theorem. The value Hui computed was $\pi(2.52e11)$ and it took him 20 hours on 60-70 workstations.

Brains win again over brawn: a well-designed, mathematically knowledgeable algorithm beats brute force!

Reference

- [1] Lagaria, Miller, and Odlyzko, in "Computing $\pi(x)$: Meissel-Lehmer Method", Mathematics of Computation, Volume 44, Number 170, 1985 4, pages 537 - 560

APL - a Client-Server Language?

by Morten Kromberg (*Insight Systems*)

Internet: insimkr@uts.uni-c.dk

It is becoming increasingly difficult to sell a product if the magic words *Client/Server* are not clearly visible on the package. As a result, the term has become broadened by marketing departments to mean everything from dumb terminal emulation performed by a PC, through remote SQL queries, to applications using the most sophisticated peer-to-peer networked messaging systems or *Object Request Brokers*.

What's New?

Is there any reason why the pragmatic APL user, intent on solving real problems (sorry, I mean pursuing real opportunities), should take notice? After all, many APL programmers were writing Client/Server applications back in 1980. Page 97 of the original SHARP APL Reference Manual by Paul Berry (copyright I.P. Sharp Associates, 1979), contains a complete coding example (almost an entire page) of a multi-user server mechanism based on an "object-oriented" communications mechanism called *Shared Variables* (if I may allow myself the liberty of widening the term object-oriented a bit).

The data processing community has succeeded in reinventing Shared Variables in a number of guises, some of which are now supported by a critical mass of applications and development tools. The driving force behind most of the market interest in these mechanisms is that they make it practical to split an application into independent Client and Server modules. Even when both modules run on the same machine, this strategy can simplify the architecture of complex systems. When client and server can run on two different machines, it becomes possible to "downsize" applications and move the presentation layer, plus some computation to workstations, which have superior price/performance ratios when compared to mainframes. In the current economic climate, this is the immediately attractive aspect of client/server computing — but in the long term, the ability to "upscale" applications is also important. When the usage of an application increases, or data must be made available to other parts of an organisation, it is often necessary to upgrade from a local or LAN-based database to a more secure or robust multi-user server environment.

Client/Server is just what APL needs

Modern workstation operating systems are perfect environments for APL. APL's hunger for CPU and memory resources can easily be satisfied by modern workstations; APL's performance is often better on a workstation than on a shared mainframe. By succumbing to Client/Server, you can finally make the manager of data processing happy and remove your APL code from his or her mainframe — and still maintain access to all that corporate data!

The evolution of tools for software development has resulted in a number of extremely useful tools and powerful paradigms for organisation (OO), distribution (SQL and Client/Server) and presentation (GUI). APL retains most of its competitive edge with respect to general-purpose manipulation and analysis, but this advantage is of little value if we cannot deal effectively with the other aspects of application development and delivery.

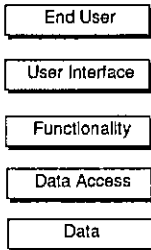
The APL vendors have come a long way in providing the mechanisms that we need to attach ourselves to standard tools available under workstation operating systems. Support for Dynamic Link Libraries (DLL), Dynamic Data Exchange (DDE) and the Clipboard have made it possible for us to plug straight into thousands of tools and applications written by people who had never heard of APL.

We need to extend this capability across the network using Client/Server techniques. The immediate requirement is to enable APL as a client, so that we can make use of all the servers that are available to users and developers using other tools than APL. But we should not forget that the unique capabilities of APL also make APL interesting as a Server — especially if we can just make APL applications available in the same way as any other service such as a database, ticket sales, or mail server — without anyone having to know that we used APL to write it!

Client/Server Models

We all know what Client/Server means, so rather than bore you with repetitions, let us take a look at how Client/Server products are marketed to the non-APL community (the so-called *Information Technology Marketplace*). The Client/Server product vendor will tell you that his or her product supports *n* distinct Client/Server models.

This means that the physical split between client and server can be inserted in n different locations in the simplified application model below:



The split can be inserted between two layers or in the middle of a layer. Four of the most common models are described in a little more detail below:

Distributed Presentation

The X-Windows system has become relatively popular in the Unix environment. With X-Windows, the split is inside the user interface layer. Some presentation code is embedded in the application on the host (X-Windows terminology calls this end the Client, just to confuse you), and the rest is in the X-Windows Server which runs at the user's end and provides *presentation services* to the application. Although this mechanism can be used to offload an important part of the presentation from the host CPU, a good deal remains, and the network load is rather heavy. The main advantage is that administration and coding is still as simple as it was on the mainframe, since the entire application is still running on a single machine.

Remote Database

With the current state of the art, Client/Server tends to mean the use of a remote SQL database. The split is between the *functionality* and *data access* layers. Despite the fact that this is the most common form of Client/Server computing, and that it is still rapidly growing, it is not the final solution. The problem is that SQL is an *awful* language for describing transactions. As the name implies, it was not designed for that. It is a *query* language. Performing all but the most trivial updates using SQL over the network is painful, inefficient and will always be a fragile mechanism.

Remote Transaction

The split is inserted in the *functionality layer*. Client and Server communicate using transactions that correspond to logical events in the application. The

conversation might consist of messages like "Debit the amount X from account Y", or "Give me the latest forecast for currency X". In the first example, the steps required to implement the transaction might be:

1. Check whether account Y exist
2. Check credit limit
3. Register transaction
4. Update balance

Using *Remote Database*, each step is likely to require at least one network transaction. With *Remote Transaction*, all the logic is embedded in a portion of the application running on the server, which is much closer to the database in terms of turnaround time, bandwidth, and reliability. This is potentially the most complex solution in terms of coding, but with good modular coding techniques (and that includes object orientation), it is the most flexible solution.

Distributed Database

Some database systems are able to distribute the data itself, so that different parts of it reside on different computers. The management of a relational database that is split in this manner is complex. No matter how sophisticated your mechanism for ensuring the data integrity (often referred to as *Two Phase Commit* mechanisms) is, you will have to deal with worst-case scenarios such as restoring a distributed database from backups of different machines. To be safe, you will have to write code that is tolerant of small failures of integrity.

Next Year's Model

The state of Client/Server is evolving in a direction where it is becoming easier to implement *Remote Transaction* systems. The first step in this direction is the implementation of added functionality in many commercially available SQL servers. You can now use *stored procedures*, or *triggers*, which means that the server has a simple procedural language which makes it possible to perform complex updates in a single call. Unfortunately there has been no standardisation of these new features, so applications that use them are not portable between different SQL databases.

The architecture of the future may be something like *CORBA* — the *Common Object Request Broker Architecture*, which enables dedicated servers to reside on an intelligent network, and allows applications to issue high-level calls on the network without knowing where in the network any particular service resides. A lot of work needs to be done to make this practical; for example, vendors must agree on a cross-network security system which makes it possible for you to log into the network once, and then use facilities under multiple host operating

systems using the privileges assigned to you by a common security system — but there is little doubt that these systems will become commonplace during the next 5 to 10 years.

What is Available Now?

Let us assume that your goal is to develop applications that make use of the cheap memory and processing power available on workstations, to provide your users with a nice, easy-to-use graphical user interface, which can be integrated seamlessly with other applications on the desktop. You should buy a workstation running Macintosh System 7, Unix, OS/2, or Microsoft Windows, and one of the excellent new APL interpreters which provide access to all local services and applications available in the selected environment.

Access to Remote Databases

The remote service you are most likely to need is an SQL database. This access will be available to you in at least four different ways:

1. A Proprietary API Extender from database vendor
2. An API Converter, such as the Q+E Library
3. ODBC: Open Database Connectivity (or ODAPI)
4. A Generic Middleware product such as SequeLink

Application Programme Interfaces for SQL databases tend to have almost identical functionality. Typical calls are:

```
LOGON . . . . You must LOGON to the service
OPEN . . . . Open a database cursor
PREPARE . . . Prepare a particular SQL statement for execution
DESCRIBE . . Return column names and data types of result set
BIND . . . . .Transfer values for variable parts of the statement
EXECUTE . . . Do it
FETCH . . . . Fetch results
COMMIT . . . Make the effect of a sequence of executes permanent
ROLLBACK . . Discard changes (opposite of COMMIT)
CLOSE . . . . Free resources allocated to a cursor
LOGOFF . . . Disconnect from the server
```

How do the four delivery mechanisms differ?

1. Proprietary API Extenders

All databases make their API available in the host environment. AP127 for APL2 or SHARP APL, and the Oracle Interface for Dyalog APL under AIX are

examples of products which hook APL interpreters up to database managers running on the same machine as APL itself.

Almost all database vendors provide an extension of the database API, which is available to programmers on workstations. Examples are SQL/NET from Oracle, INFORMIX-NET from Informix, and the DOS/Windows Application Enabler for OS/2 Database Manager. These API extenders simply make the same API which is available to programmers on the host available on a remote system.

The problem with these proprietary extenders is that they are all different, despite the fact that the functionality they provide is almost identical. Each API will use calls with slightly different parameters. If you use one of these tools, you must rewrite your application if you need to use another database.

The database vendors often provide modules which make the API easily accessible from within selected development tools; either their own, or tools from vendors with whom they have strategic partnerships. As long as you stick to these, and your network provides the (often limited) list of protocols which the API Extender supports, you are OK.

2. API Converters

The "API confusion" caused by proprietary API extenders has been the driving force behind the development of products such as the Q+E Library. The Q+E product provides a single API, which bolts on to a large variety of proprietary API extenders. Q+E converts your calls and your data to the formats required by each proprietary API. The DB+ product from TMT-Team is an interface from APL to Q+E, which in turn provides access to all database servers supported by Q+E.

Note that you still have to have a proprietary API Extender present in order to use these converters. Simultaneous access to multiple services may be complicated, since all required API extenders must be loaded simultaneously; they may have different requirements with respect to network protocols, etc.

3. ODBC

Instead of writing more API Converters, Microsoft provided a specification for a common API called **Open Database Connectivity (ODBC)**, in the hope that database and middleware vendors would write compliant ODBC drivers, and that development tool vendors will deliver tools which can use these drivers. Microsoft has declared that it will provide ODBC drivers for its own database managers and that development tools such as Visual Basic and Microsoft Access will be able to use ODBC. The only piece of code provided directly by Microsoft is the ODBC Driver Manager, a switching station which loads ODBC drivers when applications request connections to them, and passes ODBC calls to the drivers after performing a small amount of validation.

It is important to note that Microsoft and Borland primarily see tools like Access and Paradox (respectively) as *development tools* rather than database servers. Support for access to remote databases using ODBC or ODAPI — Borland and IBM's alternative standard API (the best thing about standards is that there are so many of them!) was provided at an early stage, but there were no ODBC or ODAPI interfaces giving access to data stored in Access and Paradox until recently.

The database vendors are scrambling to provide ODBC-compliant versions of their proprietary API extenders, and ODBC is rapidly reaching critical mass. This will eventually do away with the need for products like the Q+E Database Library, and Pioneer Software, the company behind this product, is moving on to the business of providing ODBC drivers on behalf of the database vendors. Providing a fully functional ODBC driver is no simple task, since the full ODBC specification is far richer than most database APIs. In fact, very few drivers support the full API, and one of the most irritating aspects of using ODBC is that you need to inquire about the level of functionality provided by a particular driver, and possibly adopt alternative strategies depending on the answer you get back. The result is that quite a bit of API Confusion remains, and one of the points made by recent Q+E advertising material is that they provide a "consistent set of ODBC drivers" (!?).

Examples of additional features of ODBC not usually found in other APIs are calls to translate SQL statements between ODBC SQL and the native SQL supported by the server; scroll backwards; do cellwise or columnwise fetches; or even retrieve columns that were not specified in earlier fetches. Many of these ODBC calls make sense when dealing with a local database such as Access or dBase, but are very hard to implement efficiently for a remote database.

The *ODBC Support* provided by tools such as Visual Basic or Excel tends to be at a much higher level than the ODBC API itself. For example, Visual Basic provides the concept of a *Dynaset*, which is created by naming a Table or View. The Visual Basic ODBC interface copies data into local storage, and you can make it easy for the user to browse or modify the data with a minimum of programming effort. However, it is often hard to control the resources used to implement this functionality, and solutions which run extremely fast when tested using a dBase driver with almost unlimited bandwidth may have totally unacceptable performance when used across the network. It is important to note that this behaviour is not a function of ODBC itself, but rather of the high-level access to it that is built into some development tools.

Using proprietary ODBC drivers, or ODBC converters that bolt on to a traditional API Extender, you still have the same problems with respect to compatibility of the underlying (network) products. Drivers from different vendors will not always peacefully inhabit a Windows system side by side. To solve this problem, you need a *generic middleware* solution.

4. Generic Middleware

Generic middleware products solve the compatibility problem by providing their own transport layer. You do not buy any communications software from your database vendor — the middleware server component connects straight to the database API on your host, performs API conversion at the host end, and uses whatever network you have available to transfer data to the middleware client component running on a workstation. This means that you only have a single product running on your workstation, even if you are using several networks to connect to a number of database systems on a variety of host platforms.

The middleware vendors are also capable of providing an ODBC driver under Windows. A single ODBC driver can thus provide access to multiple services without risk of conflicts. The fact that the middleware product is in control of both ends of the network conversation means that it can optimise the use of the network to provide better performance than a lot of proprietary ODBC drivers, which are often bolted on to API Extenders which were not designed with ODBC in mind. As an additional benefit, you get a completely consistent ODBC implementation across all supported databases.

The networked versions of AP127 available from IBM and Soliton have a "generic" middleware architecture, but these vendors provide access to a small number of clients, network protocols, host operating systems, and database managers. A product like *SequeLink* supports just about all databases running under OS/2, Unix, VMS, AS/400, VM, and MVS, over NetBIOS, SPX/IPX, more than 20 "flavours" of TCP/IP, AppleTalk, SNA and DecNET, from Macintoshes, Windows, OS/2, Unix and VMS clients.

The *SQAPL Client* product from Insight Systems is an interface to *SequeLink*. Because identical *SequeLink* APIs are available on so many platforms, *SQAPL* can provide access to all services accessible through *SequeLink* from *APL*PLUS II* under Windows and Unix, *APL2* under OS/2 and Unix, and *Dyalog APL* under Windows, OS/2 and Unix — providing exactly the same API in all environments.

Access to Non-SQL Services

At the moment, SQL and Client/Server are more or less synonymous — but I do not believe that this will continue to be true. SQL is a query language, and will continue to be used for that purpose — but SQL is not a particularly elegant or effective language for expressing updates or transactions. We need other mechanisms to initiate transactions, or to make *Remote Procedure Calls (RPCs)*.

Vendors of Unix systems often provide RPC kits, which make it possible to compile an application consisting of a client and a server part. It is very hard to provide flexible systems in this manner, because the two parts are so closely

interlinked. Changes require recompilation and testing of both ends of the application. If communication problems cause the partners to get "out of synch", it may be hard to return the system to a known state.

Vendors of *Transaction Monitors* such as IBM's Customer Information Control System (CICS) are beginning to provide API extenders in the same way as the SQL database vendors. This makes it possible to code and test transactions on the host, and use the same transactions from host applications and Client/Server applications running on workstations.

Advanced communications protocols such as TCP/IP and IBM's APPC (LU6.2) also provide mechanisms by which two consenting applications can exchange data over the network, but this approach still requires a bit too much network expertise for the comfort of the average APL programmer — and of course the solutions will be dependent on the particular network protocol being used.

CORBA

A working group with origins in the Object-Oriented community has defined CORBA — the *Common Object Request Broker Architecture*. This architecture will extend the development environments of the future (it is assumed that these will be Object-Oriented to some degree), so that they can send messages to objects, without worrying about whether the objects exist nearby, or somewhere on the network. CORBA needs to be built into the networks and integrated with security facilities of all networks and hosts involved, and the required technology is still under development in the laboratories. Once we have it, the network will more or less disappear from sight, and it will be as easy to extract data from a server on another planet as it is to open a spreadsheet. Let's hope we all have fibre-optic telephone cabling by then!

While we wait for CORBA

For the time being, Generic Middleware is as close as we can get. Products such as SequeLink provide "open" servers, which you can use to encapsulate any existing host application. By customising an open server, you can call your favourite C program, CICS or OS/400 (RPG) transactions under MVS and OS/400 — or even APL! The SQAPL Server product from Insight Systems makes APL2/6000, APL*PLUS II/UNIX and Dyalog APL/X available as SequeLink Servers under Unix. You can execute APL function calls from any SequeLink Client — not just the SQAPL Clients. You can make APL function calls from Excel under Windows, OS/2 or the Mac — from Omnis 7, WingZ, Visual Basic, Smalltalk, and so forth — using exactly the same API as you would to make SQL requests.

A Glimpse of The Future

We are beginning to get a glimpse of a future in which the users' workstation provides a single entry point to any number of local, departmental, corporate, national, or even global services — many of which will be running on dedicated server machines located somewhere on a widely branched network, but all seeming to be an integrated part of the users' desktop.

This future is not far off — and as always, APL is a step ahead. With the SQAPL Server, APL became one of the first general-purpose languages to have a dynamic function call mechanism bolted on to a generic middleware product. So far, the only language available has been SQL, plus a few operating system command shells implemented for the VAX and Unix. The APL Server is available for three of the most popular APL interpreters available under Unix.

APL will be a First Class Citizen of the new world order. From APL, we can not only call local and remote services, but we can provide services to local applications via standard calling mechanisms, from the Clipboard through DDE to DLLs, and to remote applications via Middleware. When CORBA arrives, you can be sure that APL will be there to meet it!

Acknowledgements

Many thanks to Kim Andreasen, Gitte Christensen and Martin Gfeller for helpful comments and suggestions.

A Note on Probabilities in the F-Distribution

by Keith Smillie
smillie@cs.ualberta.ca

A recent tutorial paper by Alan Mayer in *Vector* on the use of APL in analysis-of-variance calculations contained a request for methods of calculating probabilities in an F-distribution which were simpler and more satisfying than the one given there. The present note gives an algorithm in J which uses the integral adverb I introduced by Kenneth Iverson in his recent monograph *Calculus* such that the expression `f 1 x` gives the area under the graph corresponding to the verb `f` from 0 to each item in the list `x`.

The problem is to evaluate the integral

$$P_X(F) = \int_0^F \frac{\Gamma(\frac{m+n}{2})}{\Gamma(\frac{m}{2})\Gamma(\frac{n}{2})} m^{\frac{m}{2}} n^{\frac{n}{2}} x^{\frac{m+n}{2}-1} (n+mx)^{-\frac{m+n}{2}} dx$$

where m and n are, respectively, the degrees of freedom for the numerator and denominator mean squares, and F is the ratio of the corresponding mean squares. In the following list the verb `G` is the Gamma function, `B` the Beta function, `Fmn` and `Fmnx` the components of the density function the first of which depends only on the degrees of freedom, and `F` the probability density function:

```
G=. !@<:
B=. (G@[ * G@]) % G@+
Fmn=. (([^-:@[) * ([^-:@[)) % (-:@[) B (-:@[)
Fmnx=. ([ ^ <:@-:@[.@[) * ({:@[ +{.@[ *]) ^ (-@-:@(+/@[))
F=. ({.@[ Fmn {:@[) * Fmnx
```

For example, for 5 and 15 degrees of freedom the 10%, 5%, 2.5% and 1% points for the F-distribution are 2.27, 2.90, 3.58 and 4.56, respectively, and the expression

```
5 15&F I 2.27 2.90 3.58 4.56
```

has the value

```
0.8997 0.950015 0.975236 0.990321 .
```

The adverb I is defined by the following sequence of verbs (and one conjunction) which is equivalent to approximating the function by four groups of a polynomial of order ten:

```
W=. +:({#.|. @])"0 1|:@(0:,%.@(^/~@i)%>:@i=. i.@>:@+:)
ew=. +/@(-@(<:@({#@]) * i.@[)|."0 1]{.~>:@([*<:@({#@]))
EW=. ([ ew W@]) f.
ai=. ' +/@(x.&space * x.&[ * y.@(x.&grid))"0' : 2
grid=. space * i.@#[
space=. ] % <:@#@[
I=. (4 EW 5) ai
```

The interested reader is referred to Iverson's monograph for details.

Keith Smillie
Department of Computing Science
University of Alberta
Edmonton, Alberta T6G 2H1

References

- Iverson, K. E., 1993. *Calculus*. Iverson Software Inc., Toronto.
- Mayer, A., 1993, *Step by step analysis of variance* Vector, Vol.10 No.1, pp. 27 - 33.
- Standard Mathematical Tables*. Nineteenth Edition, 1971. The Chemical Rubber Company, Cleveland, Ohio.

Defined Operator Simulation for APL*PLUS II/386

by Allan Gay, Cocking & Drury Ltd

Introduction

During a recent migration of mainframe APL2 applications to APL*PLUS II/386, some defined operators were encountered. Because the target interpreter does not support defined operators, it was necessary to simulate them. This article describes how it was done.

Materials

The APL*PLUS II/386 code examples in this article all require Release 4.0 or higher, running in Evolution Level 2 mode.

For illustration purposes, we'll be using the monadic defined operator *woof*, which produces an ambi-valent derived function. Here are the APL2 statements:

```

VΔZ←Δa(F woof)Δw;ΔA;ΔW
[1]  A Compute sum of α and ω, and then F these sums together.
[2]
[3]  ΔW←+/εΔW                      A Sum ω.
[4]  →(2*⌈NC 'Δa')/MMM              A If no α, do the monadic case.
[5]  ΔA←+/εΔa                      A Sum α.
[6]  ΔZ←ΔA F ΔW                    A F the sums together.
[7]  →ZZZ
[8]
[9]  MMM:                            A MONADIC CASE:
[10] ΔZ← F ΔW                      A Just F the sum of ω.
[11]
[12] ZZZ:
    V

```

This defined operator has been invented for the purposes of this article. The name is chosen to commemorate a recent canine contretemps [FIDO 1993]. The pooch was probably a Smalltalk aficionado.

Concept

A defined function takes arrays as inputs and may produce an array as its result. A defined operator takes functions as inputs and produces a derived function as its result. The derived function then processes the input arrays supplied.

Like functions, operators may be monadic or dyadic. Unlike functions, they cannot be niladic or ambi-valent.

Simulation

In APL*PLUS II/386, it is necessary to replace each defined operator by a defined function. This frontend function first creates the derived function and then runs it. Each input function becomes a variable containing a function id.

Each function id is a character scalar or vector. It might for example be `'÷'` or that old favourite `'FOO'`. What's more, if defined operator sequences are being used, it might be the id of another derived function.

When the defined operator is monadic, there is one id; when it is dyadic, there are two.

The frontend inserts the function ids into a copy of a template variable and fixes the result as a derived function. The frontend then runs the derived function with the input variable or variables supplied. The template variable is easily produced by manually editing a copy of the APL2 defined operator's source statements.

Given some standardisation, a utility function can handle template customisation and the production of the derived function. The frontend marshalls the relevant inputs for this deriver. In the simulation, the deriver is named *derive_fn*.

Coding Invocations

To use a given simulated defined operator, we code an invocation of its frontend function wherever that operator was used. The frontend will take between two and four inputs, depending on whether or not the operator is dyadic and whether or not the derived function is dyadic.

When more than two inputs are present, we batch them up into nested vectors. Here are the defined operator prototypes and their simulated equivalents.

In APL2:

```
(LO MOP) R
L (LO MOP) R
(LO DOP RO) R
L (LO DOP RO) R
Z← (LO MOP) R
Z← L (LO MOP) R
Z← (LO DOP RO) R
Z← L (LO DOP RO) R
```

Simulation:

```
'LO' MOP R
(L 'LO') MOP R
'LO' DOP ('RO' R)
(L 'LO') DOP ('RO' R)
Z← 'LO' MOP R
Z← (L 'LO') MOP R
Z← 'LO' DOP ('RO' R)
Z← (L 'LO') DOP ('RO' R)
```

LO and *RO* are input functions to APL2 defined operators *MOP* (monadic) and *DOP* (dyadic). In the simulation, these inputs become function ids '*LO*' and '*RO*' respectively, and the defined operators become identically-named dyadic frontend functions. *L* and *R* are the input variables to the derived function. *Z* is the result of the derived function applied to its inputs.

With this approach, recoding defined operator invocations becomes a routine affair of moving parentheses around and adding quote symbols.

A Sample Invocation

Let's pause at this point to see how the defined operator *woof* fits into this scheme. It's a monadic operator which yields an ambi-valent derived function. In APL2, its header is therefore in the form

```
Z←L (LO MOP) R
```

and looks like this:

```
ΔZ←Δa(F woof)Δw;ΔA;ΔW
```

When we simulate it in APL*PLUS II/386, the frontend function's header looks like this:

```
ΔZ←Δa woof Δw;ΔA;Δf;derived_fn
```

In APL2, we might code an invocation of *woof* such as

```
3 4 5 6 ÷woof 3 2
```

In APL*PLUS II/386, we would code

```
3 ((4 5 6) '÷') woof 3 2
```

The Frontend Function

In the frontend, we carefully validate the inputs, we get the deriver to produce the derived function, and then we run it. We arrange to have the frontend tell the deriver what name we want the derived function to bear, and then we can localise that name so that the whole thing looks seamless.

Of course, if the application uses the defined operator many times, the concomitant repetition of the derivation process may be an overhead too great to be borne. If so, we might delocalise the derived function, run the deriver once to create it as a permanent resource, alter the relevant defined operator invocations to invoke the derived function instead of the frontend, and save the workspace.

Similarly, if an operator's template makes multiple references to dependent defined operators, we might group all the derivation work at the beginning of the frontend and have the template use the derived functions explicitly throughout.

And finally, if defined operators are being chained, we will have to delocalise earlier derived functions so that their ids may be passed as inputs to later frontends.

The Deriver

The deriver is a dyadic utility function which customises a copy of a named template variable with supplied data. The deriver is named *derive_fn*. Its left argument comprises one or two input function ids, depending on whether the operator is monadic or dyadic. Its right argument is a two-item vector which supplies the name of the template variable and the name which is to be allotted to the derived function.

Here is a listing of the deriver:

```

      7Δa derive_fn Δw;ΔA;Δf;Δo;ΔQ
[1]  A   Simulate defined operator.                      --- PRIVATE
[2]  A   Derive fn 2>w using template 1>w with fn[s] α.
[3]
[4]  A   α is scalar or 2-item vector
[5]  A   - each item must be a character scalar or vector naming a
[6]  A   primitive, defined, or derived function (eg. + plus tsNOT).
[7]  A   w is 2-item vector
[8]  A   w[1] is text vector name of the defined operator's BYR.
[9]  A   w[2] is text vector name which the derived function is to bear.
[10]
[11] (Δo Δf)+Δw      A   Get op-template id & derived-fn id.

```

```

[12]  ΔQ←',εΔo           ⋄ Get OVR of defined operator.
[13]  ΔQ[(ΔQ='r')/1pΔQ]←εΔf ⋄ ΔQ←εΔQ ⋄ Insert desired derived-fn id.
[14]  ΔA←c' ',(ε+Δa),' ' ⋄ Space out first input fn id for
[15]  ΔQ[(ΔQ='γ')/1pΔQ]+ΔA ⋄ ΔQ←εΔQ ⋄ syntax reasons and insert it.
[16]  →(-,2)=pΔa)/DDD ⋄ If two fn ids supplied,
[17]  ΔA←c' ',(εΔa[2]),' ' ⋄ space out the second fn id for
[18]  ΔQ[(ΔQ='l')/1pΔQ]+ΔA ⋄ ΔQ←εΔQ ⋄ syntax reasons and insert it.
[19]
[20]  DDD:
[21]  ΔQ←ODEF ΔQ           ⋄ Define the template as a function.
[22]
[23]  ⋄ (0) APLG 26AUG92
      ∇

```

As this code reveals, the deriver is a simple text substituter which relies on the presence of pre-arranged cue symbols embedded in the template. The deriver replaces these symbols with character data taken from its inputs.

The Template

The template is a text vector in the form of a OVR output. Within it, the three graphics symbols "r", "γ" and "l" are used as insertion cues. The first of the three symbols is the cue for the derived-function id. The others are used as cues for insertions of the input function ids.

Here is the *woof* defined operator's template variable, *KK_woof*. As a comparison with the listing of *woof* given at the head of this article will demonstrate, this template is a manually edited copy of *woof*'s APL2 source statements.

```

∇ Δz+Δa r Δw;Δa;Δw
[1]  ⋄ ---- SIMULATED APL2 DEFINED-OPERATOR:  γwoof
[2]  ⋄ Compute sum of a and sum of w, and then γ these sums together.
[3]
[4]  Δw←+/OENLIST Δw           ⋄ Sum w.
[5]  →(2*OENC 'Δa')/MMM       ⋄ If no a, go do the monadic case.
[6]  Δa←+/OENLIST Δa           ⋄ Sum a.
[7]  Δz+Δa γ Δw               ⋄ γ the sums together.
[8]  +ZZZ
[9]
[10]  MMM:                     ⋄ MONADIC CASE:
[11]  Δz← γ Δw                 ⋄ Just γ the sum of w.
[12]
[13]  ZZZ:
[14]
[15]  ⋄ THIS FN WAS GENERATED FROM KK_woof VIA ∇derive_fn
      ∇

```

Here is the simulation frontend for the *woof* defined operator.

```

V Δz←Δa woof Δw;ΔA;Δf;derived_fn
[1]  A Simulate the "woof" defined operator          --- PUBLIC
[2]  A  α is one of:
[3]  A      . a character scalar or vector - function id (eg. + xyz )
[4]  A      . a 2-item vector comprising
[5]  A          α[1] a left-argument to the function to be generated
[6]  A          α[2] a character scalar or vector - function id
[7]  A  w is the data on which to operate.
[8]  A  ← is the result of the derived function.
[9]
[10]  □ERROR (2*□NC'Δa')/'Vwoof MUST BE USED DYADICALLY'
[11]
[12]  □ERROR (1<ppΔa)/'Vwoof LEFT-ARG MUST BE FN ID OR A 2-ITEM VECTOR'
[13]  +((2>=Δa)∧∧/' '=□TYPE"εΔa)/MMM          A If α is fnid only, monadic use.
[14]
[15]  □ERROR (2*ρΔa)/'Vwoof NON-SIMPLE LEFT-ARG MUST BE A 2-ITEM VECTOR'
[16]  (ΔA Δf)+Δa
[17]  □ERROR (1<ppΔf)/'Vwoof LEFT-ARG 2ND ITEM MUST BE SCALAR OR VECTOR'
[18]  □ERROR (1<=Δf)/'Vwoof LEFT-ARG 2ND ITEM MUST BE SIMPLE'
[19]  □ERROR (' '*□TYPE †Δf)/'Vwoof LEFT-ARG 2ND ITEM MUST BE A FN NAME'
[20]
[21]  (<εΔf) derive_fn 'KK_woof' 'derived_fn'  A Generate Vderived_fn
[22]  Δz←ΔA derived_fn Δw                      A and run it dyadically.
[23]  +ZZZ
[24]
[25]  MMM:
[26]  □ERROR (' '*□TYPE †Δa)/'Vwoof INVOKED WITH INVALID FUNCTION ID'
[27]  (<εΔa) derive_fn 'KK_woof' 'derived_fn'  A Generate Vderived_fn
[28]  Δz←derived_fn Δw                          A and run it monadically.
[29]
[30]  ZZZ:
[31]
[32]  A {0} APG 04JUN93
V

```

In this function, the template variable is *KK_woof* and the derived function is named *derived_fn*. During the preliminary validation of arguments, messages documenting erroneous invocations of *woof* are issued; note that these relate to erroneous invocations of the frontend function, not the original defined operator. Finally, here is the derived function produced when use of the *woof* operator with Divide is simulated.

```

V Δz←Δa derived_fn Δw;ΔA;ΔW
[1]  A ---- SIMULATED APL2 DEFINED-OPERATOR:  + woof
[2]  A Compute sum of α and sum of w, and then + these sums together.
[3]
[4]  ΔW←+/□ENLIST Δw          A Sum w.

```

```

[5]    →(2*⌊FC 'Δa')/MMM      a If no α, go do the monadic case.
[6]    ΔA++/⌊ENLIST Δa        a Sum α.
[7]    ΔZ+ΔA ÷ ΔW              a + the sums together.
[8]    +ZZZ
[9]
[10]   MMM:                     a MONADIC CASE:
[11]   ΔZ+ ÷ ΔW                 a Just + the sum of w.
[12]
[13]   ZZZ:
[14]
[15]   a THIS FN WAS GENERATED FROM KK_voof VIA vderive_fn
      v

```

Performance

In the migrated applications, defined operators were used widely. Some of these uses were nested, meaning that they were coded within the statements which comprised the body of another defined operator. Moreover, some nested operators were used several times within the body of a given defined operator.

These repetitions made it convenient to have the outermost defined operator pre-generate some of its dependents' derived functions and invoke them directly. By doing this, duplicate tours through the generation process were eliminated, which helped to reduce overhead.

Even so, some performance bottlenecks were still attributable to some defined operator simulations. They were cleared by pre-generating all the relevant derived functions as global functions, saving them in the workspace, and recasting all the defined operator calls to reference the derived functions instead.

Where performance was satisfactory, defined operator simulations were left to continue generating their derived functions when required.

Conclusions

The absence of defined operators from the repertoire of the APL*PLUS II/386 interpreter is no barrier to the migration of mainframe APL2 applications. Just write a simple frontend function, recode the defined operator in the form of a template, and point the deriver at it. A little editing of the defined operator invocations is all that is then required.

Reference

[FIDO 1993] When my friend Doreen brought her dog round.

Inner Products for Range Tests

by Jonathan Barman

At our working group meeting the other day the topic came up of the rather delightful inner products that one can use to check ranges. It seemed a good idea to remind everyone what they are and how they may be derived.

The basic range test is to check if a scalar a lies within the range given by a two element vector b where $b[1]$ is less than $b[2]$. Spelling this out in full gives $(a \geq b[1]) \wedge (a \leq b[2])$. The steps needed to turn this into an inner product are as follows:

$(a \geq b[1]) \wedge (a \leq b[2])$	
$(\neg a < b[1]) \wedge (a \leq b[2])$	$\mathbf{A} \quad a \geq w \leftrightarrow \neg a < w$
$(a < b[1]) < (a \leq b[2])$	$\mathbf{A} \quad (\neg a) \wedge w \leftrightarrow a < w$
$(a < b[1]) < (a < b[2] + 1)$	$\mathbf{A} \quad a \leq w \leftrightarrow a < w + 1$ <i>Only for integers!</i>
$\neg a < b + 0 \ 1$	$\mathbf{A}$ <i>scalar extension</i>
$a < . < b + 0 \ 1$	$\mathbf{A}$ <i>definition of inner product</i>

This is the one that everyone uses, because it looks so pretty! However, there is a total of four ways in which the inner product can be derived:

$a < . < b + 0 \ 1 \quad a < . \leq b - 1 \ 0 \quad a > . > b - 1 \ 0 \quad a > . \geq b + 0 \ 1$

If you want to check that a is outside the range b then there are four more inner products, which are:

$a \leq . \geq b + 0 \ 1 \quad a \leq . > b - 1 \ 0 \quad a \geq . \leq b - 1 \ 0 \quad a \geq . < b + 0 \ 1$

It is quite unusual to see any of these. I don't think I have ever used them, but if I did I would use the first one as it is the strict negation of $a < . < b + 0 \ 1$.

The above inner products only work for a scalar integer against a two element range. If you have a vector of integers which you want to check against a number of ranges, then you have to use the outer product $\neg a \circ . < b + (\rho b) \rho \ 0 \ 1$.

Checking for overlapping ranges is almost exactly the same, only you have to reverse one of the ranges. Briefly going through the derivation again:

$(a[2] \geq b[1]) \wedge (a[1] \leq b[2])$
 $(\neg a[2] < b[1]) \wedge (a[1] < b[2] + 1)$
 $(a[2] < b[1]) < (a[1] < b[2] + 1)$
 $(\phi a) < . < b + 0 \ 1$

I normally switch this round and use $a > . < \phi b + 0 \ 1$ so I don't have to use parentheses. This form is also convenient when a is a two column matrix of ranges and b is a two element vector.

As one can reverse either the left or the right argument for sets of ranges there are 8 possible inner products:

$$\begin{array}{cccc} a > . < \phi b + 0 \ 1 & a > . \leq \phi b - 1 \ 0 & a < . > \phi b - 1 \ 0 & a < . \geq \phi b + 0 \ 1 \\ (\phi a) < . < b + 0 \ 1 & (\phi a) < . \leq b - 1 \ 0 & (\phi a) > . > b - 1 \ 0 & (\phi a) > . \geq b + 0 \ 1 \end{array}$$

The inverses, for checking that the ranges do not overlap are:

$$\begin{array}{cccc} a \geq . \geq \phi b + 0 \ 1 & a \geq . > \phi b - 1 \ 0 & a \leq . \leq \phi b - 1 \ 0 & a \leq . < \phi b + 0 \ 1 \\ (\phi a) \leq . \geq b + 0 \ 1 & (\phi a) \leq . > b - 1 \ 0 & (\phi a) \geq . \leq b - 1 \ 0 & (\phi a) \geq . < b + 0 \ 1 \end{array}$$

I normally use $a > . < \phi b + 0 \ 1$ and forget about the rest.

It is important to remember that all the numbers must be integers for these inner products to work. If you are testing floating point numbers, then the basic tests have to be used. Also, an inner product is not necessarily faster than the expanded test. It all depends on the APL interpreter that you are using.

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ

**REDUCED
PRICE**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Now Available!

IBM® APL2™ for OS/2™

APL2 for OS/2 provides the full APL2 language, APs, and tools to develop applications that are portable among OS/2, VM, TSO, AIX/6000 and Sun Solaris. "APL2/2" is available in two editions:

► **APL2/2 Entry Edition:** \$185 (U.S.)

Graphical user interface • Multiple concurrent APL2 sessions • ☐ *NA* calls to routines written in other languages • Graphics facilities • Cross-session shared variables • Output graphics and text to any OS/2-supported printer • Extensive printed *and* online documentation • Application development workspaces • Workspace sizes up to 250MB • Presentation Manager interface

► **APL2/2 Advanced Edition:** \$650 (U.S.)

Includes all of the features of APL2/2 Entry Edition, *plus*:

- Access to DB2/2 relational databases • Remote session managers • Cross-system shared variables, for network cooperative processing • TCP/IP socket interface
- Interfaces and tools for writing custom APs
- Integrated application performance analysis

Ordering: Outside the U.S., please contact your local IBM Branch Office. Within the U.S., please call IBM Direct at 1-800-IBM-CALL (1-800-426-2255). Ask for part number 89G1556 (APL2/2 Entry Edition) or part number 89G1697 (APL2/2 Advanced Edition).

...Want more details? • Internet: apl2@vnet.ibm.com
• CompuServe: go IBMAPL2 • Fax: (U.S.) 1-408-463-4488
• Tel: (U.S.) 1-408-463-APL2 (1-408-463-2752)

IBM, APL2, AIX, AIX/6000, Operating System/2, OS/2, DB2, DB2/2, VM and TSO are trademarks of the International Business Machines Corporation. Sun and Solaris are trademarks of Sun Microsystems, Inc. CompuServe is a trademark of CompuServe, Inc. Prices shown are valid for all U.S. customers, as of March 1, 1994.



Another Use of Not-Equals-Scan

by Anthony Camacho

Donald McIntyre and I wanted to put together a disk of all his J writings. He regretted that he did not have disk versions of his APL 91 paper *Mastering J* or his article for the IBM Systems Journal *Language as an Intellectual Tool: from Hieroglyphics to APL*. I said I had a Logitech Scanman color and CorelDraw 4 which has an OCR-trace facility and I would see what they could do.

I did, and found that the scanner would produce a black and white bitmap which CorelTrace would read. CorelTrace produces an encapsulated PostScript file as scanner output. It uses the same filename as the file loaded and writes a EPS file that begins thus:

```
%!PS-Adobe-2.0 EPSF-1.2
%%Creator: CorelTRACE V4.0 in Adobe Illustrator 88(TM) format
%%Title: CorelTRACE Output
%%BoundingBox: 0.0 0.0 178.0 617.7
%%TileBox: 0.0 0.0 178.0 617.7
%%CreationDate: Thurs Sept 10 14:01:30 1992
```

When I looked at the rest of the file I was most impressed. I thought the fact that it specified the typeface as *Helvetica* meant that it recognised the font. Later I discovered that its output is *always* Helvetica. It doesn't even distinguish between roman and italic or bold. Unfortunately I have no software that will read a .EPS file and extract the text from it. CorelDraw itself will import the .EPS file, but it comes in as a group of objects which cannot be edited until they are ungrouped. Each object is one line of text and I have not yet found any way to join them into a single string.

Adrian Smith suggested to me that, in PostScript, every text character will be in parentheses. It appears to be true although there are characters which are not text in parentheses too. The PostScript specification (*PostScript Language Reference Manual*, Addison- Wesley ISBN 0-201-10174-2) says that:

"A string in POSTSCRIPT is delimited by balanced parentheses... (Strings may include special characters *-&^% and balanced parentheses () (and so on)). . Within a string the '\ ' (backslash) character is used as an 'escape' for various purposes such as including unbalanced parentheses, non-printing characters and the '\ ' character itself."

I thought that I would be able to write some APL to read the .EPS file and write the ASCII text to another file quite easily. I read the file into a text vector. I relied

Now Available!

IBM® APL2™ for Sun™ SPARC® Systems

APL2 for Sun Solaris provides the full APL2 language, APs, and tools to develop applications that are portable among Sun Solaris, VM, TSO, AIX/6000 and OS/2.

APL2 for Sun Solaris features:

- Graphical user interface using Motif
- Multiple concurrent APL2 sessions
- ⌈NA calls to routines written in other languages
- Graphics facilities
- PostScript support for both text and graphics
- Extensive documentation
- Application development workspaces
- Workspace sizes up to 1GB
- X-Windows API
- Remote session managers
- Cross-system shared variables, for network cooperative processing
- TCP/IP socket interface
- Interfaces and tools for writing custom APs
- Integrated application performance analysis
- National Language Support

Prerequisites: Solaris 2.2 or later and Motif 1.2 (X11R5) or later.

Ordering: Outside the U.S., please contact your local IBM Branch Office. Within the U.S., please call IBM Direct at 1-800-IBM-CALL (1-800-426-2255). Ask for product number 5648-065. **Price:** \$1500 (U.S.). Two-month test period—satisfaction guaranteed!

- ...Want more details?**
- Internet: apl2@vnet.ibm.com
 - CompuServe: go **IBMAPL2**
 - Fax: (U.S.) 1-408-463-4488
 - Tel: (U.S.) 1-408-463-APL2 (1-408-463-2752)

IBM, APL2, AIX, AIX/6000, Operating System/2, OS/2, VM and TSO are trademarks of the International Business Machines Corporation. Sun and Solaris are trademarks of Sun Microsystems, Inc. SPARC is a trademark of SPARC International, Inc. Motif is a trademark of the Open Software Foundation. PostScript is a trademark of Adobe Systems, Inc. CompuServe is a trademark of CompuServe, Inc. Prices shown are valid for all U.S. customers, as of March 1, 1994.



on the parentheses being balanced and did a not-equals scan on a boolean that marked them, to mark all the text within the parentheses. (Gérard would be proud of me.) It didn't work because CorelTrace does not mark the unbalanced parentheses. I suppose it may assume that unbalanced parentheses are probably the result of scan failure. I wish it had marked every parenthesis within the text as unbalanced. That would have been within the specification and much easier to deal with. My reading of the specification is that CorelTrace does not meet it.

"Surely I can cope with that.", I thought. I then wasted a day trying to find elegant methods. In particular I felt sure that subtracting a plus-scan of a boolean marking the closing parentheses from a plus-scan of a boolean marking the opening parentheses ought to be the right start. It gives a numeric vector where the values rise as each opening parenthesis is passed and reduce at each closing parenthesis. The trouble is that unmatched parentheses within the text ensure that there are non-zero values for characters outside the text strings. I couldn't find a good way to reduce the values for the non-text characters reliably to zero.

Instead I tried another tack which appears to work. Below is the result. I have commented it enough for anyone to follow its working and have not saved time or space by combining lines which ought to be combined. It starts by collecting the positions of all the left and right parentheses separately. It aims to find each opening parenthesis, and remove any other opening parenthesis before the next closing parenthesis, and then to find each closing parenthesis and remove any preceding closing parenthesis closer than the next opening parenthesis to the left. I realise this really ought to be done recursively until there are no unbalanced parentheses to remove, but I chose instead to put in a test to see if the result is balanced and I've now scanned quite a few pages and never had a failure.

I had to recognise the carriage returns, which CorelTrace always renders as (`\r`), and remove the `()` pairs if any which would cause the not-equals scan to flip and mark the non-text characters from `()` to the next `()`. It is written in APLIWIN and is more or less instantaneous on my 486 given the that largest file it can process is one page, or one column.

To my delight CorelTrace recognises most italic text almost as well as roman. It seems to be quite tolerant of noise. I have scanned dirty newsprint successfully. Of course there are severe limitations. It fails on a lot of Amstrad word processor print. It will not recognise subscripts or superscripts — the latter come out as quotes. It doesn't recognise APL, however beautifully printed, or the sort of mathematical symbols Donald McIntyre is liable to use such as square root, infinity or sigma. But it does quite often read a paragraph without a single erroneous character.

New Release!

IBM® APL2™ for the RISC System/6000™

APL2 for the RISC System/6000 provides the full APL2 language, APs, and tools to develop applications that are portable among AIX/6000, VM, TSO, Sun Solaris, and OS/2.

Now Available in Release 2 of AIX APL2/6000:

New Motif-based session manager • $\square$ NA calls to routines written in other languages • Access to DB2/6000 relational databases • Remote session managers • Cross-system shared variables, for network cooperative processing • TCP/IP socket interface • Interfaces and tools for writing custom APs • Pipe support • PostScript support for both text and graphics • Asynchronous APs for multiprocessing • National Language and APL2 character support for the vi editor

Prerequisites: AIX 3.2 or later and AIXwindows Environment/6000 Version 1.2.3 or later.

Ordering: Outside the U.S., please contact your local IBM Branch Office. Within the U.S., please call IBM Direct at 1-800-IBM-CALL (1-800-426-2255). Ask for product number 5765-012.

Prices are based upon processor size, starting at just \$1050 (U.S.). Two-month test period—satisfaction guaranteed! Upgrades to this release are *free* to registered owners of APL2/6000 Release 1.

...Want more details? • Internet: apl2@vnet.ibm.com
• CompuServe: go IBMAPL2 • Fax: (U.S.) 1-408-463-4488
• Tel: (U.S.) 1-408-463-APL2 (1-408-463-2752)

IBM, APL2, APL2/6000, AIX, AIX/6000, RISC System/6000, AIXwindows, Operating System/2, OS/2, DB2, DB2/6000, VM and TSO are trademarks of the International Business Machines Corporation. Sun and Solaris are trademarks of Sun Microsystems, Inc. PostScript is a trademark of Adobe Systems, Inc. CompuServe is a trademark of CompuServe, Inc. Prices shown are valid for all U.S. customers, as of March 1, 1994.



Appendix: the Code

```

outfile extractascii from infile; tv; boo; posl; posr; mat; boomat; posrn;
                                poslp; adj; posn

[1]  * Extracts text from CorelTrace .EPS infile and writes it to outfile
[2]  tv+[]toapl []hfreed infile
[3]  posl+{tv='(')/\ptv *      positions of left parens
[4]  posr+{tv=')')/\ptv *      positions of right parens
[5]  mat+posr+. [posl *        larger of each posr and each posl
[6]  boomat+((pmat)pposl)>mat * 1 marks mat positions not to right of posl
[7]  posrn+[/mat+boomat*ptv *  select smallest posr to right of each posl
[8]  posl+(nubsieve posrn)/posl * pick leftmost posl for each posrn
[9]  mat+posl+. [posr *        smaller of each new posl and each posr
[10] boomat+((pmat)pposr)<mat * 1 marks mat positions not to left of posr
[11] poslp+[/mat-boomat*2*ptv * select largest posl to left of each posr
[12] posr+[(nubsieve poslp)/posr * pick rightmost posr for each poslp
[13] *      There ought to be as many posr entries as there are posl and each
[14] *      posr ought to be larger than the corresponding posl. So test!
[15] +((pposl)*pposr)/err *    if they aren't the same length
[16] +(~^/posl+posr)/err *     if every posr is not larger than its posl
[17] adj+~posr=i+posl *         adj keeps all but adjacent ( )
[18] posr+adj/posr o posl+adj/posl * remove adjacent pairs
[19] boo+(ptv)o0 *              make an empty boolean
[20] boo[posr,posl]+1 *         put 1s in it
[21] boo+~\boo * \ *          switch on bits between ( and ) (guaranteed in pairs)
[22] 'There are ',(#+/boo),' characters between parentheses.'
[23] posn+boo/\boo *           note start of each string in posn
[24] 'There are ',(#+posn),' strings of letters to extract.'
[25] posn+(('\ '=tv[posn+1])^'r'=tv[posn+2])/posn
[26] *      set tv to CR LF where group begins '\r'
[27] tv[posn+1]+CR o tv[posn+2]+LF
[28] 'There are '\r' pairs at positions ',#+posn
[29] tv+(boo^0,~1+boo)/tv *     select the characters required
[30] ([]toascii tv) []hfwrite outfile
[31] -o
[32] err:'It went wrong! pposl= ',(#+pposl),' and pposr= ',(#+pposr)
[33] 'posl:',(#+posl) o 'posr:',(#+posr)

```

Vectype: Preparing APL Code for Publication (Vector and APL94)

by Adrian Smith (Production Manager)

Vector has the interesting technical problem that it must cope with APL code arriving in all sorts of different character sets. For a long time, I could work with the APL*PLUS/PC $\square AV$ as a de facto standard, but all that has changed now, and code arrives in *Word* and *Write* documents having been pasted over from Dyalog/W, ISIWIN, and (shortly) APL*PLUS III. Commonality of $\square AV$ is not relevant any more, because everyone maps their characters to their TrueType fonts in a unique way. What Vector needs is some way of:

- a) encouraging as many people as possible to submit articles in my favourite VectorAPL TrueType font (so I don't need to do anything).
- b) coping easily with submissions in the native font of the APL of your choice. This looks to me (unfortunately) to be the most likely scenario.

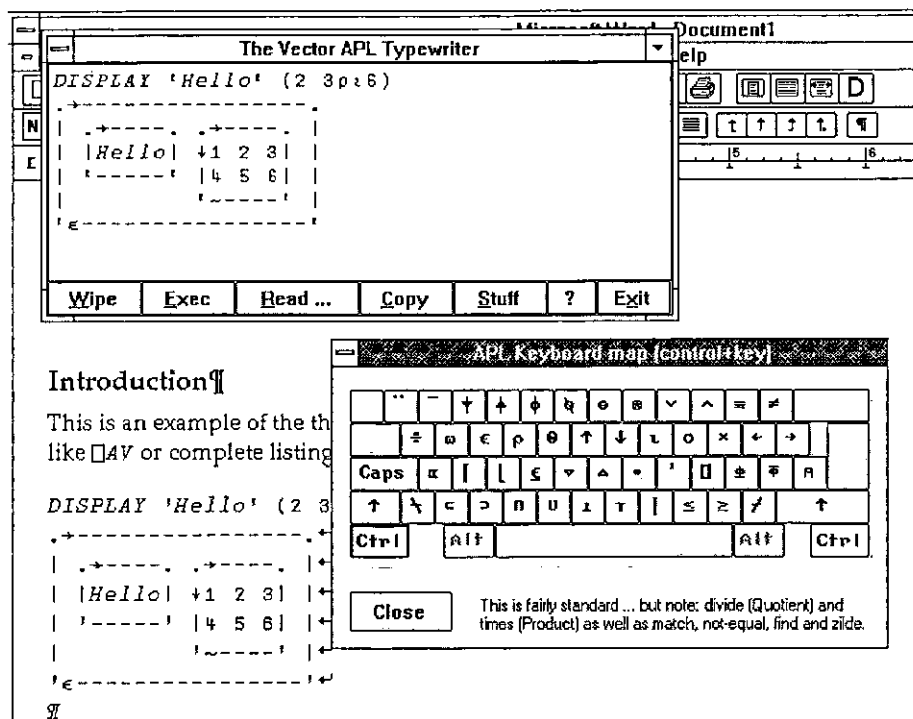
It seems to me that if *Vector* can solve this problem, the solution is likely to be of general value to the APL community. In particular, what we can offer is an APL font that includes all the APL symbols known to man, and a unified keyboard layout that you can type them with!

The Current State of the Art

The screen-snap on the facing page shows *VECTYPE* in action, and illustrates most of what it does. Basically, it is a runtime Dyalog APL workspace (no royalties necessary) which makes a simple 'ontop' window so that you can run Word full-screen and still see your APL typebox. When you 'minimize' it, it rolls up to its titlebar, which is easy to park over the Word titlebar, and much more convenient than an 'ontop' icon!

You can simply type APL text into the box (the key mappings are as standard as I could make them, given that this covers all known APLs) or you can execute simple expressions (the workspace includes some common utilities like *DISPLAY*) and have the results shown below. Normal cut-and-paste works as you would expect, to any Windows application, or you can use the <Copy> button to put the contents of the whole Vectype window into the clipboard. When you paste this (e.g. into Write), you will need to select it and set the *VectorAPL* font by hand.

Vectype will import APL code from DOS text files in any of the common $\square$ AVs (many thanks to David Appleton for checking out the APL2), and will read ISIAPL, Dyalog APL and APL*PLUS III from the Clipboard. This enables me to grab code from a Winword document in (say) ISIAPL format into the Vectype Window, and promptly stuff it back in the VectorAPL font! I feel I shall want something like this to move code easily from Dyalog/W to PLUS III and back!



If you use Winword, you can take advantage of the <Stuff> button! This sets up a DDE channel and sends the entire contents of the Vectype window directly to Word, where it is inserted at the cursor. If there is only a single line in Vectype, it simply selects it and sets the VectorAPL font for you; if there are several lines, these are entered as a separate paragraph, with 'newline' characters at the end of each line. Again, the VectorAPL font is set automatically.

If you can make use of this, it certainly helps me to get Vector together in less time with less hassle! Send me a blank 1.44Mb disk with return postage, or mail me on Compuserve 100331,644 and I will reply with a copy of Vectype and a runtime Dyalog interpreter ready to install. Then, give it away to your friends!

Index to Advertisers

Dyadic Systems Ltd	2
IBM	135, 137, 139
Manugistics	6
MicroAPL	56
Vector Back Numbers	134

All queries regarding advertising in VECTOR should be made to Gill Smith, at 0439-788385.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Anthony Camacho,
11 Auburn Road, Redland,
BRISTOL, BS6 6LS
Tel: 0272-730036
Email:

Authors wishing to use Windows Write should contact Vector Production for a copy of the Vector APL TrueType font and Vector APL typebox.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK.

Tel: 0439-788385 (any time)
Compuserve: 100331,644.

BAA: Membership Application / Renewal

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____

Address: _____

Post or zip code: _____

Country: _____

Telephone Number: _____

Membership category (please tick box):

UK private membership	£12	☐
Overseas private membership	£14	☐
Airmail supplement (not needed for Europe)	£4	☐
UK Corporate membership	£100	☐
Corporate membership overseas	£135	☐
Sustaining membership	£430	☐
Non-voting UK member (student/OAP/unemployed only)	£6	☐

PAYMENT — in Sterling only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard or Visa number.

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____|____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

(please tick the required option above)

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON, E3 5HU

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1993/94 Committee

Chairman:	David Eastwood 071-922 8866 microapl@applelink.apple.com	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Duncan Pearson 0904-603510 100265.1564@compuserve.com	143 Hull Road, YORK YO1 3JX
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Anthony Camacho 0272-730036 acamacho@cix.compulink.co.uk	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Activities:	Vacant Post	
Education:	Dr Alan Mayer 0792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Jonathan Barman 0488-648575 100116.1030@compuserve.com	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Projects:	George MacLeod 0442-878065	Grey mantle Associates Ltd., Bartrum House, Ravens Lane, BERKHAMSTED, Herts HP4 2DY
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Mark Harris 0438-310155 kestrel@apl.demon.co.uk	Kestrel Consulting, Business & Technology Centre, Bessemer Drive, STEVENAGE, Herts SG1 2DX
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Anthony Camacho	0272-730036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (0439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (0439-788385)
Support Team:	Jonathan & Bridget Barman (0488-648575), Ray Cannon (0252-874697), Richard Weber (0302-539761), Sylvia Camacho, Duncan Pearson, John Searle, David Ziemann (071-267 8032), Jon Sandles	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Compass RAD Ltd
15 Frederick Sanger Road
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 0483 302249
Fax: 0483 302279

HMW Trading Systems Ltd
Hamilton House
1 Temple Avenue
LONDON EC4V 0HA
Tel: 071 359 4212
Fax: 071 359 3325

Solton Associates Ltd
Groot Blarckenberg 53
1032 AC Amsterdam
Netherlands
Tel: +31 20 570 8733
Fax: +31 20 570 8753
Email: jfr@solton.com

Manugistics
2415 East Jefferson St
Rockville
MARYLAND 20852 USA
Tel: (301) 984 5412
Fax: (301) 984 5094

Insight Systems ApS
Nordre Strandvej 116A
DK-9160 Hellebæk
Denmark
Tel: +45 42 10 70 22
Fax: +45 42 10 75 74

Dyadic Systems Ltd
Riverside View, Basing Road
Old Basing, BASINGSTOKE
Hants. RG24 0AL
Tel: 0256 811125
Fax: 0256 811130

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 071 922 8886
Fax: 071 928 1008

Kestrel Consulting
Business & Technology Centre
Bessener Drive
STEVENAGE, Herts SG1 2DX
Tel: 0438 310155
Fax: 0438 310161
Email: kestrel@aplidenon.co.uk

Dutch APL Association
Postbus 1631
6430 BH Nieuwegein
Netherlands
Tel: 0327 42337